ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP

PHÂN TÍCH DỮ LIỆU SCADA THỜI GIAN THỰC BẰNG AI ĐỂ ĐƯA RA CẢNH BÁO SỚM

NGÀNH: KỸ THUẬT MÁY TÍNH

HỘI ĐỒNG: 5L KỸ THUẬT MÁY TÍNH

GVHD: TS. Võ Tuấn Bình

—o0o—

SVTH1: Nguyễn Đại Vinh – 2053589
SVTH2: Phạm Viết Huy – 2211269
SVTH3: Nguyễn Thành Đạt – 2210700

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 8 NĂM 2026

Lời cam đoan

Nhóm chúng em gồm Nguyễn Đại Vinh, Phạm Viết Huy và Nguyễn Thành Đạt xin cam đoan rằng đồ án "Phân tích dữ liệu SCADA thời gian thực bằng AI để đưa ra cảnh báo sớm" là kết quả nghiên cứu, thiết kế và triển khai của nhóm dưới sự hướng dẫn của thầy Võ Tuấn Bình tại Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh.

Các nội dung, số liệu thực nghiệm, kết quả đánh giá thuật toán và mã nguồn phần mềm/phần cứng được trình bày trong báo cáo là trung thực và phản ánh đúng quá trình thực hiện của nhóm. Những tài liệu, công trình và nguồn tham khảo được sử dụng trong quá trình nghiên cứu đều đã được trích dẫn đầy đủ theo đúng quy định học thuật.

Nhóm chúng em chịu trách nhiệm về tính trung thực của nội dung báo cáo, nếu có sai phạm về sao chép hoặc vi phạm liêm chính học thuật, chúng em xin chịu trách nhiệm theo quy định của nhà trường.

Thành phố Hồ Chí Minh, Tháng 8 năm 2026

Nhóm sinh viên thực hiện

Nguyễn Đại Vinh – 2053589
Phạm Viết Huy – 2211269
Nguyễn Thành Đạt – 2210700

Lời cảm ơn

Trước hết, nhóm chúng em xin gửi lời cảm ơn chân thành đến Ban Giám hiệu Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh, cùng quý thầy cô Khoa Khoa học và Kỹ thuật Máy tính đã trang bị kiến thức nền tảng và tạo điều kiện thuận lợi để chúng em học tập, nghiên cứu và thực hiện đồ án tốt nghiệp.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Võ Tuấn Bình, giảng viên hướng dẫn đề tài "Phân tích dữ liệu SCADA thời gian thực bằng AI để đưa ra cảnh báo sớm". Sự định hướng và những góp ý của thầy đã giúp nhóm xác định rõ bài toán nghiên cứu, lựa chọn phương pháp tiếp cận và từng bước hoàn thiện hệ thống.

Nhóm cũng xin cảm ơn gia đình và bạn bè đã động viên, chia sẻ và hỗ trợ chúng em trong suốt quá trình học tập cũng như thực hiện đề tài. Những sự hỗ trợ đó là nguồn khích lệ để các thành viên phối hợp, vượt qua khó khăn trong quá trình triển khai và hoàn thành báo cáo.

Mặc dù đã nỗ lực trong quá trình nghiên cứu và kiểm thử, do giới hạn về thời gian, kinh nghiệm thực tiễn và điều kiện tiếp cận hệ thống SCADA ngoài hiện trường, đồ án vẫn còn những thiếu sót. Nhóm chúng em mong nhận được những nhận xét và góp ý từ quý thầy cô, Hội đồng đánh giá để tiếp tục hoàn thiện phương pháp nghiên cứu, nâng cao độ tin cậy của hệ thống và phát triển đề tài trong tương lai.

Nhóm chúng em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện

Nguyễn Đại Vinh – 2053589
Phạm Viết Huy – 2211269
Nguyễn Thành Đạt – 2210700

Mục lục

Lời cam đoan **2**

Lời cảm ơn **3**

1 Tổng quan đề tài **12**
1.1 Bối cảnh và tính cấp thiết của đề tài 12
1.2 Hạn chế của kiến trúc SCADA truyền thống 13
1.2.1 Mô hình lưu lượng băng thông mạng 13
1.2.2 Mô hình thời gian phản ứng sự cố 13
1.3 Giải pháp đề xuất 13
1.3.1 Đánh giá tính ưu việt của giải pháp thông qua đối chiếu kiến trúc . 16
1.4 Mục tiêu, đối tượng và phạm vi nghiên cứu 17
1.4.1 Mục tiêu nghiên cứu 17
1.4.2 Đối tượng nghiên cứu 18
1.4.3 Phạm vi nghiên cứu 18
1.5 Bố cục của đồ án 20

2 Cơ sở lý thuyết **21**
2.1 Mô hình khí động học và đường cong công suất tua-bin gió 21
2.2 Nền tảng toán học của thuật toán XGBoost 22
2.2.1 So sánh các phương pháp tìm điểm phân nhánh 24
2.3 Biến đổi lượng giác cho dữ liệu thời gian 25
2.4 Nguyên lý phát hiện bất thường dựa trên quy tắc thống kê ba Sigma . . . 27
2.4.1 Xác lập ngưỡng cảnh báo 29
2.4.2 Mô hình xác định sai lệch phục vụ giám sát vận hành 31
2.5 Cơ chế lưu trữ và nén dữ liệu chuỗi thời gian 31
2.5.1 Cấu trúc động cơ Time-Structured Merge Tree 31
2.5.2 Các giải thuật nén dữ liệu chuỗi thời gian 31
2.6 Giao thức truyền thông SMTP/TLS và bảo mật thư điện tử 34
2.6.1 TLS trên cổng 587 34
2.6.2 Trình tự SMTP STARTTLS ở mức giao thức 34
2.6.3 Bản chất của App Password 35
2.6.4 So sánh App Password và OAuth 2.0 36

3 Thiết kế kiến trúc hệ thống SCADA Edge **38**
3.1 Kiến trúc phân tầng hệ thống: Biên, Lưu trữ và Giám sát 38
3.1.1 Phân bổ chức năng trên thiết bị biên 40
3.1.2 Sơ đồ tuần tự của luồng dữ liệu 41
3.2 Phân tích phần cứng biên Orange Pi 5 Plus 42
3.3 Container hóa hạ tầng bằng Docker Compose và phân bổ tài nguyên tĩnh . 45
3.3.1 Cơ chế hoạt động của ảo hóa mức hệ điều hành 45
3.3.2 Đặc thù triển khai trên thiết bị tính toán biên 46
3.3.3 Kế hoạch lưu trữ dữ liệu 46
3.3.4 Đặc tả triển khai các dịch vụ bằng Docker Compose 47
3.3.5 Tối ưu hóa tài nguyên Docker trên thiết bị biên 47
3.4 Thiết kế mạng bridge cô lập scada_net và cơ chế phân giải DNS nội bộ . . 49
3.5 Bảo mật và quản lý thông tin xác thực qua biến môi trường 50

4 Hiện thực luồng dữ liệu thời gian thực và mô hình AI tại biên **53**
4.1 Hiện thực pipeline tiền xử lý dữ liệu 53
4.1.1 Đặc tả tệp dữ liệu lưu trữ 53
4.1.2 Các đặc trưng sử dụng trong suy luận 54
4.1.3 Xử lý đặc trưng chu kỳ thời gian 56
4.1.4 Tiền xử lý nhiễu hệ thống 56
4.1.5 Chuẩn hóa và phân chia dữ liệu theo thời gian 57
4.1.6 Pipeline tiền xử lý tổng thể 58
4.2 Huấn luyện, tối ưu tham số và cơ chế dừng sớm cho XGBoost 58
4.2.1 Bộ tham số được sử dụng trong mã nguồn 58
4.2.2 Cơ chế early stopping 59
4.2.3 Các độ đo hồi quy 60
4.3 Thiết kế cấu trúc thư mục và đóng gói artifact 61
4.3.1 Thiết lập mô hình và các thành phần cần thiết 62
4.4 Truyền phát dữ liệu thời gian thực và suy luận biên tức thời 63
4.4.1 Sơ đồ khối kiến trúc luồng truyền phát dữ liệu 63
4.4.2 Chuyển đổi dữ liệu lịch sử thành luồng 64
4.4.3 Chuỗi xử lý suy luận tại biên 66
4.5 Thiết kế lược đồ chuỗi thời gian InfluxDB v2 và kiểm chứng dữ liệu 67
4.5.1 Phân tích dữ liệu 67
4.5.2 Mô hình hóa độ biến thiên chuỗi thời gian 68
4.5.3 Ghi dữ liệu kiểm thử qua giao thức dòng 68

4.6 Cơ chế giảm tần suất mẫu tự động bằng ngôn ngữ Flux 70

5 Hệ thống giám sát, thiết lập cảnh báo và thông báo sự cố. 72
5.1 Xây dựng bảng điều khiển Grafana thời gian thực 72
5.1.1 Cấu hình nguồn dữ liệu trên Grafana 72
5.1.2 Khung bên trái: Độ lệch công suất 73
5.1.3 Công suất thực tế và công suất dự báo 73
5.1.4 Cấu hình biến động nội suy và đặc tả bảng điều khiển 74
5.2 Thiết kế quy tắc cảnh báo tự động theo cấu trúc đánh giá A–B–C 76
5.2.1 Khối A: truy vấn residual bằng Flux 77
5.2.2 Khối B: rút gọn bằng Last 78
5.2.3 Khối C: so sánh với ngưỡng ba Sigma 79
5.2.4 Thiết lập dữ liệu cảnh báo 79
5.3 Vận hành máy trạng thái và kỹ thuật chống rung cảnh báo 80
5.3.1 Vòng đời của một alert instance 80
5.3.2 Trạng thái No Data và Error 81
5.3.3 Kỹ thuật giảm rung cảnh báo 81
5.4 Tích hợp giao thức SMTP STARTTLS qua cổng 587 và xác thực Google App Password 84
5.4.1 Vai trò của SMTP 84
5.4.2 Nguyên tắc quản lý secret 86
5.4.3 Phân tích các biến môi trường 86
5.5 Thiết kế Notification Policy và tùy biến giao diện email bằng ngôn ngữ Go Template 87
5.5.1 Khởi tạo Contact Point 87
5.5.2 Notification Policy 88
5.5.3 Tùy biến nội dung email bằng Go Template 90

6 Triển khai thực nghiệm, đánh giá kết quả và hướng phát triển 92
6.1 Quy trình chuyển giao và vận hành trên nền tảng nhúng Ubuntu 24.04 ARM64 92
6.1.1 Ubuntu 24.04 trên kiến trúc aarch64 92
6.1.2 Gán địa chỉ ổn định bằng DHCP reservation 94
6.1.3 Điều khiển từ xa bằng SSH và Visual Studio Code 96
6.1.4 Cài đặt Docker Engine trên Orange Pi 97
6.1.5 Phân tích Dockerfile của AI exporter 98

6.1.6 Khởi tạo InfluxDB v2 . . . 99
6.1.7 Dashboard Grafana trên Orange Pi . . . 99
6.1.8 Mức độ hoàn thành . . . 101
6.2 Kiểm chứng luồng cảnh báo khép kín End-to-End . . . 101
6.2.1 Đánh giá hiệu năng mô hình . . . 101
6.2.2 Phân tích số liệu thực tế trên Grafana . . . 103
6.2.3 Kịch bản kiểm thử định lượng . . . 107
6.2.4 Trường hợp 1: Sự cố xảy ra và email cảnh báo FIRING . . . 108
6.2.5 Trường hợp 2: Hệ thống được phục hồi và email RESOLVED . . . 109
6.2.6 Dòng thời gian khép kín . . . 110
6.2.7 Kiểm thử và tiêu chí hoàn thành . . . 111
6.3 Thử nghiệm tính bền vững dữ liệu và phục hồi sự cố . . . 112
6.3.1 Kiểm thử khả năng tự phục hồi . . . 112
6.3.2 Xác thực độ bền vững của dữ liệu . . . 113
6.4 Phân tích các hạn chế kỹ thuật hiện tại và giải pháp khắc phục . . . 114
6.4.1 Phân tích underfitting và hướng giải quyết . . . 114
6.4.2 Khóa dependency và xung đột estimator tags . . . 115
6.4.3 Docker Compose, volume và mạng dịch vụ . . . 117
6.4.4 Khởi động lại và kiểm tra . . . 117
6.4.5 So sánh máy tính cá nhân và Orange Pi . . . 118
6.5 Định hướng phát triển hệ thống trong tương lai . . . 118
6.5.1 Phân tích độ phức tạp và khả năng mở rộng . . . 119

Danh mục hình

1.1 Cấu trúc cảm biến và luồng thu thập dữ liệu 12
1.2 Lưu đồ thuật toán giám sát và cảnh báo bất thường tại Edge Gateway . . 15
1.3 Sơ đồ đối chiếu kiến trúc và luồng xử lý dữ liệu giữa mô hình SCADA truyền thống (a) và mô hình tích hợp Edge AI (b) 16
1.4 Phân loại và định tuyến payload sau quá trình xử lý tại thiết bị biên . . . 16
2.1 Đường cong công suất lý tưởng với bốn vùng vận hành của tua-bin gió . . 22
2.2 Lưu đồ huấn luyện XGBoost với phương pháp histogram 25
2.3 Mã hóa thời gian bằng tọa độ sin–cos 27
2.4 Ý nghĩa thống kê của vùng ba Sigma 28
2.5 Phân phối residual tuyệt đối và ngưỡng cảnh báo ba Sigma 30
2.6 Minh họa phép XOR giữa hai giá trị công suất liên tiếp trong mã hóa Gorilla 34
2.7 Trình tự nâng cấp SMTP sang TLS bằng STARTTLS 35
2.8 Tạo App Password cho Grafana SCADA, khóa đã được che trong báo cáo . 37
3.1 Kiến trúc tổng quan 39
3.2 Kiến trúc triển khai SCADA–AI trên Orange Pi 5 Plus 41
3.3 UML Sequence Diagram cho một chu kỳ suy luận và ghi dữ liệu 42
3.4 Mặt trên Orange Pi 5 Plus và các giao tiếp phần cứng phục vụ SCADA Edge 44
3.5 Phân luồng nhiệm vụ đề xuất trên kiến trúc big.LITTLE của RK3588 . . . 44
3.6 Mặt dưới bo mạch với khe M.2 M-Key dành cho SSD NVMe 45
3.7 So sánh cấu trúc phân tầng giữa máy ảo và container, Việt hóa từ tài liệu Docker[5] 46
3.8 Luồng mạng từ giao diện vật lý qua NAT tới bridge và các container . . . 50
3.9 Luồng truyền tải dữ liệu và cơ chế xác thực phân quyền giữa các thành phần 51
3.10 Luồng luân chuyển secret từ host tới tiến trình container 52
4.1 Pipeline tiền xử lý dữ liệu 58
4.2 Diễn biến RMSE validation và mốc chất lượng tốt nhất trước khi dừng sớm 60
4.3 Kiến trúc véc-tơ đặc trưng, bộ chuẩn hóa và mô hình suy luận 63
4.4 Lưu đồ tổng thể của thuật toán streaming và suy luận 64
4.5 Cơ chế phát lại dữ liệu lịch sử theo nhịp một giây 65
4.6 Phân rã một bản ghi InfluxDB Line Protocol thành bốn thành phần 69
4.7 Xác thực bản ghi dữ liệu lưu trữ thành công trên giao diện InfluxDB Data Explorer 70
4.8 Biểu đồ tuần tự của tác vụ Flux tổng hợp dữ liệu theo cửa sổ năm phút . . 71
5.1 Cấu hình thông số kết nối nguồn dữ liệu Flux trên Grafana 72

5.2 Kết quả kiểm thử kết nối nguồn dữ liệu thành công trên Grafana 73
5.3 Chuỗi đánh giá Query–Reduce–Threshold 76
5.4 Máy trạng thái cảnh báo của Grafana 80
5.5 Residual dao động quanh hai ngưỡng và tác dụng của hysteresis 83
5.6 Biểu đồ thời gian phối hợp residual, máy trạng thái và kênh SMTP 83
5.7 UML Sequence Diagram của luồng đánh giá và định tuyến cảnh báo qua email 85
5.8 Cấu hình SMTP thực nghiệm, App Password đã được che 87
5.9 Contact Point ALERT chuyển thông báo tới email kỹ sư vận hành 88
5.10 UML Activity Diagram của quá trình định tuyến và dựng thông báo 89
6.1 Xác thực Ubuntu 24.04.3 LTS, kernel Rockchip và địa chỉ IP trên Orange Pi 93
6.2 Orange Pi xuất hiện trong danh sách DHCP client tại địa chỉ 192.168.0.105 95
6.3 Truyền địa chỉ MAC với IP dành riêng cho Orange Pi 95
6.4 Visual Studio Code kết nối Orange Pi qua Remote–SSH 97
6.5 Docker Engine hoạt động trên nền tảng linux/arm64 98
6.6 Sơ đồ phân tầng kiến trúc image của AI exporter 99
6.7 Dashboard SCADA chạy trên Grafana của Orange Pi 5 Plus 100
6.8 Tương quan giữa công suất thực tế và công suất dự báo trên tập test . . . 102
6.9 Độ quan trọng của các đặc trưng theo tiêu chí gain 103
6.10 Cấu hình Alert Rule A–B–C và trạng thái Firing của tua-bin T1 104
6.11 Dashboard chuyển sang ALERTING khi residual vượt ngưỡng 106
6.12 Email FIRING được gửi đồng thời với trạng thái ALERTING trên Dashboard 108
6.13 Email RESOLVED xác nhận hệ thống trở về vùng an toàn 109
6.14 Tỷ trọng các thành phần trong độ trễ cảnh báo đầu cuối trung bình 111
6.15 Đường cong công suất thực tế và các mẫu gió mạnh–công suất thấp được bóc tách 115

Danh mục bảng

1.1 Đối chuẩn đa tiêu chí giữa kiến trúc SCADA truyền thống và SCADA Edge AI đề xuất . . . 17
1.2 Tiêu chí đánh giá nghiệm thu hệ thống . . . 19
1.3 Bố cục của đồ án . . . 20
2.1 Đặc tả kỹ thuật bốn vùng vận hành của tua-bin gió . . . 22
2.2 So sánh các phương pháp tìm điểm phân nhánh trong XGBoost . . . 25
2.3 Đối chứng khoảng cách giữa mã hóa tuyến tính và mã hóa lượng giác chu kỳ 26
2.4 Phân tích độ nhạy và xác suất báo động giả theo quy tắc bội số độ lệch chuẩn 29
2.5 Ví dụ diễn giải trạng thái theo residual . . . 30
2.6 Minh họa sai phân timestamp khi lấy mẫu đều một giây . . . 33
2.7 So sánh xác thực SMTP bằng App Password và OAuth 2.0 . . . 36
3.1 Đặc tả phần cứng Orange Pi 5 Plus phục vụ SCADA Edge . . . 43
3.2 Ma trận phân bổ tài nguyên và chính sách phục hồi container . . . 48
3.3 Quy hoạch không gian mạng và ánh xạ cổng dịch vụ . . . 49
3.4 Ma trận phân quyền mã xác thực trong hạ tầng giám sát . . . 51
3.5 Kiểm soát vòng đời thông tin xác thực . . . 52
4.1 Thống kê số lượng mẫu bị loại bỏ theo các luật nhận diện nhiễu . . . 54
4.2 Thống kê mô tả các biến trong tập dữ liệu sau làm sạch . . . 55
4.3 Hệ số tương quan Pearson đối với công suất thực tế P_{act} . . . 56
4.4 Phân chia dữ liệu huấn luyện và đánh giá . . . 58
4.5 Cấu hình XGBoost thực tế trong xgboost_train.py . . . 59
4.6 Tóm tắt kết quả quá trình huấn luyện . . . 60
4.7 So sánh MAE và RMSE trong bài toán dự báo công suất . . . 61
4.8 Phân định vai trò các module và tệp triển khai tại Edge Node . . . 62
4.9 Phân tích dữ liệu SCADA và kết quả suy luận tại biên . . . 67
4.10 Ma trận thực thi kiểm thử ghi dữ liệu qua InfluxDB Write API . . . 69
4.11 Đặc tả toán tử đường ống chuỗi thời gian bằng Flux . . . 71
5.1 Đặc tả cấu hình các thành phần giao diện trên bảng điều khiển giám sát . 75
5.2 So sánh tổng hợp cửa sổ và trung bình trượt . . . 78
5.3 Ma trận chuyển trạng thái của động cơ cảnh báo . . . 81
5.4 Đặc tả cấu hình SMTP và chính sách định tuyến thông báo . . . 86
5.5 Ánh xạ Go Template sang thông tin vận hành . . . 91
6.1 Thông tin môi trường được ghi nhận trên thiết bị . . . 94
6.2 Ma trận thực thi kiểm thử triển khai và phục hồi hệ thống . . . 96

6.3 Đối chiếu mục tiêu triển khai đồ án với minh chứng thực nghiệm 101
6.4 Kết quả đánh giá mô hình 101
6.5 Đo kiểm hiệu năng truy vấn Flux và kết xuất đồ thị Grafana 104
6.6 Cấu hình Data Source của Alert Rule 105
6.7 Ma trận kiểm thử Alert Rule 107
6.8 Tóm tắt các mốc của vòng đời cảnh báo 110
6.9 Phân rã định lượng độ trễ cảnh báo đầu cuối 110
6.10 Checklist nghiệm thu chức năng email cảnh báo 111
6.11 Tiêu chí đánh giá khả năng phục hồi sau SIGKILL 113
6.12 Phân tích nguyên nhân underfitting tại vùng gió mạnh 114
6.13 Ma trận kiểm chứng dependency cho AI exporter 116
6.14 Ma trận dịch vụ container trong cấu hình triển khai 117
6.15 Đo kiểm tài nguyên trên PC x86–64 và Orange Pi 5 Plus ARM64 118
6.16 Ước lượng quy mô dữ liệu theo số tua-bin trong cửa sổ năm phút 119

Chương 1: Tổng quan đề tài

1.1 Bối cảnh và tính cấp thiết của đề tài

Trong lĩnh vực năng lượng tái tạo, đặc biệt là điện gió, các hệ thống điều khiển giám sát và thu thập dữ liệu SCADA đóng vai trò huyết mạch trong việc thu thập dữ liệu và giám sát trạng thái hoạt động của các tua-bin. Một tua-bin gió hiện đại liên tục sinh ra lượng lớn dữ liệu chuỗi thời gian từ hàng loạt cảm biến như tốc độ gió, nhiệt độ môi trường, tốc độ rotor và công suất phát điện. Việc giám sát chặt chẽ các thông số này không chỉ giúp tối ưu hóa hiệu suất mà còn mang ý nghĩa sống còn trong việc phát hiện sớm các bất thường, từ đó ngăn chặn những hỏng hóc cơ học nghiêm trọng có thể dẫn đến chi phí bảo trì khổng lồ.

Việc giám sát hiệu suất đòi hỏi hệ thống phải thu thập các trường dữ liệu chuỗi thời gian với tần số lấy mẫu rất cao. Các thông số cốt lõi bao gồm tốc độ gió đầu vào, tốc độ quay của rotor, nhiệt độ của hộp số, và công suất phát điện thực tế.

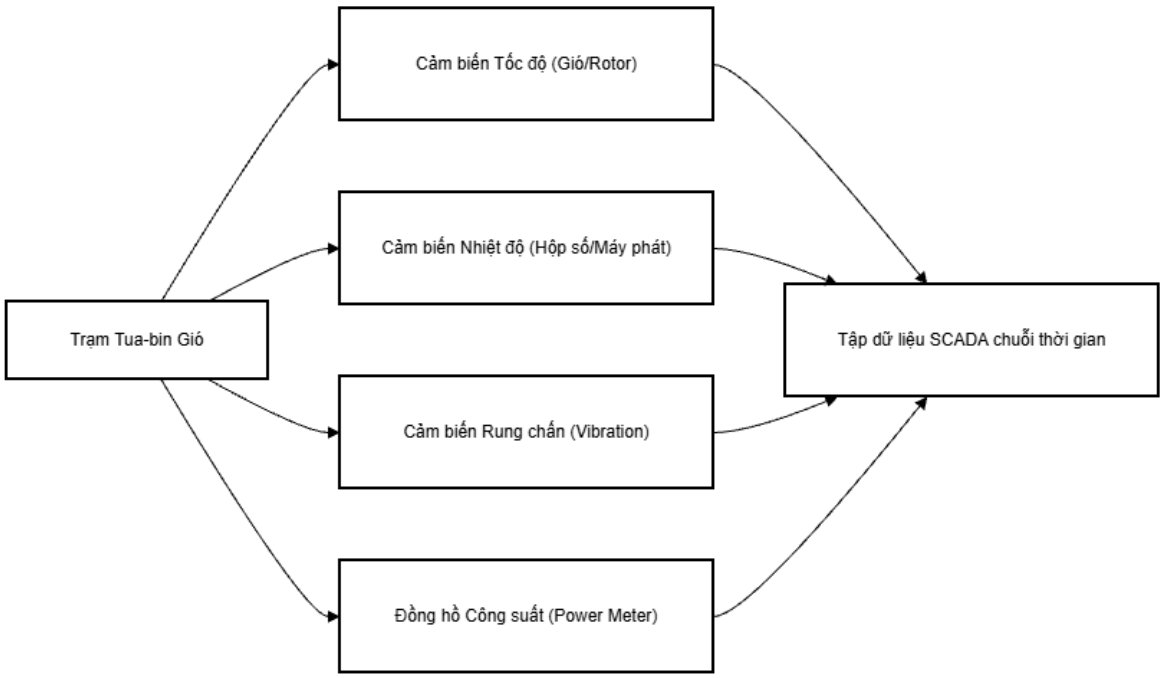


Figure 1.1: Cấu trúc cảm biến và luồng thu thập dữ liệu

1.2 Hạn chế của kiến trúc SCADA truyền thống

1.2.1 Mô hình lưu lượng băng thông mạng

Theo mô hình SCADA truyền thống, toàn bộ dữ liệu thô thu thập từ các trạm tua-bin được truyền trực tiếp về máy chủ trung tâm hoặc đám mây để lưu trữ và phân tích. Với K tua-bin, tần số lấy mẫu f_s và kích thước trung bình S_{raw} của một bản ghi thô, lưu lượng truyền thông lý thuyết là

$$BW_{\text{trad}} = K \cdot f_s \cdot S_{\text{raw}} \quad \text{(bytes/s)} \qquad [6;7]. \tag{1.1}$$

Biểu thức cho thấy tải đường truyền tăng tuyến tính theo số tua-bin, tần số lấy mẫu và kích thước bản ghi. Khi số cảm biến hoặc số trạm tăng, việc truyền liên tục dữ liệu thô qua mạng diện rộng làm gia tăng áp lực lên băng thông uplink trước khi máy chủ có thể thực hiện suy luận.

1.2.2 Mô hình thời gian phản ứng sự cố

Thời gian phản ứng của kiến trúc tập trung được phân rã thành

$$\begin{aligned} T_{\text{trad}} = T_{\text{sense}} + T_{\text{uplink}} + T_{\text{cloud_infer}} \\ + T_{\text{downlink}} + T_{\text{action}} \end{aligned} \qquad [8;9]. \tag{1.2}$$

Trong đó, T_{uplink} và T_{downlink} phụ thuộc vào chất lượng mạng diện rộng. Tổng thời gian từ khi phát sinh bất thường đến khi hệ thống đưa ra phản ứng vì vậy chịu tác động của cả độ trễ truyền thông và độ trễ tính toán. Khi kết nối bị gián đoạn, dữ liệu không đến được khối suy luận tập trung và hệ thống mất khả năng đưa ra quyết định tức thời.

1.3 Giải pháp đề xuất

Hệ thống giám sát trong đồ án đề xuất một kiến trúc phân tán tích hợp lớp tính toán biên nhằm khắc phục các hạn chế của mô hình quản lý tập trung. Bằng cách sử dụng vi máy tính Orange Pi làm cổng giao tiếp biên đặt ngay tại tháp tua-bin theo kiến trúc điện toán biên công nghiệp, năng lực phân tích được dịch chuyển đến sát nguồn sinh dữ liệu. Bo mạch này không chỉ trung chuyển gói tin mà còn đóng vai trò như một thiết bị xử lý độc lập, giúp cân bằng giữa chi phí triển khai, tiêu thụ năng lượng và khả năng xử lý thời gian thực.

Cốt lõi của giải pháp này là mô hình học máy XGBoost được nhúng trực tiếp vào hệ



điều hành của bo mạch biên. Đây là thuật toán dựa trên hệ thống cây quyết định, đặc biệt tối ưu cho việc xử lý dữ liệu chuỗi thời gian và tiêu tốn ít tài nguyên phần cứng hơn nhiều so với các mô hình học sâu. Ưu điểm này cho phép quá trình dự đoán diễn ra liên tục, mượt mà mà không gây quá tải cho thiết bị.

Quy trình giám sát và phát hiện lỗi được vận hành khép kín hoàn toàn tại chỗ. Dữ liệu cảm biến thô sau khi thu thập sẽ được tiền xử lý và đưa vào mô hình XGBoost để tính toán mức công suất kỳ vọng. Hệ thống sau đó đối chiếu kết quả này với công suất phát điện thực tế để trích xuất giá trị phần dư. Nếu độ lệch vượt qua ngưỡng an toàn thiết lập, bo mạch sẽ lập tức xác định trạng thái bất thường và kích hoạt cảnh báo nội bộ. Cuối cùng, thiết bị biên đóng gói các thông số đã qua xử lý để gửi về máy chủ InfluxDB trung tâm, giúp giảm tải băng thông mạng và duy trì khả năng phân tích cục bộ khi kết nối mạng diện rộng bị gián đoạn.

Với tần số gửi metrics f_{down}, kích thước bản ghi tổng hợp S_{metrics}, kích thước sự kiện cảnh báo S_{alert} và tần suất sự kiện bất thường kỳ vọng $P(\text{anomaly})$, lưu lượng của kiến trúc Edge AI được mô hình hóa bởi

$$BW_{\text{edge}} = K \cdot (f_{\text{down}} \cdot S_{\text{metrics}} + P(\text{anomaly}) \cdot S_{\text{alert}}) \quad \text{(bytes/s)} \quad [6;7]. \tag{1.3}$$

Trong biểu thức trên, $P(\text{anomaly})$ được quy đổi thành số sự kiện kỳ vọng trong một giây để bảo đảm tính nhất quán về đơn vị. Hiệu quả cắt giảm băng thông được xác định bởi

$$\eta_{BW} = \left(1 - \frac{BW_{\text{edge}}}{BW_{\text{trad}}}\right) \times 100\% \quad [6;7]. \tag{1.4}$$

Giá trị η_{BW} biểu diễn tỷ lệ giảm lưu lượng nhờ chuyển bước lọc và tổng hợp dữ liệu đến gần nguồn phát sinh, phù hợp với nguyên lý giảm tải truyền thông của điện toán biên. Giá trị này chỉ được xem là kết quả định lượng khi các kích thước tải tin và tần số sự kiện được đo trên hệ thống thực nghiệm.

Do suy luận được thực hiện tại thiết bị biên, thời gian phản ứng được rút gọn thành

$$T_{\text{edge}} = T_{\text{sense}} + T_{\text{edge_infer}} + T_{\text{action}} \quad [8]. \tag{1.5}$$

Khoảng thời gian được tiết kiệm so với kiến trúc tập trung là

$$\Delta T_{\text{saved}} = T_{\text{trad}} - T_{\text{edge}} \tag{1.6}$$

$$= T_{\text{uplink}} + T_{\text{downlink}} + T_{\text{cloud_infer}} - T_{\text{edge_infer}} \tag{1.7}$$

$$\approx T_{\text{uplink}} + T_{\text{downlink}} \quad [8?], \tag{1.8}$$

với phép xấp xỉ cuối được sử dụng khi thời gian suy luận tại biên và trên máy chủ có cùng bậc độ lớn. Cách phân rã này tách rõ độ trễ truyền thông khỏi độ trễ tính toán trong kiến trúc biên.

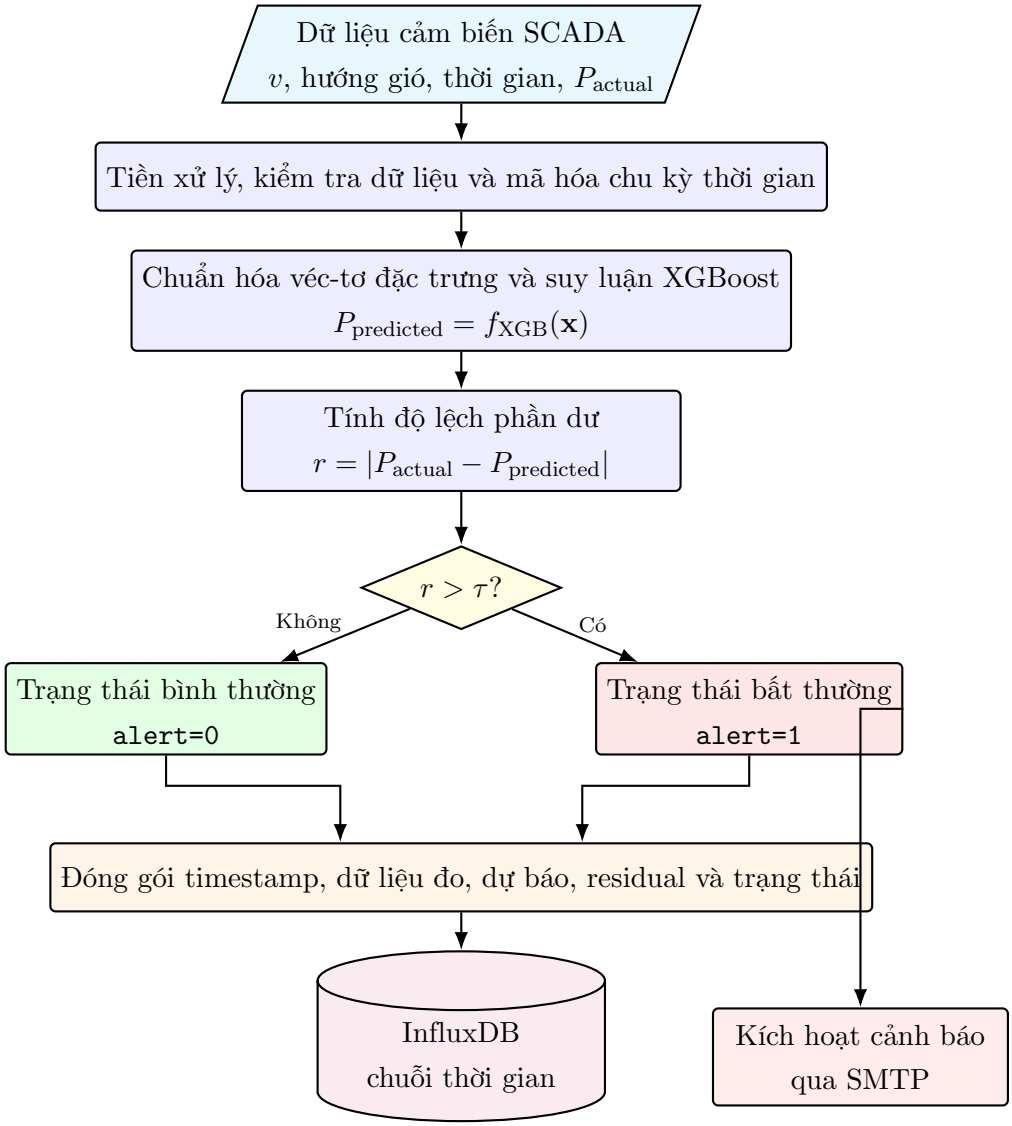


Figure 1.2: Lưu đồ thuật toán giám sát và cảnh báo bất thường tại Edge Gateway

1.3.1 Đánh giá tính ưu việt của giải pháp thông qua đối chiếu kiến trúc

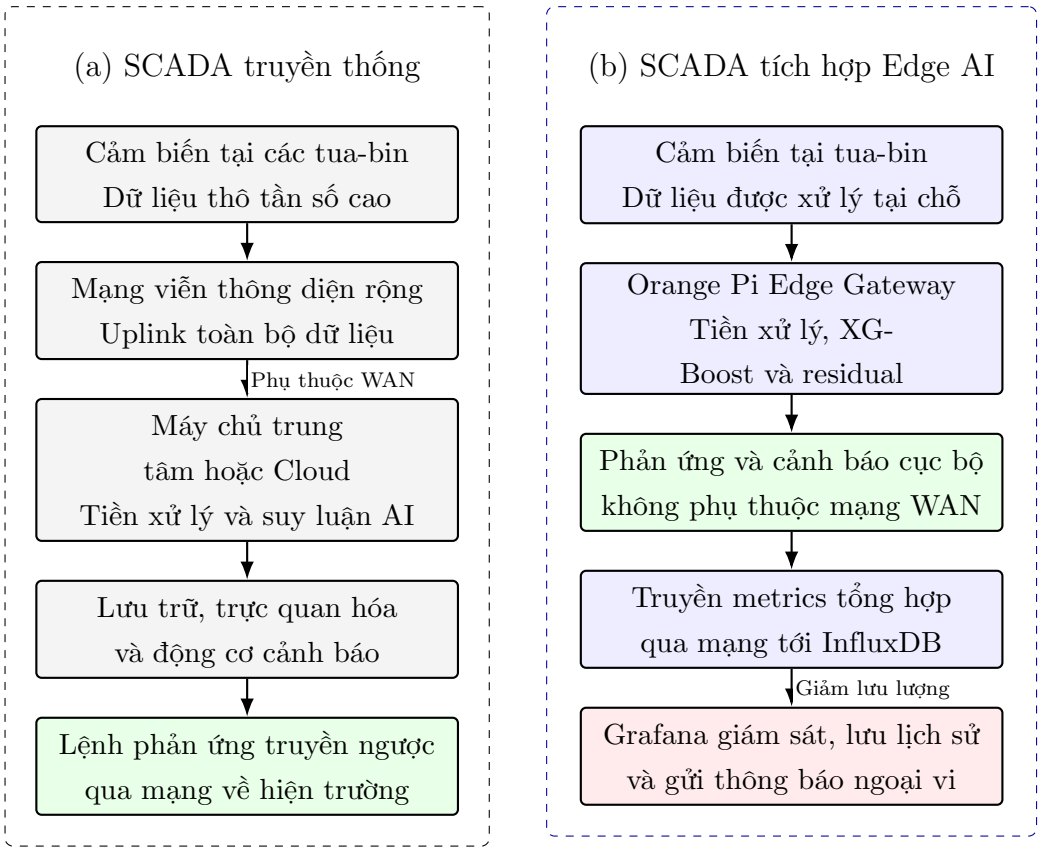


Figure 1.3: Sơ đồ đối chiếu kiến trúc và luồng xử lý dữ liệu giữa mô hình SCADA truyền thống (a) và mô hình tích hợp Edge AI (b)

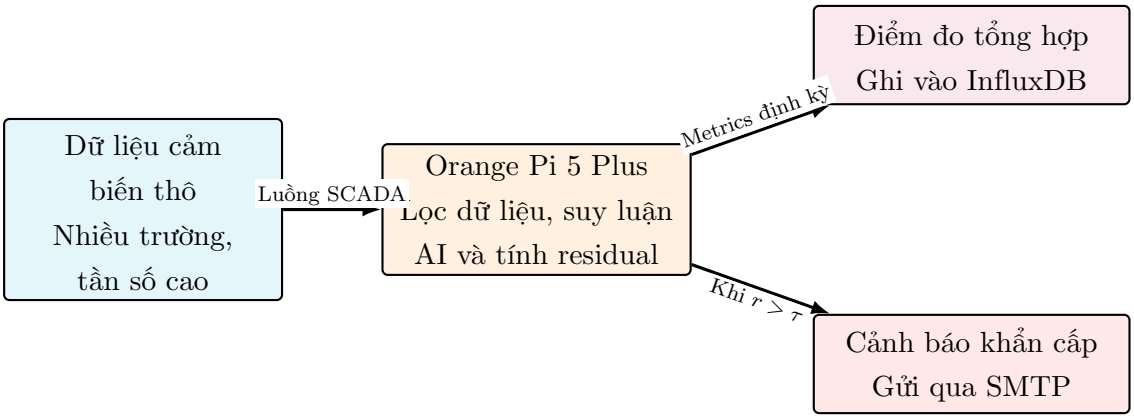


Figure 1.4: Phân loại và định tuyến payload sau quá trình xử lý tại thiết bị biên

Các khác biệt kỹ thuật giữa hai kiến trúc trong fig. 1.3 được hệ thống hóa trong table 1.1.

Table 1.1: Đối chuẩn đa tiêu chí giữa kiến trúc SCADA truyền thống và SCADA Edge AI đề xuất

Tiêu chuẩn kỹ thuật	SCADA truyền thống xử lý tập trung hoặc trên đám mây	SCADA Edge AI đề xuất
Vị trí xử lý học máy	Máy chủ trung tâm hoặc nền tảng đám mây	Cổng biên Orange Pi đặt gần nguồn dữ liệu tua-bin
Lưu lượng băng thông mạng (BW)	BW_{trad} được xác định theo eq. (1.1), truyền liên tục bản ghi thô	BW_{edge} được xác định theo eq. (1.3), chỉ truyền chỉ số tổng hợp và cảnh báo
Độ trễ phát hiện và cảnh báo (T_{response})	T_{trad} gồm độ trễ uplink, suy luận đám mây và downlink	T_{edge} không phụ thuộc uplink và downlink trong vòng phản ứng cục bộ
Khả năng tiếp tục vận hành khi đứt mạng viễn thông	Khâu phân tích và cảnh báo bị gián đoạn khi mất kết nối tới máy chủ	Tiếp tục suy luận, tính residual và kích hoạt cảnh báo cục bộ tại thiết bị biên
Tải tính toán trên máy chủ trung tâm	Tiếp nhận dữ liệu thô và thực hiện suy luận cho toàn bộ tua-bin	Tập trung lưu trữ chuỗi thời gian, trực quan hóa và quản lý cảnh báo

1.4 Mục tiêu, đối tượng và phạm vi nghiên cứu

1.4.1 Mục tiêu nghiên cứu

Để định hướng quá trình hiện thực, nhóm xác định mục tiêu đầu tiên là xây dựng luồng dữ liệu truyền phát liên tục bằng cơ chế đọc tuần tự từ một tệp tĩnh. Cách tổ chức này cho phép tái tạo luồng tín hiệu cảm biến theo nhịp lấy mẫu có kiểm soát. Tiếp theo, nhóm thực hiện tiền xử lý và trích xuất đặc trưng ngay trong vòng lặp: giá trị thời gian được biến đổi thành các đặc trưng lượng giác, sau đó véc-tơ đầu vào được chuẩn hóa trước khi suy luận.

Trọng tâm kỹ thuật của đồ án là nhúng trực tiếp mô hình XGBoost đã huấn luyện vào kịch bản Python để dự đoán công suất kỳ vọng cho từng mẫu tại thiết bị biên. Từ kết quả dự đoán, nhóm tính residual, so sánh với ngưỡng đã cấu hình và gán cờ bất thường. Cuối mỗi chu kỳ, các giá trị đo lường, dự đoán và trạng thái được đóng gói thành Point rồi ghi đồng bộ vào InfluxDB.

1.4.2 Đối tượng nghiên cứu

Tập dữ liệu đầu vào được sử dụng để tái tạo trạng thái vận hành của tua-bin trong môi trường thử nghiệm. Mỗi bản ghi mô tả các điều kiện gió và công suất tại một mốc thời gian xác định. Không gian trạng thái vì vậy gồm các biến môi trường tác động đến năng lượng đầu vào và đại lượng công suất thực tế ở đầu ra.

Hệ thống trích xuất năm trường từ mỗi bản ghi, gồm một trường thời gian, ba đặc trưng đầu vào mô tả điều kiện gió và công suất thiết kế, cùng một biến mục tiêu là công suất thực tế. Biến mục tiêu không được đưa vào véc-tơ suy luận mà chỉ được dùng để tính residual sau khi mô hình đã tạo dự đoán.

1.4.3 Phạm vi nghiên cứu

Về phạm vi, hệ thống chưa được đấu nối trực tiếp với PLC hoặc cảm biến tại trạm điện gió, vì vậy nhóm sử dụng tệp `test.csv` làm nguồn tín hiệu giả lập. Hàm `time.sleep(1)` được đặt ở cuối vòng lặp để duy trì nhịp phát một mẫu mỗi giây. Nhằm phù hợp với tài nguyên của thiết bị biên và quy mô thử nghiệm hiện tại, nhóm chưa triển khai nền tảng môi giới thông điệp trung gian. Toàn bộ chu trình đọc tệp, tiền xử lý, suy luận và ghi dữ liệu được tổ chức trong kịch bản `stream_exporter.py`.

Trong phạm vi nghiệm thu của đồ án, các điều kiện xác minh và công cụ kiểm chứng được chuẩn hóa trong table 1.2.

Table 1.2: Tiêu chí đánh giá nghiệm thu hệ thống

Phân lớp kỹ thuật	Điều kiện cần xác minh	Phương pháp
Phần cứng và hệ điều hành	Kernel nhận đúng kiến trúc aarch64, thiết bị vận hành không giao diện và cho phép quản trị từ xa	`uname -m`, `lscpu` và terminal Ubuntu
Mạng và truy cập từ xa	Địa chỉ IP của thiết bị biên ổn định sau khi khởi động lại hoặc gia hạn DHCP, phiên SSH được thiết lập thành công	DHCP reservation, `ip addr` và SSH
Hạ tầng container	Image tương thích ARM64, các dịch vụ có chính sách khởi động lại và duy trì trạng thái ổn định	Build log, `docker compose ps` và container logs
Lưu trữ chuỗi thời gian	Dữ liệu InfluxDB và cấu hình Grafana được bảo toàn sau khi tạo lại container	Named volumes và InfluxDB Data Explorer
AI và truyền phát dữ liệu	Artifact được nạp đúng, tiến trình tạo dự báo, residual và ghi Point liên tục	Nhật ký AI Exporter và truy vấn dữ liệu kiểm thử
Giám sát và cảnh báo	Dashboard hiển thị công suất thực tế, dự báo và residual, đồng thời phát đúng trạng thái FIRING và RESOLVED	Grafana Dashboard, Alert Rule và email thử nghiệm
An toàn cấu hình	Token, App Password và mật khẩu quản trị không nằm trong mã nguồn hoặc hình ảnh công khai	Rà soát repository, tệp môi trường và ảnh minh chứng

1.5 Bố cục của đồ án

Table 1.3: Bố cục của đồ án

Chương	Tên chương	Tổng quan
1	Tổng quan đề tài	Xác lập bài toán, mô hình hóa băng thông và độ trễ, xác định mục tiêu, đối tượng, phạm vi và tiêu chí nghiệm thu.
2	Cơ sở lý thuyết	Trình bày mô hình khí động học, XGBoost, mã hóa chu kỳ thời gian, quy tắc ba Sigma, TSDB và SMTP/TLS.
3	Thiết kế kiến trúc hệ thống SCADA Edge	Thiết kế kiến trúc phân tầng, lựa chọn Orange Pi 5 Plus, container hóa dịch vụ, mạng bridge và cơ chế quản lý thông tin xác thực.
4	Hiện thực luồng dữ liệu thời gian thực và mô hình AI tại biên	Xây dựng pipeline tiền xử lý, huấn luyện XGBoost, đóng gói artifact, suy luận streaming, ghi InfluxDB và giảm tần suất mẫu.
5	Hệ thống giám sát, thiết lập cảnh báo và thông báo sự cố	Hiện thực Dashboard Grafana, chuỗi đánh giá A–B–C, máy trạng thái cảnh báo, SMTP STARTTLS và Notification Policy.
6	Triển khai thực nghiệm, đánh giá kết quả và hướng phát triển	Chuyển giao lên Ubuntu ARM64, kiểm thử End-to-End, đánh giá phục hồi dữ liệu, phân tích hạn chế và đề xuất hướng phát triển.

Chương 2: Cơ sở lý thuyết

2.1 Mô hình khí động học và đường cong công suất tua-bin gió

Quá trình chuyển đổi động năng của dòng khí thành cơ năng quay trục và điện năng trong tua-bin gió được mô tả thông qua phương trình khí động học:

$$P_{\text{mech}}(t) = \frac{1}{2}\rho\pi R^2 v^3(t) C_p(\lambda, \beta) \quad [2;10]. \tag{2.1}$$

Trong đó:

- ρ là mật độ không khí tại độ cao đặt tâm cánh quạt (kg/m^3).
- πR^2 là diện tích quét của cánh quạt (m^2) với bán kính cánh quạt R.
- $v(t)$ là vận tốc gió tức thời tại thời điểm t (m/s).
- C_p là hệ số hiệu suất khí động lực học của turbine, phụ thuộc vào tỉ số tốc độ đầu cánh λ và góc nghiêng cánh quạt β. Tỉ số λ được xác định theo vận tốc góc ω của trục quay rotor:

$$\lambda = \frac{\omega R}{v(t)} \tag{2.2}$$

Theo định luật giới hạn Betz, giá trị cực đại của hệ số công suất đạt $C_{p,\max} = \frac{16}{27} \approx 0.593$. Trong điều kiện vận hành công nghiệp thực tế, hệ số này thường thấp hơn giới hạn lý thuyết do tổn hao khí động học, cơ khí và điện.

Đường cong công suất được mô hình hóa bằng hàm giải tích từng khúc tương ứng với bốn vùng vận hành:

$$P(v) = \begin{cases} 0, & v < v_{\text{in}} \quad \text{(Vùng I: dưới cut-in)}, \\ \dfrac{1}{2}\rho\pi R^2 v^3 C_p(\lambda_{\text{opt}}, \beta_0)\,, & v_{\text{in}} \le v < v_{\text{rated}} \quad \text{(Vùng II: MPPT)}, \\ P_{\text{rated}}, & v_{\text{rated}} \le v \le v_{\text{out}} \quad \text{(Vùng III: pitch control)}, \\ 0, & v > v_{\text{out}} \quad \text{(Vùng IV: cut-out)}. \end{cases} \quad [1;11]. \tag{2.3}$$

Trong mô hình thử nghiệm, các mốc vận hành được thiết lập tại $v_{\text{in}} = 3\,\text{m}\,\text{s}^{-1}$, $v_{\text{rated}} = 12\,\text{m}\,\text{s}^{-1}$, $v_{\text{out}} = 25\,\text{m}\,\text{s}^{-1}$ và $P_{\text{rated}} = 1500\,\text{kW}$. Các đặc trưng điều khiển của từng vùng được tổng hợp trong table 2.1.

Table 2.1: Đặc tả kỹ thuật bốn vùng vận hành của tua-bin gió

Vùng vận hành	Dải vận tốc gió (v)	Góc pitch (β)	Tốc độ rotor (ω)	Mục tiêu điều khiển	Công suất đầu ra (P)
Vùng I	$v < 3\,\mathrm{m\,s^{-1}}$	$\beta = 90°$, vị trí giảm lực nâng	$\omega = 0$	Chờ gió, ngắt kết nối máy phát	0 kW
Vùng II	$3\,\mathrm{m\,s^{-1}} \le v < 12\,\mathrm{m\,s^{-1}}$	$\beta = 0°$ (cố định)	Biến thiên theo v	Tối ưu hóa $C_{p,\max}$ qua tỷ số λ_{opt}	$P \propto v^3$ ($0 \to 1500$ kW)
Vùng III	$12\,\mathrm{m\,s^{-1}} \le v \le 25\,\mathrm{m\,s^{-1}}$	$0° < \beta \le 30°$	Giữ định mức ω_{rated}	Xả năng lượng gió thừa, bảo vệ cơ khí	Giữ ổn định 1500 kW
Vùng IV	$v > 25\,\mathrm{m\,s^{-1}}$	$\beta = 90°$ và phanh cơ	$\omega \to 0$	Dừng khẩn cấp, chống quá tải cấu trúc	0 kW

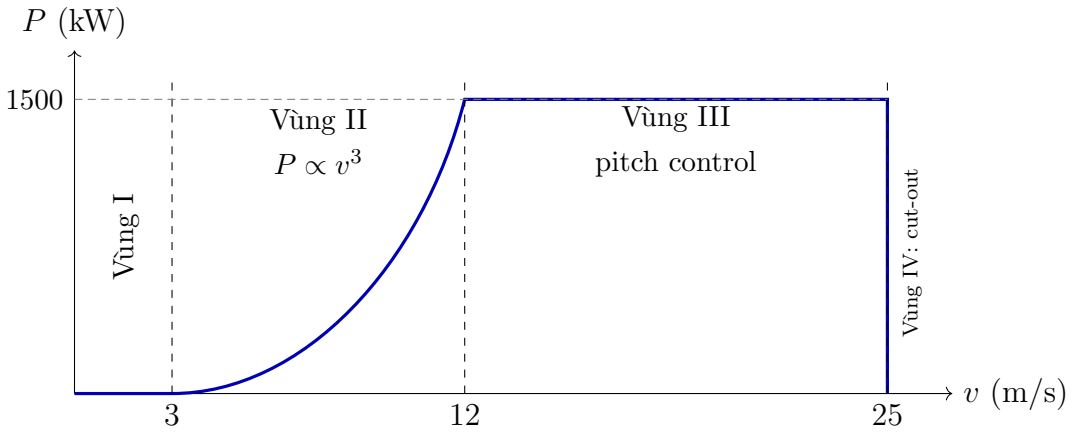


Figure 2.1: Đường cong công suất lý tưởng với bốn vùng vận hành của tua-bin gió

Hàm từng khúc và bảng đặc tả cho thấy quan hệ bậc ba theo tốc độ gió chỉ áp dụng trong Vùng II. Từ vận tốc định mức trở lên, bộ điều khiển thay đổi góc pitch để giới hạn công suất và bảo vệ cơ khí. Do đó, cùng một tốc độ gió có thể tương ứng với nhiều mức công suất nếu trạng thái pitch, phanh, giới hạn lưới hoặc bảo trì khác nhau [12].

2.2 Nền tảng toán học của thuật toán XGBoost

XGBoost là phương pháp gradient boosting trên tập hợp cây quyết định. Ở vòng boosting thứ t, cây mới f_t được cộng vào dự báo hiện tại theo

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i) \qquad [13;14;15], \tag{2.4}$$

trong đó η là tốc độ học. Mỗi cây tập trung sửa phần sai số còn lại của tập hợp cây trước đó.

Hàm mục tiêu gồm thành phần mất mát và thành phần điều chuẩn:

$$\mathcal{J}^{(t)} = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i^{(t)}\right) + \sum_{k=1}^{t} \Omega(f_k). \tag{2.5}$$

Với `objective=reg:squarederror`, hàm mất mát là

$$l(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2 . \tag{2.6}$$

Đạo hàm cấp một và cấp hai của hàm mất mát tại dự báo của vòng trước được xác định bởi

$$g_i = \left.\frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i}\right|_{\hat{y}_i = \hat{y}_i^{(t-1)}} = 2\left(\hat{y}_i^{(t-1)} - y_i\right), \tag{2.7}$$

$$h_i = \left.\frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}\right|_{\hat{y}_i = \hat{y}_i^{(t-1)}} = 2. \tag{2.8}$$

Việc giữ hệ số 2 trong cả gradient và Hessian bảo đảm các đạo hàm nhất quán với định nghĩa squared error trong eq. (2.6). Nếu hàm mất mát được chuẩn hóa thêm hệ số $1/2$, hai đại lượng tương ứng sẽ trở thành $g_i = \hat{y}_i^{(t-1)} - y_i$ và $h_i = 1$.

Độ phức tạp của một cây được điều chuẩn bởi

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2, \tag{2.9}$$

trong đó T là số lá, w_j là trọng số lá, γ là chi phí tạo lá mới và λ giới hạn độ lớn trọng số. Thành phần này chống lại cây quá sâu và các lá có dự báo cực đoan.

XGBoost dùng khai triển Taylor bậc hai của hàm mất mát. Nếu g_i và h_i lần lượt là gradient và Hessian, mục tiêu gần đúng của cây mới là

$$\widetilde{\mathcal{J}}^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t). \tag{2.10}$$

Với một cấu trúc cây cố định q, mỗi mẫu được truyền tới một trong T lá và cây trả về trọng số của lá tương ứng:

$$f_t(x) = w_{q(x)}, \qquad q : \mathbb{R}^d \to \{1, 2, \ldots, T\}. \tag{2.11}$$

Gọi $I_j = \{i \mid q(x_i) = j\}$ là tập mẫu tại lá thứ j. Tổng gradient và Hessian trên lá được ký hiệu

$$G_j = \sum_{i \in I_j} g_i, \qquad H_j = \sum_{i \in I_j} h_i. \tag{2.12}$$

Thay biểu diễn cây và thành phần điều chuẩn vào eq. (2.10), hàm mục tiêu theo các trọng số lá trở thành

$$\widetilde{\mathcal{J}}^{(t)}(q, \mathbf{w}) = \sum_{j=1}^{T} \left[G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2 \right] + \gamma T. \tag{2.13}$$

Đạo hàm riêng theo w_j được đặt bằng không để giải bài toán cực tiểu trên từng lá:

$$\frac{\partial \widetilde{\mathcal{J}}^{(t)}}{\partial w_j} = G_j + (H_j + \lambda) w_j = 0. \tag{2.14}$$

Do đó, trọng số tối ưu tại lá thứ j là

$$w_j^* = -\frac{G_j}{H_j + \lambda} \qquad [15]. \tag{2.15}$$

Thay w_j^* trở lại hàm mục tiêu thu được điểm tối ưu của cấu trúc cây q:

$$\widetilde{\mathcal{J}}^*(q) = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T \qquad [15]. \tag{2.16}$$

Giá trị càng nhỏ biểu thị cấu trúc cây càng phù hợp sau khi đã cân bằng giữa mức giảm sai số và độ phức tạp số lá.

Độ lợi của phép chia một nút thành hai nhánh trái và phải được tính bởi

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \qquad [15]. \tag{2.17}$$

Phép chia chỉ được chấp nhận khi độ lợi đủ lớn, tạo ra cơ chế cắt tỉa ngay trong quá trình xây cây.

2.2.1 So sánh các phương pháp tìm điểm phân nhánh

Với `tree_method=hist`, các giá trị liên tục được lượng tử hóa thành K bin. Gradient và Hessian của các mẫu trong cùng bin được cộng dồn, sau đó thuật toán chỉ đánh giá các biên bin thay vì quét toàn bộ giá trị thực[15;16]. So sánh định tính giữa các phương pháp xây cây được trình bày trong table 2.2.

Table 2.2: So sánh các phương pháp tìm điểm phân nhánh trong XGBoost

Thuật toán phân nhánh	Chi phí tính toán đặc trưng	Mức chiếm dụng RAM	Cách tìm điểm chia	Đánh giá triển khai ARM64
Exact Greedy	Cao, phải sắp xếp và quét các giá trị ứng viên	Cao, phụ thuộc số mẫu và số đặc trưng	Duyệt các điểm chia trên giá trị thực đã sắp xếp	Chi phí lớn khi số mẫu tăng, không phù hợp cấu hình tài nguyên hạn chế
Approximate	Giảm số điểm chia bằng các điểm phân vị ứng viên	Trung bình	Xây dựng quantile sketch và đánh giá các điểm chia xấp xỉ	Có khả năng mở rộng nhưng phát sinh chi phí tạo và cập nhật sketch
Histogram	Xây histogram từ dữ liệu và quét K bin trên mỗi đặc trưng	Thấp hơn, phần thống kê histogram xấp xỉ $O(Kd)$	Lượng tử hóa cố định vào K bin và cộng dồn g_i, h_i	Phù hợp nhất trong ba lựa chọn đối với Orange Pi 5 Plus và pipeline của đồ án

Độ phức tạp thực tế còn phụ thuộc số tầng cây, độ thưa của dữ liệu, số luồng xử lý và cách XGBoost tái sử dụng histogram. Vì vậy, bảng trên được dùng để giải thích lựa chọn thuật toán trong phạm vi cấu hình đồ án, không phải kết luận rằng một phương pháp luôn tối ưu cho mọi tập dữ liệu.

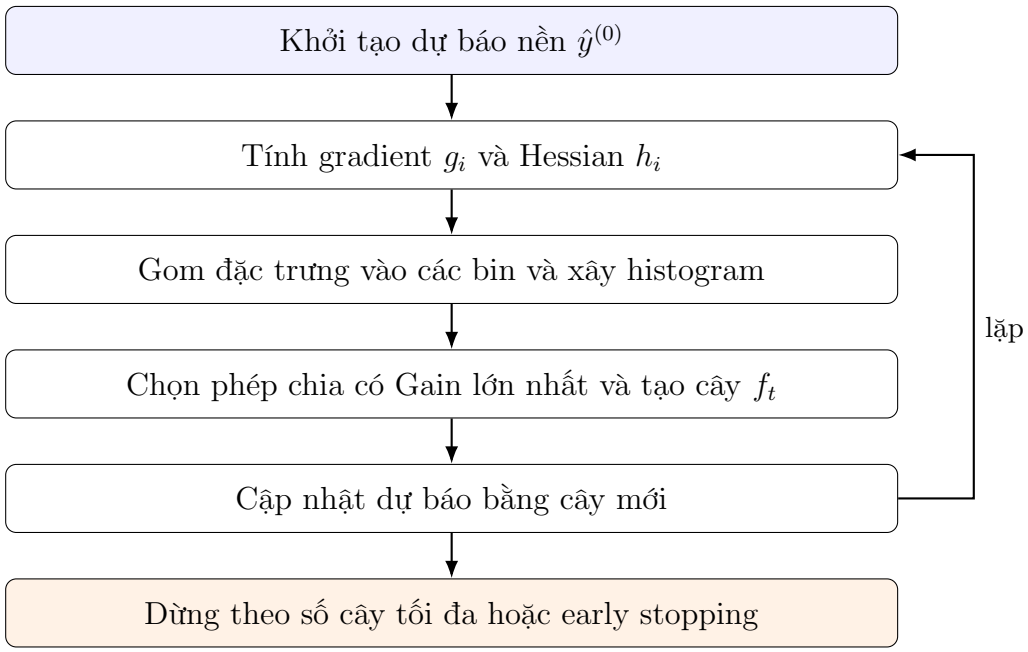


Figure 2.2: Lưu đồ huấn luyện XGBoost với phương pháp histogram

2.3 Biến đổi lượng giác cho dữ liệu thời gian

Trong hàm `preprocess_datetime`, timestamp được phân rã thành giờ, phút và tháng. Nếu giữ thời gian dưới dạng số tuyến tính, khoảng cách số học không phản ánh đúng quan

hệ tuần hoàn tại ranh giới ngày. Số phút tính từ đầu ngày được xác định bởi

$$m_{\text{day}} = 60h + m, \qquad 0 \le m_{\text{day}} < 1440. \tag{2.18}$$

Các đặc trưng chu kỳ ngày được tạo bởi

$$\text{Day}_{\sin} = \sin\left(2\pi \frac{m_{\text{day}}}{1440}\right), \tag{2.19}$$

$$\text{Day}_{\cos} = \cos\left(2\pi \frac{m_{\text{day}}}{1440}\right). \tag{2.20}$$

Với tháng $q \in \{1, \ldots, 12\}$, cặp đặc trưng mùa vụ là

$$\text{Month}_{\sin} = \sin\left(2\pi \frac{q}{12}\right), \tag{2.21}$$

$$\text{Month}_{\cos} = \cos\left(2\pi \frac{q}{12}\right). \tag{2.22}$$

Công thức trên bám đúng mục đích: tháng được lấy trực tiếp trong miền 1 đến 12 rồi chia cho 12. So với cách dùng $(q-1)/12$, biểu diễn này chỉ quay toàn bộ đường tròn một góc cố định và vẫn bảo toàn quan hệ lân cận giữa tháng 12 và tháng 1.

Gọi $\mathbf{x}(t) = (\text{Day}_{\sin}(t), \text{Day}_{\cos}(t))$ là véc-tơ lượng giác của thời điểm t. Khoảng cách Euclidean giữa hai thời điểm sau mã hóa được tính bởi

$$d_{L_2}(\mathbf{x}_1, \mathbf{x}_2) = \|\mathbf{x}_1 - \mathbf{x}_2\|_2 = \sqrt{\left(\text{Day}_{\sin,1} - \text{Day}_{\sin,2}\right)^2 + \left(\text{Day}_{\cos,1} - \text{Day}_{\cos,2}\right)^2} \quad \text{[3;17]}. \tag{2.23}$$

Nếu hai thời điểm cách nhau Δm phút theo khoảng cách vòng tròn ngắn nhất, khoảng cách trên đường tròn đơn vị tương đương

$$d_{L_2} = 2\sin\left(\pi \frac{\Delta m}{1440}\right), \qquad 0 \le \Delta m \le 720. \tag{2.24}$$

Kết quả đối chứng trong table 2.3 cho thấy mã hóa tuyến tính tạo bước nhảy lớn tại nửa đêm, trong khi khoảng cách lượng giác vẫn phản ánh đúng độ gần nhau thực tế.

Table 2.3: Đối chứng khoảng cách giữa mã hóa tuyến tính và mã hóa lượng giác chu kỳ

Mốc thời gian t_1	Mốc thời gian t_2	Chênh lệch thực	Khoảng cách tuyến tính $\lvert\Delta t\rvert$	Tọa độ lượng giác $(\text{Day}_{\sin}, \text{Day}_{\cos})$	Khoảng cách Euclidean d_{L_2}
23:59	00:01 hôm sau	2 phút	$\lvert 1439 - 1\rvert = 1438$ (sai lệch cực đại)	$(-0.0044, 0.9999)$ và $(0.0044, 0.9999)$	0.0087 (rất nhỏ, giữ tính liên tục)
12:00	00:00	12 giờ	$\lvert 720 - 0\rvert = 720$	$(0, -1)$ và $(0, 1)$	2.0000 (đối xứng nửa chu kỳ)

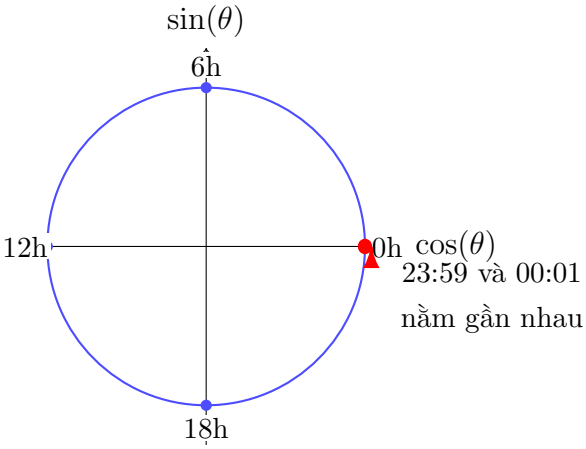


Figure 2.3: Mã hóa thời gian bằng tọa độ sin–cos

Hai tọa độ sin và cos cùng được sử dụng để xác định duy nhất vị trí trên đường tròn. Nhờ vậy, mô hình có thể học biến thiên ngày–đêm và mùa vụ mà không tạo bước nhảy nhân tạo ở ranh giới chu kỳ[18;19].

2.4 Nguyên lý phát hiện bất thường dựa trên quy tắc thống kê ba Sigma

Residual tại thời điểm i được định nghĩa là

$$r_i = |y_i - \hat{y}_i| \,. \tag{2.25}$$

Từ n residual đại diện cho trạng thái vận hành bình thường, kỳ vọng mẫu và độ lệch chuẩn mẫu được ước lượng bởi

$$\mu_r = \frac{1}{n}\sum_{i=1}^{n} r_i, \tag{2.26}$$

$$\sigma_r = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(r_i - \mu_r)^2}. \tag{2.27}$$

Mẫu số $n-1$ trong eq. (2.27) là hiệu chỉnh Bessel, phù hợp với cách `pandas.Series.std()` ước lượng độ lệch chuẩn mẫu. Ngưỡng cảnh báo một phía là

$$\tau = \mu_r + 3\sigma_r \qquad [4;20]. \tag{2.28}$$

Hàm chỉ thị quyết định trạng thái nhị phân tại thời điểm t được định nghĩa là

$$a(t) = \mathbb{I}(r(t) > \tau) = \begin{cases} 1, & r(t) > \tau \quad \text{ứng viên bất thường,} \\ 0, & r(t) \leq \tau \quad \text{vận hành bình thường.} \end{cases} \tag{2.29}$$

Với biến có phân phối chuẩn, khoảng $\mu \pm 3\sigma$ chứa xấp xỉ 99.7% quan sát. Do residual không âm, hệ thống chỉ quan tâm phần đuôi phải. Điểm vượt ngưỡng được xem là ứng viên bất thường, không phải bằng chứng chắc chắn của hỏng hóc; kết luận cuối cùng còn phụ thuộc trạng thái pitch, phanh và mã vận hành [12;21?].

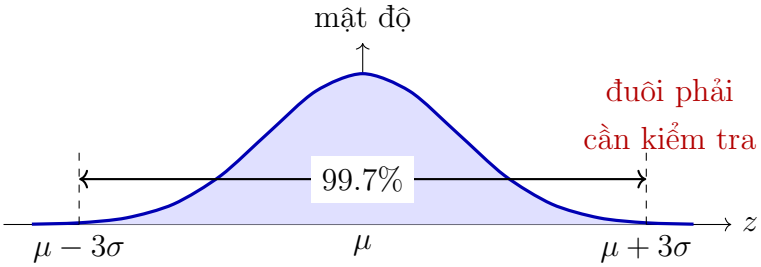


Figure 2.4: Ý nghĩa thống kê của vùng ba Sigma

Với biến chuẩn hóa $Z \sim \mathcal{N}(0, 1)$, xác suất một quan sát nằm ngoài miền đối xứng $\mu \pm k\sigma$ được xác định bởi

$$P(|r - \mu_r| > k\sigma_r) = 2\left[1 - \Phi(k)\right], \tag{2.30}$$

trong đó $\Phi(k)$ là hàm phân phối tích lũy của phân phối chuẩn tắc. Độ nhạy lý thuyết của các mức ngưỡng được so sánh trong table 2.4.

Table 2.4: Phân tích độ nhạy và xác suất báo động giả theo quy tắc bội số độ lệch chuẩn

Bội số ngưỡng	Công thức ngưỡng τ	Xác suất hai phía $P(\lvert r-\mu_r\rvert > k\sigma_r)$	Số mẫu ngoài ngưỡng trung bình trên 10 000 mẫu	Đánh giá khả năng áp dụng
1σ	$\mu_r + 1\sigma_r$	31.73%	$\approx$ 3 173 mẫu	Độ nhạy quá cao, dễ gây báo cảnh báo
2σ	$\mu_r + 2\sigma_r$	4.55%	$\approx$ 455 mẫu	Có thể sử dụng cho mức Warning hoặc cảnh báo sớm
3σ – lựa chọn của đồ án	$\mu_r + 3\sigma_r$	0.27%	$\approx$ 27 mẫu	Mức cân bằng được lựa chọn giữa độ nhạy và độ tin cậy
3.5σ	$\mu_r + 3.5\sigma_r$	0.046%	$\approx$ 4.6 mẫu	Ngưỡng chặt hơn, có nguy cơ bỏ sót dấu hiệu sự cố ban đầu

Các xác suất trong table 2.4 được tính cho miền hai phía nhằm thể hiện quy tắc $\mu \pm k\sigma$ thông dụng. Hệ thống của đồ án sử dụng residual không âm và điều kiện một phía $r > \mu_r + k\sigma_r$, vì vậy xác suất vượt ngưỡng lý thuyết là

$$P(r > \mu_r + k\sigma_r) = 1 - \Phi(k), \tag{2.31}$$

tức bằng một nửa xác suất và số mẫu kỳ vọng trong bảng. Chẳng hạn, ngưỡng 3σ một phía tương ứng khoảng 0.135%, hay xấp xỉ 13.5 mẫu trên 10 000 mẫu nếu giả thiết phân phối chuẩn được thỏa mãn. Các giá trị này là tham chiếu lý thuyết; tỷ lệ báo động giả thực tế phải được xác nhận trên residual của tập calibration vì phân phối quan sát có thể lệch phải.

2.4.1 Xác lập ngưỡng cảnh báo

Từ tập đánh giá, trung bình residual là $\mu_r = 344.21\,\text{kW}$ và độ lệch chuẩn là $\sigma_r = 532.62\,\text{kW}$. Ngưỡng cảnh báo được tính bởi

$$\tau = \mu_r + 3\sigma_r = 344.21 + 3 \times 532.62 = 1942.07\,\text{kW}. \tag{2.32}$$

Theo eq. (2.29), một mẫu được gắn cờ khi $|P_{\text{actual}} - P_{\text{predicted}}| > 1942.07\,\text{kW}$. Ngưỡng này là đường cơ sở ban đầu. Residual thực tế có thể lệch phải hoặc thay đổi theo tốc độ gió, do đó cần kiểm tra histogram, Q–Q plot và tỷ lệ báo động giả theo từng regime trước khi cố định cho vận hành dài hạn.

Trong mã nguồn, residual được tính trực tiếp trên tập test bằng `np.abs(y_test -`

`y_pred`), độ lệch chuẩn được lấy bằng `pandas.Series.std()`, mặc định dùng hiệu chỉnh mẫu $ddof = 1$. Cùng tập test vừa được dùng để báo cáo MAE/RMSE, vừa được dùng để hiệu chỉnh ngưỡng. Cách làm này phù hợp cho prototype, nhưng để đánh giá khách quan hơn nên ước lượng μ_r và σ_r trên một tập calibration/validation bình thường, sau đó giữ test hoàn toàn độc lập.

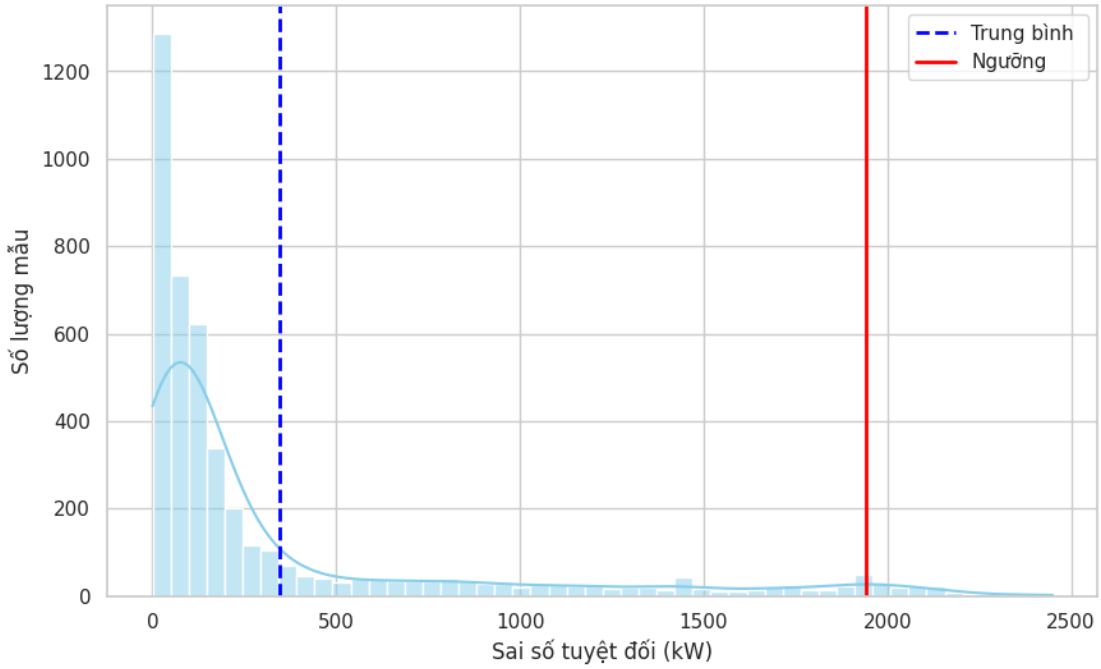


Figure 2.5: Phân phối residual tuyệt đối và ngưỡng cảnh báo ba Sigma

Histogram trong fig. 2.5 có dạng lệch phải: phần lớn mẫu tập trung ở vùng residual thấp, trong khi một số ít quan sát tạo thành đuôi dài. Đường nét đứt biểu diễn residual trung bình và đường đỏ đánh dấu ngưỡng 1942.07 kW. Khoảng cách lớn giữa hai đường cho thấy rule ba Sigma chỉ lựa chọn các sai lệch nằm sâu trong đuôi phân phối, qua đó tạo cơ sở thống kê ban đầu cho cơ chế cảnh báo.

Table 2.5: Ví dụ diễn giải trạng thái theo residual

Residual (kW)	So với ngưỡng	Trạng thái sơ bộ	Diễn giải
115.20	Thấp hơn	Bình thường	Sai số nằm trong vùng thông thường
1942.07	Bằng	Chưa vượt	
1961.93	Cao hơn	Ứng viên bất thường	Kiểm tra pitch, phanh và mã trạng thái

2.4.2 Mô hình xác định sai lệch phục vụ giám sát vận hành

Nhằm phát hiện sớm các hiện tượng suy giảm hiệu suất hoặc hư hỏng cơ khí tiềm ẩn, hệ thống thu thập giá trị công suất đo đạc thực tế $P_{\text{actual}}(t)$ và so sánh trực tiếp với công suất ước lượng $P_{\text{predicted}}(t)$ thu được từ mô hình tham chiếu:

$$e(t) = P_{\text{actual}}(t) - P_{\text{predicted}}(t) \tag{2.33}$$

Để triệt tiêu các thành phần nhiễu tần số cao phát sinh do luồng gió xoáy cục bộ và sai số lượng tử hóa của cảm biến, giá trị sai lệch được lọc thông qua thuật toán trung bình động làm mượt hàm mũ [22]:

$$S(t) = \alpha \cdot e(t) + (1 - \alpha) \cdot S(t-1), \quad 0 < \alpha \leq 1 \tag{2.34}$$

Với hệ số suy giảm α được điều chỉnh theo chu kỳ lấy mẫu để cân bằng giữa độ nhạy phát hiện sự cố và khả năng triệt tiêu nhiễu.

2.5 Cơ chế lưu trữ và nén dữ liệu chuỗi thời gian

2.5.1 Cấu trúc động cơ Time-Structured Merge Tree

Nhóm lựa chọn InfluxDB v2 vì động cơ Time-Structured Merge Tree được thiết kế cho khối lượng ghi chuỗi thời gian liên tục. Trong kiến trúc này, dữ liệu mới trước hết được ghi nối tiếp vào Write-Ahead Log và đồng thời biểu diễn trong cache ở RAM. WAL tạo cơ sở để phục hồi các phép ghi gần nhất sau khi tiến trình dừng bất thường, còn cache cho phép truy vấn dữ liệu chưa được chuyển thành tệp TSM [? ?].

Theo chu kỳ của storage engine, nội dung cache được snapshot và hợp nhất thành các tệp TSM chỉ đọc. Những tệp này lưu dữ liệu series theo thứ tự thời gian và sử dụng cách mã hóa phù hợp với từng kiểu dữ liệu. Khi dữ liệu đã được ghi an toàn vào TSM, các phân đoạn WAL tương ứng mới có thể được loại bỏ. Cơ chế phối hợp giữa WAL, cache và TSM giúp hệ thống cân bằng độ bền của thao tác ghi, tốc độ truy vấn và dung lượng lưu trữ trên thiết bị biên [23].

2.5.2 Các giải thuật nén dữ liệu chuỗi thời gian

Để giảm lượng dữ liệu phải ghi lên bộ nhớ flash, TSM lựa chọn bộ mã hóa theo kiểu giá trị thay vì áp dụng một thuật toán duy nhất cho toàn bộ Point. Với timestamp, storage engine trước hết tính khoảng cách giữa các thời điểm liên tiếp. Khi chu kỳ lấy mẫu ổn định, các giá trị delta lặp lại có thể được biểu diễn hiệu quả bằng run-length encoding; các

trường hợp còn lại được đóng gói bằng phương pháp mã hóa số nguyên phù hợp. Các kỹ thuật đóng gói từ 64 bit và giải mã số nguyên theo lô cho thấy việc tổ chức bit ảnh hưởng trực tiếp đến cả tỷ lệ nén lẫn thông lượng giải mã[24;25]. Vì vậy, nhận định mỗi timestamp chỉ cần một bit chỉ đúng trong trường hợp lý tưởng khi các delta hoàn toàn đồng nhất, không phải mức dung lượng cố định cho mọi chuỗi.

Đặc tả logic bitwise của mã hóa Gorilla XOR: Gọi v_i và v_{i-1} là hai giá trị công suất thực liên tiếp. Trước khi thực hiện phép XOR, mỗi giá trị được diễn giải dưới dạng từ nhị phân IEEE 754 64 bit:

$$b_i = \mathrm{IEEE754}_{64}(v_i), \qquad b_{i-1} = \mathrm{IEEE754}_{64}(v_{i-1}). \tag{2.35}$$

Sai phân bitwise giữa hai mẫu liên tiếp là

$$\Delta_i = b_i \oplus b_{i-1}, \tag{2.36}$$

trong đó $\oplus$ là phép XOR theo từng bit. Với $\Delta_i \neq 0$, số bit 0 dẫn đầu và kết thúc được xác định bởi

$$L_i = \mathrm{clz}(\Delta_i), \qquad T_i = \mathrm{ctz}(\Delta_i), \tag{2.37}$$

trong đó `clz` và `ctz` lần lượt là *Count Leading Zeros* và *Count Trailing Zeros*. Độ dài vùng bit có ý nghĩa là

$$\ell_i = 64 - L_i - T_i, \qquad B_i = \mathrm{slice}(\Delta_i, L_i, \ell_i), \tag{2.38}$$

với B_i là chuỗi ℓ_i bit có ý nghĩa sau khi loại bỏ các bit 0 ở hai đầu. Ký hiệu $\|$ biểu diễn phép nối các chuỗi bit. Logic đóng gói được đặc tả bởi

$$\mathcal{E}(\Delta_i) = \begin{cases} \texttt{0}, & \Delta_i = 0, \\ \texttt{10}\|B_i, & \Delta_i \neq 0 \ \wedge \ L_i = L_{i-1} \ \wedge \ T_i = T_{i-1}, \\ \texttt{11}\|\,\mathrm{bin}_5(L_i)\|\,\mathrm{bin}_6(\ell_i)\|B_i, & \text{trường hợp vùng bit có ý nghĩa thay đổi.} \end{cases} \tag{2.39}$$

[26].

Nhánh thứ nhất chỉ cần một bit điều khiển khi hai giá trị liên tiếp hoàn toàn giống nhau. Nhánh thứ hai tái sử dụng cửa sổ bit của mẫu trước, còn nhánh thứ ba lưu lại vị trí bắt đầu bằng 5 bit, độ dài vùng có ý nghĩa bằng 6 bit và nội dung B_i. Trong hiện thực Gorilla đầy đủ, cửa sổ trước còn có thể được tái sử dụng khi vùng bit mới nằm trọn bên trong cửa sổ cũ; điều kiện bằng nhau trong eq. (2.39) là dạng đặc tả rút gọn dùng trong đồ án.

Mức nén thực tế phụ thuộc đặc tính biến thiên của dữ liệu và không thể quy về một số byte cố định cho mọi điểm đo. Snappy được sử dụng cho các khối chuỗi ký tự, không phải tầng nén bao ngoài cho toàn bộ dữ liệu số. Cơ chế lựa chọn mã hóa theo kiểu dữ liệu của TSM được trình bày trong tài liệu InfluxDB [27].

Minh họa mã hóa khoảng thời gian lấy mẫu. Xét một chuỗi SCADA được phát đều với chu kỳ một giây. Gọi $\Delta_i = t_i - t_{i-1}$ là khoảng cách giữa hai timestamp liên tiếp và $D_i = \Delta_i - \Delta_{i-1}$ là sai phân bậc hai. Kết quả tính toán trên sáu mẫu liên tiếp được trình bày trong table 2.6.

Table 2.6: Minh họa sai phân timestamp khi lấy mẫu đều một giây

Timestamp thực tế t_i	Khoảng cách Δ_i (s)	Delta-of-Delta D_i (s)
10:00:00	–	–
10:00:01	1	–
10:00:02	1	0
10:00:03	1	0
10:00:04	1	0
10:00:05	1	0

Sau hai timestamp đầu tiên, toàn bộ giá trị D_i đều bằng 0. Bảng trên là mô hình trực quan của nguyên lý Delta-of-Delta: thay vì lưu lại một timestamp 64 bit cho từng mẫu, bộ mã hóa chỉ cần biểu diễn quy luật khoảng cách không thay đổi. Trong TSM, cơ chế timestamp thực tế mang tính thích nghi: timestamp được chuyển thành delta, được scale khi có ước số chung thích hợp và được mã hóa run-length khi các delta bằng nhau. Vì vậy, nhịp phát ổn định một giây là trường hợp đặc biệt thuận lợi cho bộ mã hóa timestamp, dù không nên đồng nhất hoàn toàn cơ chế TSM với thuật toán Delta-of-Delta trong bảng minh họa [27].

Minh họa phép XOR của mã hóa Gorilla: Giả sử công suất dự đoán ở hai chu kỳ liên tiếp lần lượt là 1420.0 kW và 1420.5 kW. Biểu diễn IEEE 754 64 bit của hai giá trị này chỉ khác nhau trong một vùng bit rất nhỏ. Phép XOR do Gorilla sử dụng tạo ra kết quả trong fig. 2.6.

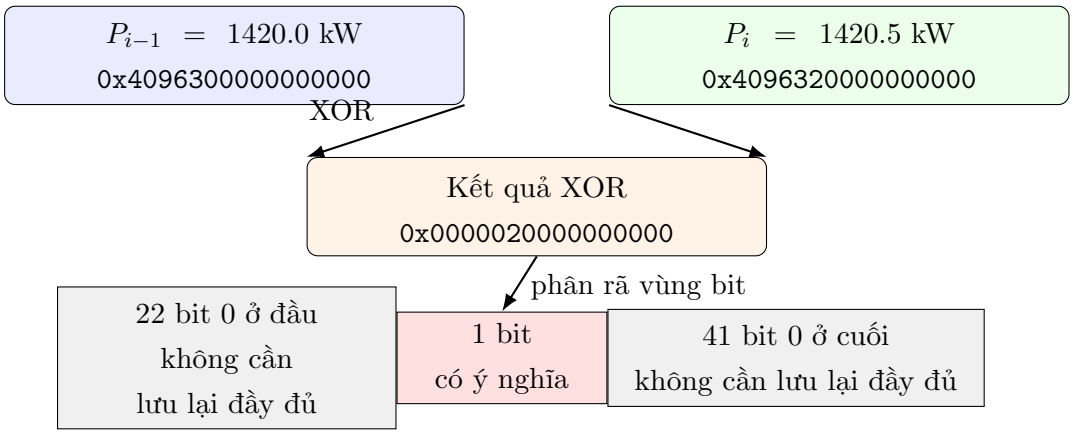


Figure 2.6: Minh họa phép XOR giữa hai giá trị công suất liên tiếp trong mã hóa Gorilla

Ví dụ cho thấy XOR chỉ còn một bit có ý nghĩa nằm giữa 22 bit 0 ở đầu và 41 bit 0 ở cuối. Bộ mã hóa lưu các bit điều khiển mô tả vùng có ý nghĩa cùng phần bit thay đổi, thay vì ghi lại toàn bộ giá trị 64 bit thứ hai. Tín hiệu công suất của tua-bin thường thay đổi liên tục nhưng không nhảy ngẫu nhiên giữa hai chu kỳ kế tiếp, do đó phép XOR thường tạo ra vùng bit cần lưu nhỏ hơn biểu diễn ban đầu. Tuy nhiên, hiệu quả sẽ giảm khi giá trị biến động mạnh hoặc thay đổi không liên tục[27].

2.6 Giao thức truyền thông SMTP/TLS và bảo mật thư điện tử

2.6.1 TLS trên cổng 587

Cấu hình sử dụng `smtp.gmail.com:587`. Với cổng 587, kết nối thường bắt đầu ở dạng SMTP và được nâng cấp sang TLS bằng STARTTLS trước khi truyền thông tin xác thực và nội dung thư. Grafana hỗ trợ chính sách STARTTLS và các thiết lập xác minh chứng chỉ trong nhóm cấu hình SMTP[28;29].

TLS cung cấp tính bí mật và toàn vẹn trên đường truyền, nhưng không thay thế việc quản lý secret. Nếu mật khẩu ứng dụng được ghi trực tiếp trong repository hoặc ảnh báo cáo, người có quyền đọc file vẫn có thể sử dụng credential đó. Vì vậy cấu hình mạng và quản lý secret phải được xem là hai lớp bảo vệ độc lập.

2.6.2 Trình tự SMTP STARTTLS ở mức giao thức

STARTTLS không tạo một kết nối TCP thứ hai. Client trước hết thiết lập phiên SMTP rõ trên cổng 587, kiểm tra server có công bố năng lực STARTTLS, sau đó yêu cầu nâng cấp chính phiên TCP hiện tại thành kênh TLS. Sau khi bắt tay TLS hoàn tất, client phải gửi lại EHLO trong kênh đã mã hóa trước khi xác thực và truyền thư[30].

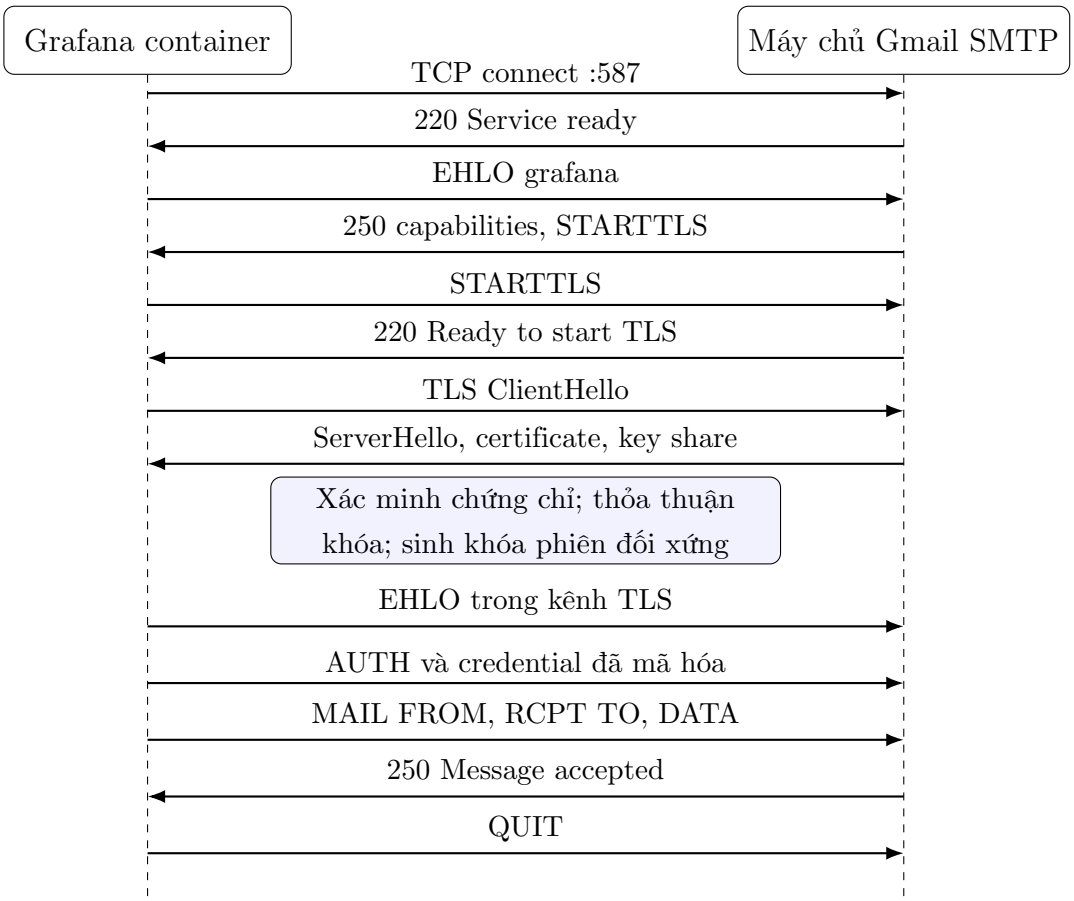


Figure 2.7: Trình tự nâng cấp SMTP sang TLS bằng STARTTLS

Trong TLS hiện đại, mật mã bất đối xứng và chứng chỉ được dùng để xác thực máy chủ và thiết lập bí mật chung, dữ liệu ứng dụng sau đó được bảo vệ bằng khóa phiên đối xứng để đạt hiệu năng cao hơn[31]. Vì vậy có ba thuộc tính cần phân biệt: mã hóa đường truyền, xác thực danh tính máy chủ và xác thực tài khoản SMTP. Cờ `skip_verify=true` có thể vẫn tạo kênh mã hóa nhưng bỏ bước kiểm tra danh tính chứng chỉ, khiến client khó phát hiện máy chủ giả mạo.

2.6.3 Bản chất của App Password

Google App Password là mã 16 ký tự dành cho ứng dụng không hỗ trợ luồng *Sign in with Google*. Nó chỉ khả dụng khi tài khoản đã bật xác minh hai bước. Đây là credential riêng cho ứng dụng, không phải mật khẩu đăng nhập chính và cũng không phải một luồng OAuth2 do Grafana thực hiện[32].

Ưu điểm của App Password là có thể thu hồi riêng mà không thay đổi mật khẩu tài khoản. Hạn chế là ứng dụng vẫn giữ một secret tĩnh, nếu secret bị lộ, người khác có thể

thử xác thực cho đến khi nó bị thu hồi. Google khuyến nghị chỉ dùng App Password khi ứng dụng không hỗ trợ phương thức đăng nhập hiện đại hơn [32].

2.6.4 So sánh App Password và OAuth 2.0

App Password và OAuth 2.0 giải quyết cùng nhu cầu cho ứng dụng truy cập dịch vụ nhưng có mô hình tin cậy khác nhau. OAuth 2.0 cấp access token có thời hạn và có thể dùng refresh token để xin token mới mà không lưu mật khẩu tài khoản tại ứng dụng [33]. Đối với SMTP Gmail, client phải triển khai cơ chế SASL XOAUTH2 và quản lý vòng đời token [34].

Table 2.7: So sánh xác thực SMTP bằng App Password và OAuth 2.0

Tiêu chí	App Password	OAuth 2.0/XOAUTH2
Thông tin xác thực chính	Secret tĩnh 16 ký tự	Access token ngắn hạn và có thể có refresh token
Phạm vi quyền	Gắn với quyền truy cập của ứng dụng/tài khoản	Có scope và consent do authorization server quản lý
Vòng đời	Dùng đến khi bị thu hồi hoặc thay đổi bảo mật	Access token hết hạn, client làm mới theo quy trình
Thu hồi	Thu hồi riêng App Password	Thu hồi grant/token mà không đổi mật khẩu
Lưu trữ	Cần bảo vệ một secret tĩnh	Phải bảo vệ client secret và refresh token
Độ phức tạp	Thấp, phù hợp cấu hình user/password	Cao hơn do authorization, callback và refresh token
Cơ chế SMTP	AUTH qua kênh TLS	SASL XOAUTH2 qua kênh TLS
Rủi ro chính	Secret bị sao chép và dùng lại	Token/refresh token bị đánh cắp hoặc triển khai refresh sai

Nếu access token được cấp tại t_i với thời gian sống T_{TTL}, thời điểm hết hạn là

$$t_e = t_i + T_{TTL}. \tag{2.40}$$

Trước hoặc sau t_e, client dùng refresh token để yêu cầu access token mới. Cơ chế này giới

hạn thời gian hữu ích của một access token bị lộ, nhưng refresh token lại trở thành secret dài hạn cần bảo vệ nghiêm ngặt.

Trong cấu hình thực nghiệm, Grafana SMTP nhận trực tiếp user và password từ biến môi trường nên App Password là phương án tích hợp đơn giản. Cấu hình OAuth dùng để đăng nhập giao diện Grafana không tự động biến SMTP client thành XOAUTH2 client. Muốn dùng OAuth2 thuần túy với Gmail cần một SMTP client hoặc relay có khả năng lấy, làm mới và trình bày token XOAUTH2, đây là phần phức tạp hơn cấu hình Docker Compose hiện tại, chứ không phải hạn chế của bản thân giao thức SMTP.

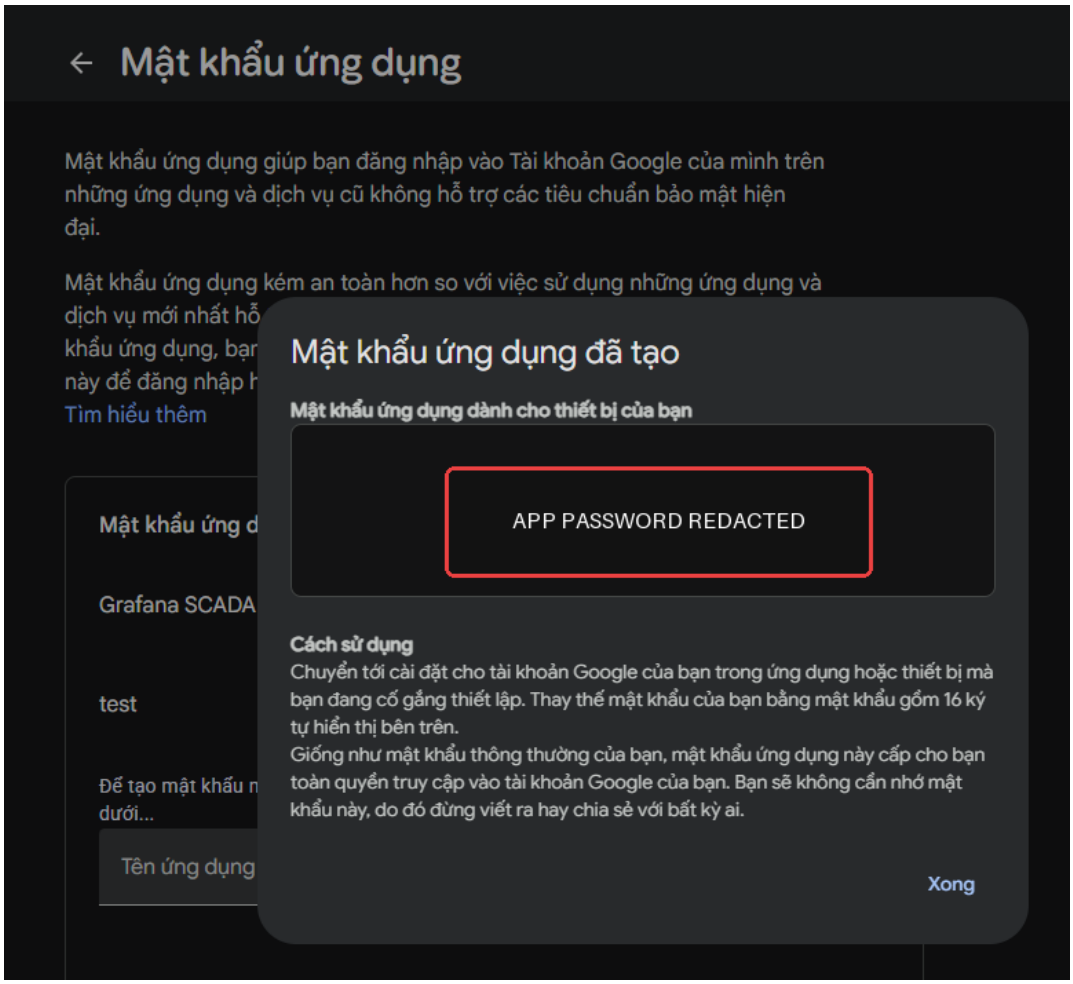


Figure 2.8: Tạo App Password cho Grafana SCADA, khóa đã được che trong báo cáo

Ảnh minh chứng cho thấy App Password đã được tạo cho ứng dụng Grafana SCADA. Bản báo cáo sử dụng ảnh đã che khóa.

Chương 3: Thiết kế kiến trúc hệ thống SCADA Edge

3.1 Kiến trúc phân tầng hệ thống: Biên, Lưu trữ và Giám sát

Trong phạm vi hiện tại của đồ án, hệ thống SCADA giám sát tua-bin gió tích hợp trí tuệ nhân tạo biên được thiết kế và kiểm thử dựa trên mô hình ba lớp phân tán của mạng lưới vạn vật kết nối công nghiệp. Do những giới hạn khách quan về mặt trang thiết bị phần cứng đo lường tại môi trường công nghiệp thực tế, đồ án hiện được vận hành thông qua một luồng dữ liệu giả lập. Việc sử dụng kịch bản phần mềm để mô phỏng dữ liệu không làm giảm đi tính thực tiễn của kiến trúc, mà ngược lại, nó thể hiện tư duy tiếp cận theo hướng bản sao. Phương pháp này cho phép nhóm nghiên cứu tinh chỉnh và đánh giá toàn diện mức độ phản hồi, cũng như độ chính xác của thuật toán học máy trong nhiều kịch bản vận hành khắc nghiệt trước khi tiến hành đấu nối với các cảm biến vật lý thực địa.

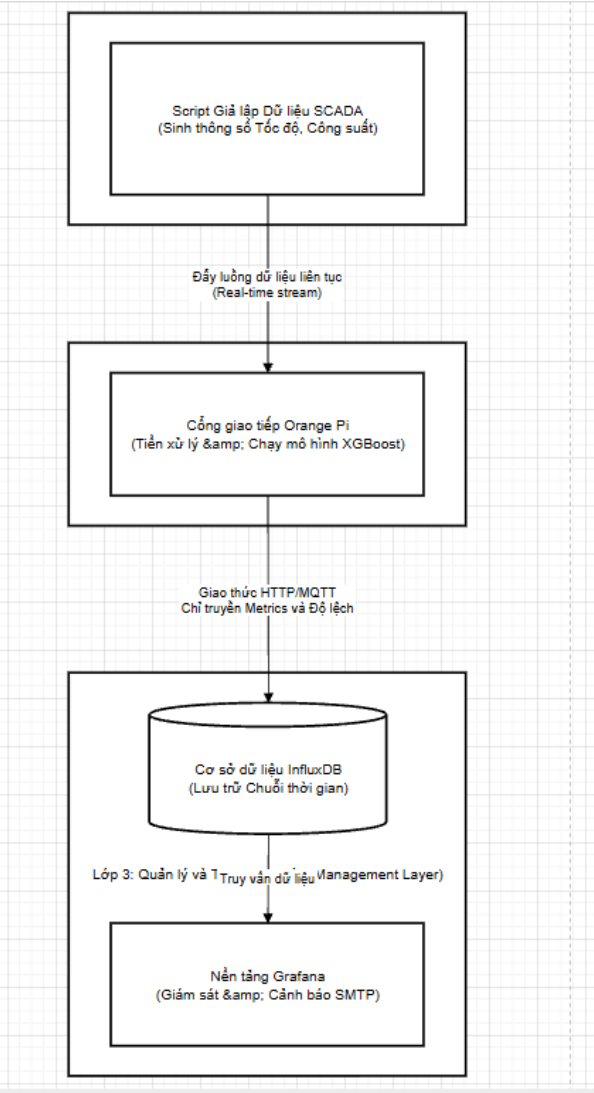


Figure 3.1: Kiến trúc tổng quan

Kiến trúc tổng thể được phân rã thành ba phân vùng độc lập, hoạt động liên kết chặt chẽ với nhau nhằm ngăn ngừa tình trạng thắt cổ chai dữ liệu và cô lập rủi ro tại từng cấp độ.

Tại tầng thấp nhất là Lớp mô phỏng dữ liệu. Phân lớp này đóng vai trò như một mỏ neo đầu vào của toàn bộ hệ thống. Thay vì sử dụng cụm cảm biến cơ học để lấy mẫu, hệ thống sử dụng một kịch bản mã nguồn được lập trình sẵn để sinh ra các trường dữ liệu chuỗi thời gian. Tập dữ liệu này tái tạo lại chính xác các biến thiên vật lý của tua-bin gió trong thực tế, bao gồm các thông số cốt lõi như tốc độ gió môi trường, nhiệt độ hoạt động của hộp số và mức công suất phát điện định mức. Việc kiểm soát hoàn toàn kịch bản đầu vào giúp hệ thống dễ dàng tạo ra các trường hợp sự cố giả định, từ đó kiểm chứng khả

năng phát hiện lỗi của bộ máy học máy ở lớp tiếp theo.

Đóng vai trò cầu nối trung tâm là Lớp xử lý biên. Phân lớp này được vận hành bởi vi máy tính Orange Pi, chịu tải trọng tính toán cao nhất trong cấu trúc hiện tại. Bo mạch này không đơn thuần làm nhiệm vụ chuyển tiếp các gói tin do script sinh ra, mà nó hoạt động như một màng lọc dữ liệu thông minh độc lập. Orange Pi trực tiếp hấp thụ luồng dữ liệu thời gian thực, tiến hành các bước tiền xử lý chuẩn hóa và nạp thẳng vào mô hình suy luận XGBoost. Việc đặt khối tính toán học máy tại đây tạo ra một rào chắn hiệu quả, giúp giữ lại toàn bộ khối lượng dữ liệu thô dư thừa ở hiện trường và chỉ cho phép những thông tin thực sự có giá trị đi tiếp vào mạng lưới diện rộng.

Tại tầng cao nhất là Lớp quản lý và trực quan hóa. Phân lớp này đại diện cho phòng điều khiển trung tâm, nơi toàn bộ các dịch vụ hạ tầng được ảo hóa và vận hành ổn định. Nhiệm vụ của phân lớp này được chia làm hai luồng chính: tiếp nhận các chỉ số độ lệch do Orange Pi gửi về để lưu trữ vĩnh viễn vào cơ sở dữ liệu chuỗi thời gian InfluxDB, đồng thời sử dụng nền tảng Grafana để truy vấn ngược lại khối dữ liệu này. Từ đó, Grafana sẽ vẽ nên các biểu đồ giám sát trực quan và liên tục quét qua các đồ thị để đối chiếu với ngưỡng an toàn, sẵn sàng kích hoạt quy trình gửi thư cảnh báo khẩn cấp khi hệ thống phát hiện sự cố.

Sự phân tách minh bạch giữa ba lớp kiến trúc này đảm bảo rằng trong tương lai, khi đồ án tiến tới giai đoạn tiếp theo, nhóm thực hiện chỉ cần thay thế kịch bản giả lập ở Lớp 1 bằng hệ thống vi điều khiển và cảm biến thực tế mà không cần phải can thiệp vào phần còn lại.

3.1.1 Phân bổ chức năng trên thiết bị biên

Để tối ưu hóa quá trình vận hành, nhóm đã thiết kế kiến trúc triển khai thành ba vùng chứa logic hoạt động độc lập nhưng liên kết chặt chẽ với nhau.

Đầu tiên, vùng chứa AI Exporter đóng vai trò là lõi xử lý, được nhóm lập trình để đọc các mẫu dữ liệu SCADA, tái tạo các đặc trưng thời gian, nạp bộ chuẩn hóa scaler cùng mô hình XGBoost. Sau khi thực hiện suy luận, tiến trình này sẽ tự động đẩy các data points sang cơ sở dữ liệu. Tiếp theo, vùng chứa InfluxDB được nhóm cấu hình để chuyên đảm nhiệm vai trò lưu trữ dữ liệu chuỗi thời gian. Cuối cùng, vùng chứa Grafana được nhóm tích hợp để truy vấn luồng dữ liệu từ InfluxDB, phục vụ công tác trực quan hóa trên Dashboard và liên tục đánh giá các quy tắc cảnh báo Alert Rule.

Về cơ chế định tuyến mạng, nhóm đã chủ động công bố các cổng 8086 của InfluxDB và 3000 của Grafana ra mạng LAN cục bộ. Thiết lập này cho phép máy trạm quản trị có thể dễ dàng truy cập, thao tác và theo dõi hệ thống thông qua giao diện Web và SSH. Ngược

lại, đối với luồng giao tiếp truyền dữ liệu giữa AI Exporter và InfluxDB, nhóm đã thiết lập định tuyến hoàn toàn thông qua mạng ảo nội bộ của Docker Compose bằng service name. Giải pháp này không chỉ giúp hệ thống tự động nhận diện địa chỉ mạng mà còn tăng cường tính bảo mật, tránh việc bộc lộ luồng dữ liệu SCADA nội bộ ra bên ngoài.

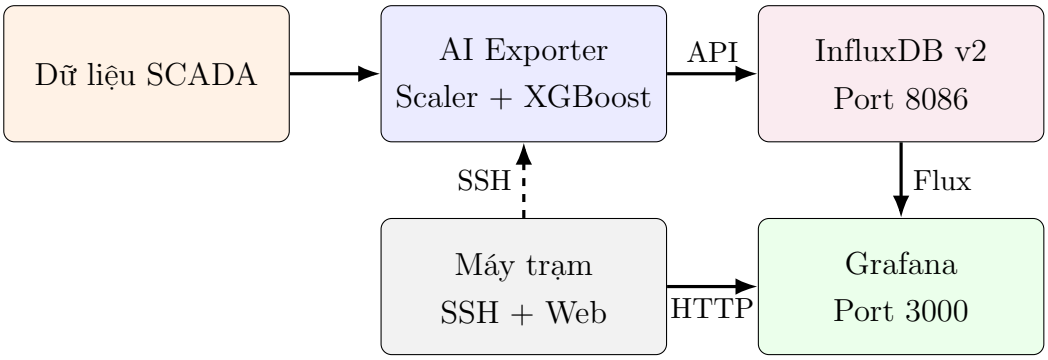


Figure 3.2: Kiến trúc triển khai SCADA–AI trên Orange Pi 5 Plus

3.1.2 Sơ đồ tuần tự của luồng dữ liệu

Để mô tả rõ thứ tự gọi giữa các thành phần, nhóm mô hình hóa một chu kỳ xử lý bằng UML Sequence Diagram trong fig. 3.3. Bốn lifeline tương ứng nguồn dữ liệu `test.csv`, chương trình `stream_exporter.py`, mô hình XGBoost và InfluxDB. Các phép tiền xử lý, so sánh ngưỡng và đóng gói point là self-call tại exporter vì được thực hiện trong cùng tiến trình Python.

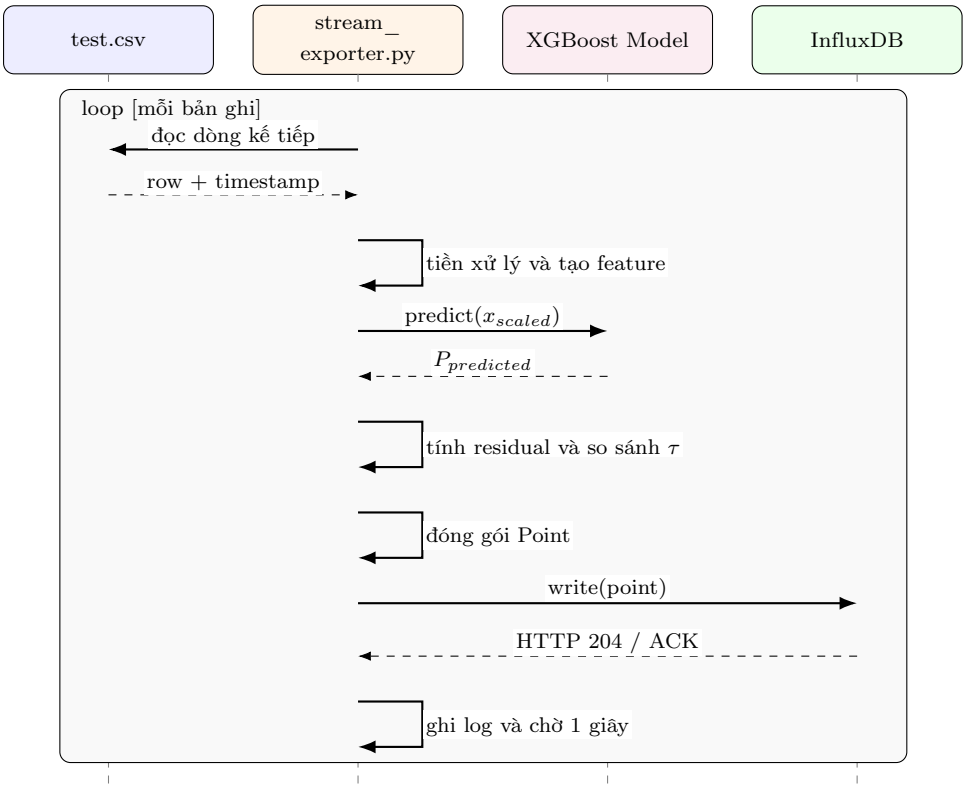


Figure 3.3: UML Sequence Diagram cho một chu kỳ suy luận và ghi dữ liệu

Sơ đồ làm rõ chức năng tệp CSV chỉ cung cấp bản ghi, mô hình chỉ trả kết quả dự báo, còn exporter chịu trách nhiệm xây vector đầu vào, áp dụng ngưỡng và giao tiếp với InfluxDB. Với chế độ ghi đồng bộ, vòng lặp chỉ chuyển sang bản ghi tiếp theo sau khi lời gọi ghi hoàn tất hoặc phát sinh ngoại lệ. Khi bổ sung retry, nhánh lỗi cần được mô hình hóa riêng để tránh lặp vô hạn và tránh ghi trùng point.

3.2 Phân tích phần cứng biên Orange Pi 5 Plus

Orange Pi 5 Plus sử dụng SoC Rockchip RK3588 với hai cụm lõi ARM64 không đồng nhất: bốn Cortex-A76 hiệu năng cao và bốn Cortex-A55 tiết kiệm năng lượng. Cách tổ chức big.LITTLE cho phép bộ lập lịch ưu tiên tải suy luận cho cụm hiệu năng cao, trong khi các luồng nền và I/O có thể sử dụng cụm tiết kiệm năng lượng[35?]. Các thông số liên quan trực tiếp đến bài toán SCADA Edge được chuẩn hóa trong table 3.1.

Table 3.1: Đặc tả phần cứng Orange Pi 5 Plus phục vụ SCADA Edge

Thành phần phần cứng	Thông số kỹ thuật chi tiết	Phân luồng nhiệm vụ trong đồ án	Ưu thế so với Raspberry Pi 4/PC
Cụm CPU hiệu năng cao	4× Cortex-A76, tối đa 2.4 GHz	Tiền xử lý, lượng giác hóa đặc trưng và suy luận XGBoost	Năng lực xử lý đơn luồng và đa luồng cao hơn lớp Cortex-A72 của Raspberry Pi 4; tiêu thụ điện thấp hơn PC phổ thông
Cụm CPU tiết kiệm năng lượng	4× Cortex-A55, tối đa 1.8 GHz	HTTP API, ghi WAL, truy vấn nền và phục vụ giao diện Grafana	Tách tải nền khỏi luồng suy luận và giảm chi phí năng lượng khi tải thấp
Bộ nhớ	LPDDR4/LPDDR4X, cấu hình thử nghiệm 16 GB	Cache InfluxDB, tiến trình Grafana và artifact mô hình	Dung lượng đủ cho ba dịch vụ mà không cần RAM kiểu PC có kích thước và công suất lớn hơn
M.2 M-Key	NVMe 2280, PCIe 3.0 ×4, thông lượng công bố tới khoảng 2000 MB s^{-1}	Lưu WAL, tệp TSM và dữ liệu Grafana bền vững	IOPS và độ trễ ghi phù hợp hơn microSD cho workload cơ sở dữ liệu liên tục
Mạng Ethernet	Hai cổng 2.5 Gbit s^{-1}	Tách mạng trường SCADA và mạng quản trị/trung tâm khi triển khai thực địa	Cho phép phân vùng vật lý mà không cần bổ sung card mạng USB

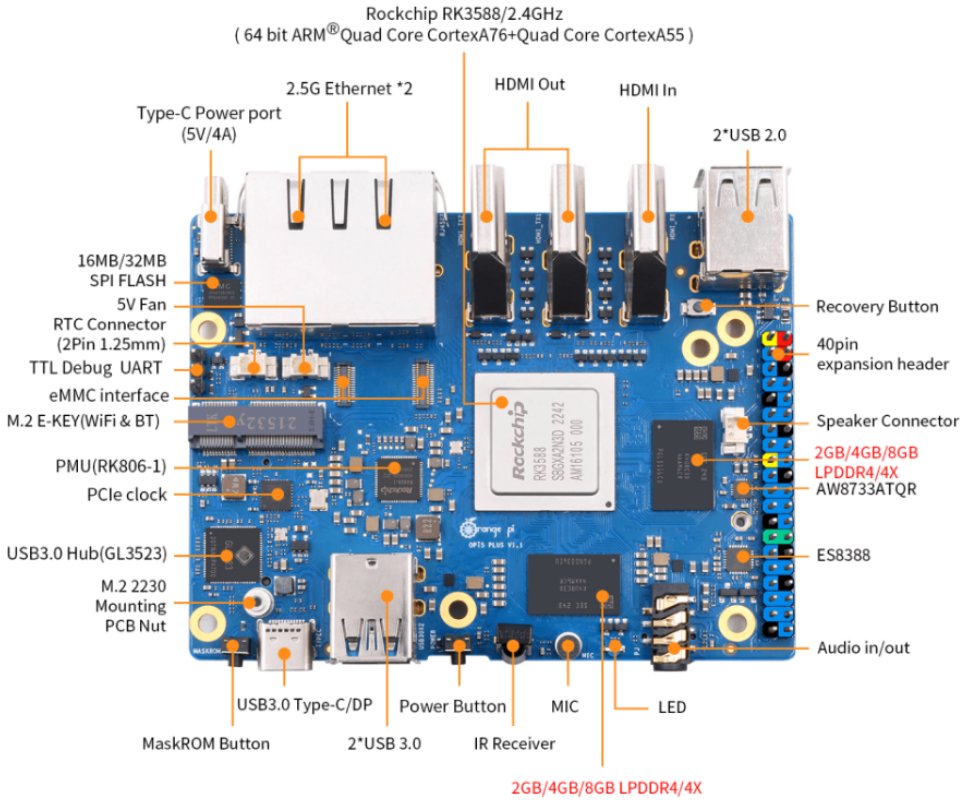


Figure 3.4: Mặt trên Orange Pi 5 Plus và các giao tiếp phần cứng phục vụ SCADA Edge

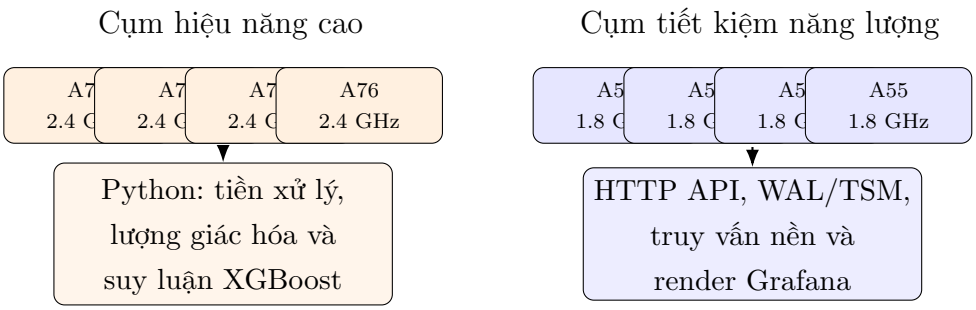


Figure 3.5: Phân luồng nhiệm vụ đề xuất trên kiến trúc big.LITTLE của RK3588

Nếu có K tua-bin, tần số ghi f_s, kích thước trung bình S_{point} của một Point và hệ số khuếch đại ghi WAF, tải ghi lý thuyết trong một ngày là

$$D_{\text{day}} = K f_s S_{\text{point}}\,\text{WAF} \times 86400 \quad \text{(bytes/day)} \quad [?]. \tag{3.1}$$

Đại lượng này là cơ sở để chọn dung lượng và độ bền ổ đĩa. WAF phải được đo từ khối

lượng ghi thực tế của WAL, compaction TSM và filesystem; vì vậy báo cáo không giả định một tuổi thọ cố định cho microSD. NVMe được ưu tiên nhờ giao tiếp PCIe, IOPS và khả năng công bố chỉ số độ bền ghi, còn microSD chỉ phù hợp hơn cho boot hoặc thử nghiệm tải thấp.

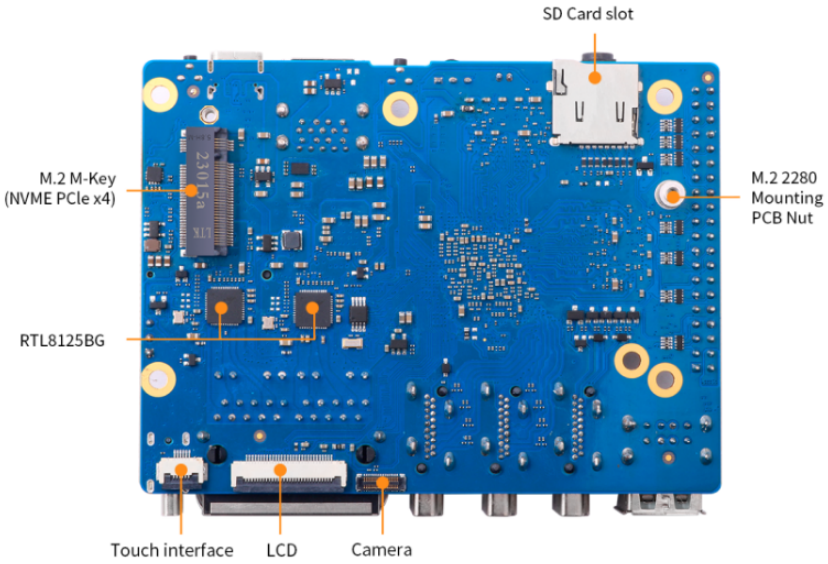


Figure 3.6: Mặt dưới bo mạch với khe M.2 M-Key dành cho SSD NVMe

3.3 Container hóa hạ tầng bằng Docker Compose và phân bổ tài nguyên tĩnh

3.3.1 Cơ chế hoạt động của ảo hóa mức hệ điều hành

Khác với kiến trúc máy ảo truyền thống đòi hỏi hypervisor và hệ điều hành khách riêng, container chia sẻ kernel của hệ điều hành máy chủ.

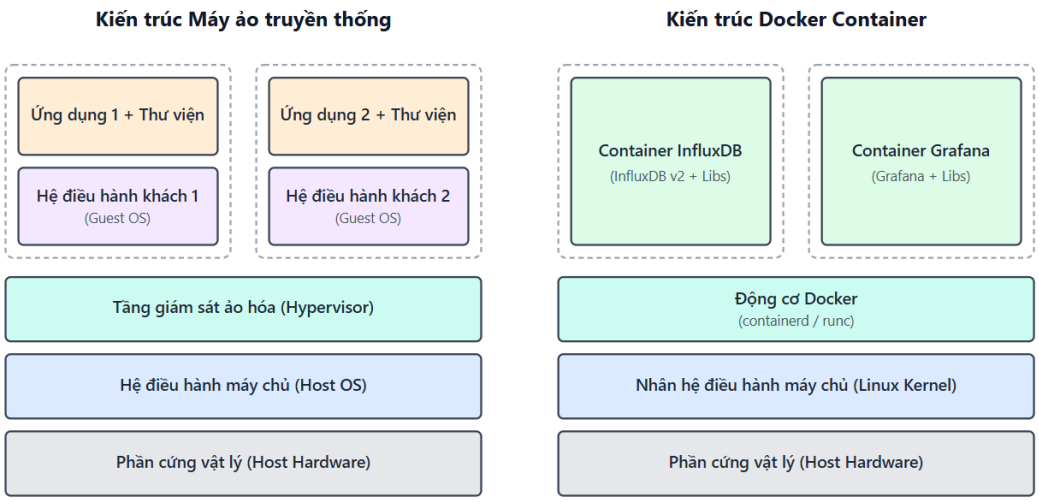


Figure 3.7: So sánh cấu trúc phân tầng giữa máy ảo và container, Việt hóa từ tài liệu Docker [5]

Giải pháp này dựa trên hai cơ chế cốt lõi của Linux kernel:

- Nhóm điều khiển cgroups: Theo dõi, phân bổ và giới hạn CPU, RAM cùng băng thông I/O cho từng nhóm tiến trình, qua đó hạn chế một container chiếm toàn bộ tài nguyên của host.
- Namespaces: Cô lập các container theo không gian tiến trình, giao diện mạng, bảng định tuyến, mount point, hostname và cơ chế IPC.

3.3.2 Đặc thù triển khai trên thiết bị tính toán biên

Hạ tầng giám sát phân tán tại biên đòi hỏi hệ thống phải đạt tốc độ khởi động nhanh, mức chiếm dụng bộ nhớ tĩnh thấp và khả năng tái lập cao. Việc sử dụng container giúp rút ngắn thời gian khởi động dịch vụ xuống còn vài giây, đồng thời giảm thiểu lượng tài nguyên bộ nhớ bị lãng phí cho các tiến trình nền của hệ điều hành khách.

3.3.3 Kế hoạch lưu trữ dữ liệu

Tầng ghi tạm của container không được dùng cho dữ liệu cần tồn tại sau khi tạo lại service. InfluxDB và Grafana sử dụng named volume do Docker quản lý để bảo toàn dữ liệu và quyền truy cập; AI Exporter dùng bind mount chỉ đọc đối với artifact mô hình và cấu hình. `tmpfs` chỉ phù hợp với dữ liệu tạm không cần phục hồi. Phân bổ cụ thể được tích hợp vào ma trận tài nguyên ở table 3.2.

3.3.4 Đặc tả triển khai các dịch vụ bằng Docker Compose

Docker Compose sử dụng một tệp YAML để khai báo service, network, volume và biến môi trường của ứng dụng nhiều container[36]. Thay vì lặp lại cấu hình YAML trong nội dung chính, các thuộc tính quyết định kiến trúc được chuẩn hóa thành hai ma trận đối chiếu. Table 3.2 tổng hợp hạn ngạch CPU, RAM, chính sách khởi động lại và kiểu lưu trữ của từng dịch vụ; table 3.3 đặc tả DNS nội bộ, cổng container, ánh xạ NAT và phạm vi truy cập. Hai bảng này là biểu diễn cô đọng của các nhóm thuộc tính `deploy.resources`, `restart`, `volumes`, `networks` và `ports`. Tệp Compose đầy đủ chỉ được giữ trong Phụ lục A để phục vụ tái lập triển khai.

Dịch vụ cơ sở dữ liệu chuỗi thời gian: InfluxDB sử dụng image phiên bản 2.7 và công bố cổng 8086 từ container ra host. Named volume `influxdb_data` được gắn tại `/var/lib/influxdb2` để dữ liệu không phụ thuộc vòng đời container. Truyền cổng chỉ cung cấp đường truy cập mạng, quyền ghi dữ liệu vẫn phải được kiểm soát bằng organization, bucket và token.

Thứ tự khởi động và khởi động lại: Thuộc tính `depends_on` thiết lập thứ tự khởi tạo container nhưng không tự chứng minh InfluxDB đã sẵn sàng nhận truy vấn. Để kiểm soát readiness chặt chẽ hơn, cấu hình cần bổ sung healthcheck và điều kiện phụ thuộc vào trạng thái healthy. Chính sách `restart=always` yêu cầu Docker khởi động lại tiến trình sau crash hoặc sau khi daemon hoạt động trở lại, đây là cơ chế phục hồi dịch vụ cơ bản, không tương đương kiến trúc tính sẵn sàng cao vì hệ thống vẫn chỉ có một instance cho mỗi service.

3.3.5 Tối ưu hóa tài nguyên Docker trên thiết bị biên

Theo mặc định, container có thể sử dụng tài nguyên mà kernel của host cho phép. Docker chuyển giới hạn CPU và bộ nhớ xuống Linux cgroups để ngăn một dịch vụ chiếm toàn bộ tài nguyên của Orange Pi[37;38].

Với dịch vụ s, tỷ lệ hạn ngạch CPU được xác định bởi thời gian chạy tối đa trong mỗi chu kỳ CFS:

$$q_s = \frac{\tau_{\text{quota},s}}{\tau_{\text{period}}} = \text{CPU_limit}_s \qquad [39;40]. \tag{3.2}$$

Để tổng giới hạn khai báo không vượt số lõi logic trên thiết bị, cấu hình được kiểm tra theo

$$\sum_{s \in \mathcal{S}} \text{CPU_limit}_s = 2.0 + 1.0 + 2.0 = 5.0 \leq N_{\text{cores}} = 8. \tag{3.3}$$



Ba lõi tương đương còn lại tạo khoảng dự phòng cho kernel, IRQ và I/O; đây là ngân sách cấu hình, không phải bảo đảm rằng mọi dịch vụ luôn nhận đúng số lõi vật lý hoặc không phát sinh độ trễ.

Bộ nhớ được theo dõi theo tổng bộ nhớ thường trú và cache trang:

$$\mathrm{Mem}_{\mathrm{used},s}(t) = \mathrm{Mem}_{\mathrm{RSS},s}(t) + \mathrm{Mem}_{\mathrm{Cache},s}(t). \tag{3.4}$$

Điều kiện ngân sách bộ nhớ của host là

$$\mathrm{Mem}_{\mathrm{used},s}(t) \le \mathrm{Mem_limit}_s, \tag{3.5}$$

$$\sum_{s\in\mathcal{S}} \mathrm{Mem_limit}_s + \mathrm{Mem}_{\mathrm{OS_reserved}} \le \mathrm{RAM}_{\mathrm{total}}. \tag{3.6}$$

Với cấu hình đề xuất, tổng giới hạn container là 2 GB; cộng 1 GB dự phòng hệ điều hành vẫn nhỏ hơn 16 GB RAM. Khoảng dư này giảm nguy cơ OOM nhưng không loại bỏ hoàn toàn lỗi do memory leak, cache tăng đột biến hoặc tiến trình ngoài container.

Table 3.2: Ma trận phân bổ tài nguyên và chính sách phục hồi container

Dịch vụ	CPU (lõi)	RAM (MB)	Restart policy	Kiểu lưu trữ
`influxdb`	Reserve: 0.50 Limit: 2.0	Reserve: 512 Limit: 1024	`always`	Named volume `influxdb_data`
`grafana`	Reserve: 0.25 Limit: 1.0	Reserve: 256 Limit: 512	`always`	Named volume `grafana_data`
`ai_exporter`	Reserve: 0.50 Limit: 2.0	Reserve: 256 Limit: 512	`unless-stopped`	Bind mount Mô hình và cấu hình

CFS điều tiết thời gian CPU bằng quota, còn memory cgroup áp giới hạn sử dụng và có thể kích hoạt OOM khi tiến trình vượt ranh giới. Hai cơ chế tạo cô lập tài nguyên giữa các container nhưng không tạo High Availability vì mỗi dịch vụ vẫn chỉ có một instance.

Hiệu lực của trường `deploy` phụ thuộc chế độ triển khai và phiên bản Compose đang sử dụng. Trước khi nghiệm thu, nhóm cần chạy `docker compose config`, kiểm tra cấu hình container bằng `docker inspect` và quan sát CPU/RAM qua `docker stats`. Ngưỡng 1 GB và 512 MB là cấu hình ban đầu, giá trị cuối cùng phải được hiệu chỉnh bằng thử nghiệm tải, tỷ lệ ghi dữ liệu và số lượng dashboard thực tế.

3.4 Thiết kế mạng bridge cô lập scada_net và cơ chế phân giải DNS nội bộ

Mạng `scada_net` sử dụng user-defined bridge, dải quy hoạch `172.20.0.0/16` và gateway `172.20.0.1`. Docker gắn mỗi container vào bridge bằng một cặp giao diện `veth` và cung cấp DNS nhúng tại `127.0.0.11`; vì vậy các dịch vụ trao đổi bằng tên thay vì phụ thuộc IP container có thể thay đổi[41;42].

Table 3.3: Quy hoạch không gian mạng và ánh xạ cổng dịch vụ

Dịch vụ	IP bridge đề xuất	DNS Docker nội bộ	Cổng container	NAT trên Orange Pi	Phạm vi truy cập
InfluxDB	`172.20.0.2`	`http://influxdb:8086`	8086/TCP	`192.168.0.105:8086`	Nội bộ và quản trị
Grafana	`172.20.0.3`	`http://grafana:3000`	3000/TCP	`192.168.0.105:3000`	Trình duyệt trong LAN
AI Exporter	`172.20.0.4`	`ai_exporter`	Không công bố	Không ánh xạ	Chỉ giao tiếp nội bộ tới InfluxDB
DNS nhúng	`127.0.0.11`	Bộ phân giải Docker	53/UDP,TCP	Không ánh xạ	Chỉ bên trong network namespace container

Các IP trong bảng là quy hoạch tĩnh đề xuất; nếu Compose không khai báo `ipam` và `ipv4_address`, Docker có thể cấp địa chỉ khác. Cấu hình ứng dụng vẫn phải dùng DNS service name để tránh phụ thuộc các địa chỉ này.

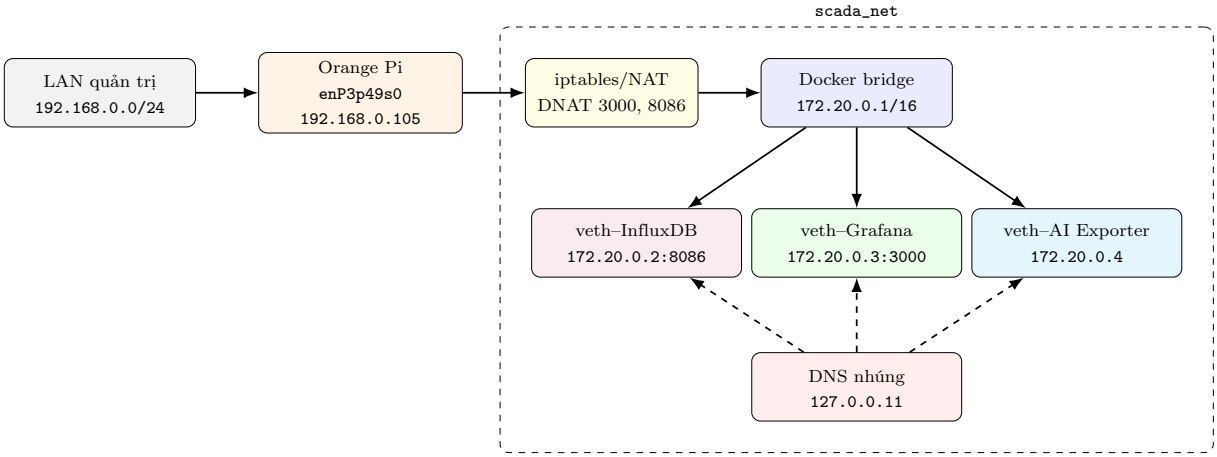


Figure 3.8: Luồng mạng từ giao diện vật lý qua NAT tới bridge và các container

Ánh xạ `8086:8086` và `3000:3000` phục vụ truy cập từ host hoặc các thiết bị bên ngoài. Lưu lượng Grafana–InfluxDB bên trong `scada_net` không đi vòng qua cổng đã publish trên host. Mạng bridge tạo lớp cô lập và định tuyến thuận tiện, nhưng không thay thế xác thực InfluxDB, token truy cập, firewall và chính sách TLS nếu lưu lượng cần đi qua mạng vật lý.

3.5 Bảo mật và quản lý thông tin xác thực qua biến môi trường

Để tuân thủ nguyên tắc tối thiểu quyền, nhóm không sử dụng token toàn quyền cho các ứng dụng ngoại vi. Thay vào đó, mỗi thành phần nhận một token riêng với phạm vi tác vụ đúng bằng nhu cầu vận hành. Tiến trình Python tại thiết bị biên chỉ có quyền ghi vào bucket `turbine_metrics`, nếu token này bị lộ, đối tượng sử dụng trái phép không mặc nhiên có quyền đọc dữ liệu lịch sử hoặc thay đổi tài nguyên quản trị. Grafana sử dụng token chỉ đọc, qua đó ngăn giao diện trực quan hóa thực hiện thao tác ghi hoặc xóa dữ liệu nguồn. Token toàn quyền chỉ dành cho quản trị viên và không được cung cấp cho container ứng dụng.

Table 3.4: Ma trận phân quyền mã xác thực trong hạ tầng giám sát

Thành phần truy cập	Quyền đọc	Quyền ghi	Phạm vi tác vụ
Tiến trình truyền dữ liệu Python	Không	Có	Ghi vào khối lưu trữ turbine
Nguồn dữ liệu Grafana	Có	Không	Đọc từ khối lưu trữ turbine
Quản trị viên hệ thống	Có	Có	Toàn quyền trên tổ chức quản trị

Các token được cung cấp tại thời điểm triển khai qua biến môi trường và không xuất hiện trực tiếp trong mã nguồn. Biện pháp này giảm nguy cơ lộ credential khi chia sẻ repository, nhưng không tạo khả năng bảo vệ tuyệt đối: tệp `.env` vẫn phải nằm ngoài Git, có quyền đọc hạn chế và được thay thế ngay khi nghi ngờ bị lộ. Trong môi trường sản xuất, secret store của nền tảng triển khai là lựa chọn phù hợp hơn việc lưu token trong tệp văn bản.

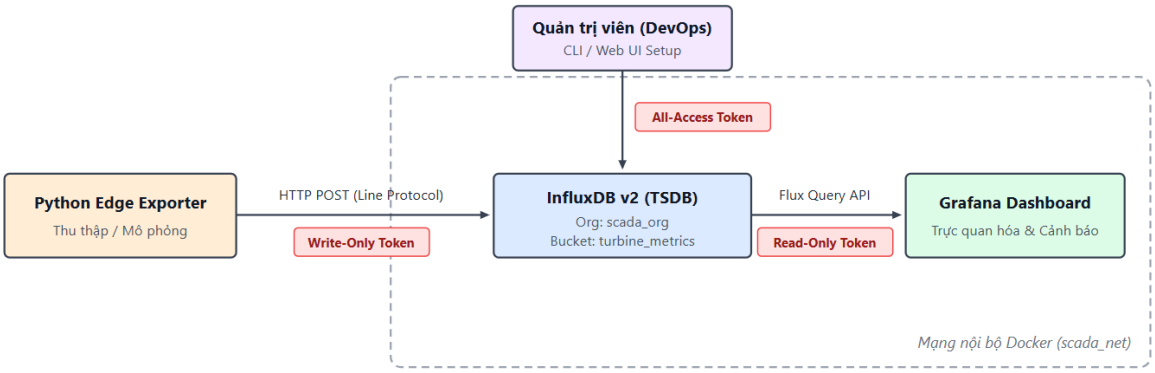


Figure 3.9: Luồng truyền tải dữ liệu và cơ chế xác thực phân quyền giữa các thành phần

Tệp `.env` chỉ tồn tại trên host, nằm ngoài Git và được Compose dùng để nội suy những biến được khai báo rõ trong cấu hình service[43;44]. Sau khi container khởi tạo, tiến trình đọc giá trị từ `os.environ`; `python-dotenv` chỉ cần thiết khi chạy chương trình trực tiếp trên host trong môi trường phát triển[45].

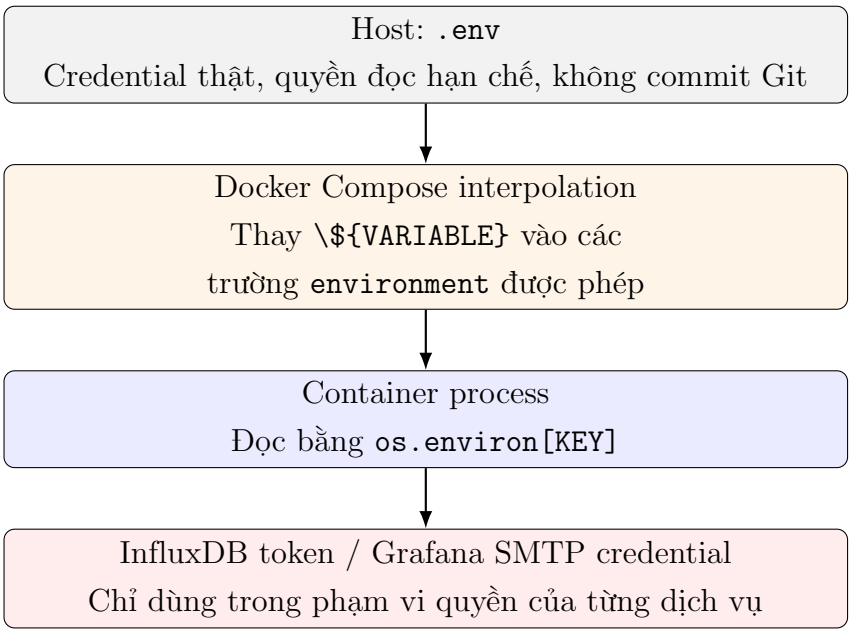


Figure 3.10: Luồng luân chuyển secret từ host tới tiến trình container

Table 3.5: Kiểm soát vòng đời thông tin xác thực

Giai đoạn	Kiểm soát bắt buộc	Phương pháp kiểm chứng
Lưu trên host	Tệp ngoài Git, quyền đọc tối thiểu, không xuất hiện trong ảnh báo cáo	`git status`, `git grep`, quyền tệp
Nội suy Compose	Chỉ truyền biến cần thiết cho đúng service	`docker compose config` với đầu ra đã che
Thực thi container	Tiến trình đọc biến bắt buộc bằng `os.environ[KEY]` và dừng rõ ràng khi thiếu	Log khởi động không chứa giá trị secret
Thu hồi	Đổi token/App Password khi nghi ngờ lộ và khởi tạo lại container	Kiểm thử credential cũ bị từ chối

Chương 4: Hiện thực luồng dữ liệu thời gian thực và mô hình AI tại biên

4.1 Hiện thực pipeline tiền xử lý dữ liệu

4.1.1 Đặc tả tệp dữ liệu lưu trữ

Tệp đầu vào sử dụng định dạng CSV, trong đó các giá trị của cùng một thời điểm nằm trên một hàng và được phân cách bằng dấu phẩy. So với cấu trúc XML hoặc JSON lồng nhau, dạng bảng phẳng này đơn giản hóa quá trình phân tích cú pháp và phù hợp với thao tác đọc tuần tự bằng Pandas. Tuy nhiên, việc nạp toàn bộ tệp vào RAM chỉ phù hợp khi kích thước dữ liệu nằm trong giới hạn bộ nhớ của thiết bị, với dữ liệu lớn hơn, pipeline cần chuyển sang đọc theo từng khối hoặc theo luồng.

Tệp bắt đầu bằng một dòng tiêu đề khai báo tên cột, tiếp theo là các dòng dữ liệu tuần tự. Cấu trúc bảng gồm năm trường cốt lõi của tua-bin gió và không yêu cầu biến đổi cấu trúc phức tạp trước khi đưa vào vòng lặp. Báo cáo chỉ xem đây là định dạng của bộ dữ liệu thử nghiệm, không mặc định rằng mọi hệ thống SCADA công nghiệp đều xuất dữ liệu theo cùng một schema.

Đối chiếu các timestamp liền kề, chẳng hạn 19:00 và 19:10, cho thấy khoảng cách lấy mẫu trong tệp lịch sử là mười phút. Đây là đặc tính của bộ dữ liệu đang sử dụng, không phải tần số lấy mẫu bắt buộc của mọi hệ thống SCADA.

Khoảng cách mười phút chỉ phản ánh nhãn thời gian trong dữ liệu lịch sử. Khi phát lại, kịch bản lấy từng bản ghi theo thứ tự và đưa vào pipeline với nhịp một giây một mẫu. Phép thay đổi này tạo tải tuần tự để kiểm thử hệ thống, nhưng không làm tăng độ phân giải thông tin của dữ liệu gốc và không thay thế một nguồn cảm biến thời gian thực.

Nhóm phân chia các trường dữ liệu thành ba vai trò: trường thời gian, đặc trưng đầu vào và giá trị thực tế dùng để đối chiếu. Việc loại `LV ActivePower (kW)` khỏi véc-tơ đầu vào ngăn mô hình sử dụng trực tiếp giá trị cần dự đoán. Đây là yêu cầu về tính đúng đắn của suy luận, khái niệm rò rỉ dữ liệu chủ yếu cần được kiểm soát ở giai đoạn chia tập và huấn luyện mô hình.

Kết quả thực nghiệm của bước làm sạch được tổng hợp trong table 4.1. Hai luật được áp dụng theo thứ tự ưu tiên: mẫu thuộc luật công suất bằng không được đếm trước, luật suy giảm công suất chỉ đếm các mẫu còn lại. Cách đếm loại trừ này ngăn một mẫu thỏa đồng thời hai điều kiện bị trừ hai lần.

Table 4.1: Thống kê số lượng mẫu bị loại bỏ theo các luật nhận diện nhiễu

Tập dữ liệu	Số mẫu ban đầu	$\mathcal{C}_{\text{zero}}$	$\mathcal{C}_{\text{low_power}}$	Số mẫu sau làm sạch	Tỷ lệ loại bỏ
Tập huấn luyện	35 800	1 821	230	33 749	5.73%
Tập kiểm định	10 650	542	62	10 046	5.67%
Tập kiểm tra	4 725	241	27	4 457	5.67%
Toàn bộ dữ liệu	51 175	2 604	319	48 252	5.71%

4.1.2 Các đặc trưng sử dụng trong suy luận

Trong quá trình xử lý dữ liệu thời gian thực, hệ thống không sử dụng trực tiếp toàn bộ các dữ liệu có trong bản ghi SCADA làm dữ liệu đầu vào cho mô hình học máy. Thay vào đó, các trường cần thiết được lựa chọn và kết hợp với một số đặc trưng được tính toán từ thời gian ghi nhận để tạo thành véc-tơ đầu vào cho mô hình.

Danh sách các giá trị đầu vào được khai báo trong tệp cấu hình `config.json`. Khi chương trình thực thi, `main.py` đọc danh sách này và sử dụng đúng các trường được khai báo để xây dựng véc-tơ đặc trưng. Cách tổ chức này giúp thống nhất dữ liệu đầu vào giữa quá trình huấn luyện mô hình và quá trình suy luận trên thiết bị biên.

Các đặc trưng đầu vào được chia thành hai nhóm. Nhóm thứ nhất gồm các đặc trưng được lấy trực tiếp từ dữ liệu SCADA:

1. `Wind Speed (m/s)`: Tốc độ gió đo được tại tua-bin, là một trong những thông số có ảnh hưởng trực tiếp đến công suất phát của tua-bin.

2. `Theoretical_Power_Curve (KWh)`: Công suất lý thuyết tương ứng với điều kiện vận hành, được sử dụng làm thông tin tham chiếu cho công suất thực tế của tua-bin.

3. `Wind Direction (?)`: Hướng gió được ghi nhận tại thời điểm đo, cung cấp thông tin về hướng tác động của luồng gió lên tua-bin.

Nhóm thứ hai gồm các đặc trưng thời gian được hệ thống tính toán từ trường `Date/Time`. Các đặc trưng này sử dụng hàm lượng giác để biểu diễn tính chu kỳ của thời gian:

4. `Day_sin`: Giá trị sin biểu diễn vị trí của thời điểm trong chu kỳ một ngày.

5. `Day_cos`: Giá trị cos biểu diễn vị trí của thời điểm trong chu kỳ một ngày.

6. `Month_sin`: Giá trị sin biểu diễn vị trí của thời điểm trong chu kỳ một năm.

7. `Month_cos`: Giá trị cos biểu diễn vị trí của thời điểm trong chu kỳ một năm.

Quá trình tiền xử lý được hình thức hóa thành ánh xạ từ bản ghi quan trắc thô sang véc-tơ suy luận bảy chiều:

$$\Phi : \mathbf{s}_t = \begin{bmatrix} t & v_t & P_{\text{theo},t} & \theta_t & P_{\text{act},t} \end{bmatrix}^T \longmapsto \mathbf{x}_t = \begin{bmatrix} v_t \\ P_{\text{theo},t} \\ \theta_t \\ \sin(2\pi m_t/1440) \\ \cos(2\pi m_t/1440) \\ \sin(2\pi q_t/12) \\ \cos(2\pi q_t/12) \end{bmatrix} \in \mathbb{R}^7. \tag{4.1}$$

Trong đó $m_t = 60h_t + \text{minute}_t$ và q_t là tháng. $P_{\text{act},t}$ không xuất hiện trong $\mathbf{x}_t$ mà chỉ được giữ làm biến mục tiêu và dùng để tính residual sau suy luận. Thứ tự bảy thành phần được cố định trong `config.json`.

Table 4.2: Thống kê mô tả các biến trong tập dữ liệu sau làm sạch

Biến	Đơn vị	Mean	Std	Min	25%	Median	75%	Max
Wind Speed (v)	m/s	7.55	4.22	0.00	4.14	7.25	10.30	25.00
Theoretical Power (P_{theo})	kWh	1492.17	1466.01	0.00	161.86	1063.77	3195.89	3600.00
Wind Direction (θ)	độ	123.64	93.44	0.00	49.31	73.05	201.70	359.99
`Day_sin`	–	0.00	0.71	-1.00	-0.71	0.00	0.71	1.00
`Day_cos`	–	0.00	0.71	-1.00	-0.71	0.00	0.71	1.00
`Month_sin`	–	0.04	0.70	-1.00	-0.50	0.00	0.87	1.00
`Month_cos`	–	-0.08	0.71	-1.00	-0.87	0.00	0.50	1.00
Target: P_{act}	kW	1307.68	1319.72	0.00	109.18	825.84	2482.35	3600.00

Table 4.3: Hệ số tương quan Pearson đối với công suất thực tế P_{act}

Biến đầu vào	Pearson r	Mức tương quan	Ý nghĩa vật lý
Theoretical Power Curve	+0.949	Rất mạnh, dương	Giá trị tham chiếu theo đường cong công suất lý thuyết
`Wind Speed`	+0.912	Rất mạnh, dương	Đại diện năng lượng động học của luồng gió
`Wind Direction`	−0.063	Yếu, âm	Phản ánh hướng đón gió và điều kiện yaw
`Day_sin/Day_cos`	+0.012/ − 0.018	Rất yếu	Biến thiên tuần hoàn trong ngày
`Month_sin/Month_cos`	−0.041/ + 0.078	Yếu	Thành phần mùa vụ và điều kiện khí quyển

Mối liên hệ tuyến tính được lượng hóa bằng hệ số tương quan Pearson[46]. Kết quả cho thấy đường cong công suất lý thuyết và vận tốc gió là hai biến chi phối, phù hợp với các mô hình giám sát công suất trang trại gió dựa trên dữ liệu[47]. Tương quan không đồng nghĩa quan hệ nhân quả và không phản ánh đầy đủ các tương tác phi tuyến mà XGBoost có thể học.

4.1.3 Xử lý đặc trưng chu kỳ thời gian

Bốn tọa độ lượng giác trong eq. (4.1) bảo toàn tính lân cận qua ranh giới ngày và năm. Toàn bộ mã hiện thực hàm `preprocess_datetime` cùng danh sách feature được chuyển sang Phụ lục A; phần nội dung chính chỉ giữ đặc tả toán học và hợp đồng thứ tự đặc trưng.

4.1.4 Tiền xử lý nhiễu hệ thống

Gọi tập dữ liệu thô là

$$\mathcal{D} = \{(v_i, P_i, P_{\text{theo},i}, \theta_i)\}_{i=1}^{N}. \tag{4.2}$$

Luật thứ nhất nhận diện trạng thái có gió trên cut-in nhưng công suất gần bằng không:

$$\mathcal{C}_{\text{zero}} = \left\{ i \in \mathcal{D} \mid v_i > 3.5\,\text{m}\,\text{s}^{-1} \wedge P_i \leq 0.1\,\text{kW} \right\}. \tag{4.3}$$

Để các thống kê loại bỏ không đếm trùng, luật suy giảm công suất được áp dụng trên phần còn lại:

$$\mathcal{C}_{\text{low_power}} = \left\{ i \in \mathcal{D} \setminus \mathcal{C}_{\text{zero}} \mid v_i > 14\,\text{m}\,\text{s}^{-1} \wedge P_i < 1500\,\text{kW} \right\}. \tag{4.4}$$

Tập dùng để huấn luyện đường cong công suất cơ sở và kích thước của nó là

$$\mathcal{D}_{\text{clean}} = \mathcal{D} \setminus \left(\mathcal{C}_{\text{zero}} \cup \mathcal{C}_{\text{low_power}} \right), \tag{4.5}$$

$$N_{\text{clean}} = N - N_{\text{zero}} - N_{\text{low_power}}. \tag{4.6}$$

Các luật chỉ xác định mẫu không phù hợp để học trạng thái phát điện bình thường; chúng không chứng minh chắc chắn lỗi thiết bị. Khi có mã trạng thái pitch, phanh hoặc bảo trì, phương án tốt hơn là phân lớp chế độ vận hành thay vì loại mẫu cứng.

4.1.5 Chuẩn hóa và phân chia dữ liệu theo thời gian

MinMaxScaler biến đổi đặc trưng x_j theo

$$x'_j = \frac{x_j - x_{j,\min}}{x_{j,\max} - x_{j,\min}}. \tag{4.7}$$

Ở dạng véc-tơ, phép biến đổi là

$$\mathbf{x}'_t = \mathbf{W}_{\text{scale}} \left(\mathbf{x}_t - \mathbf{x}_{\min} \right), \qquad \mathbf{W}_{\text{scale}} = \text{diag}\left(\frac{1}{x_{\max,j} - x_{\min,j}} \right)_{j=1}^{7} \quad [48;49]. \tag{4.8}$$

Scaler chỉ được fit trên tập train, sau đó tái sử dụng cho validation và test. XGBoost không bắt buộc chuẩn hóa vì phép chia cây ít nhạy với biến đổi đơn điệu; scaler vẫn được giữ để bảo đảm hợp đồng artifact giữa huấn luyện và suy luận. Mã fit và lưu scaler được chuyển sang Phụ lục A.

Tập dữ liệu được chia theo thứ tự thời gian thay vì trộn ngẫu nhiên để tránh sử dụng thông tin tương lai trong quá trình huấn luyện. Quy mô ba tập được trình bày trong table 4.4.

Table 4.4: Phân chia dữ liệu huấn luyện và đánh giá

Tập dữ liệu	Số mẫu	Vai trò
Train	33 749	Fit scaler và mô hình
Validation	10 046	Theo dõi sai số và early stopping
Test	4 457	Đánh giá cuối cùng

4.1.6 Pipeline tiền xử lý tổng thể

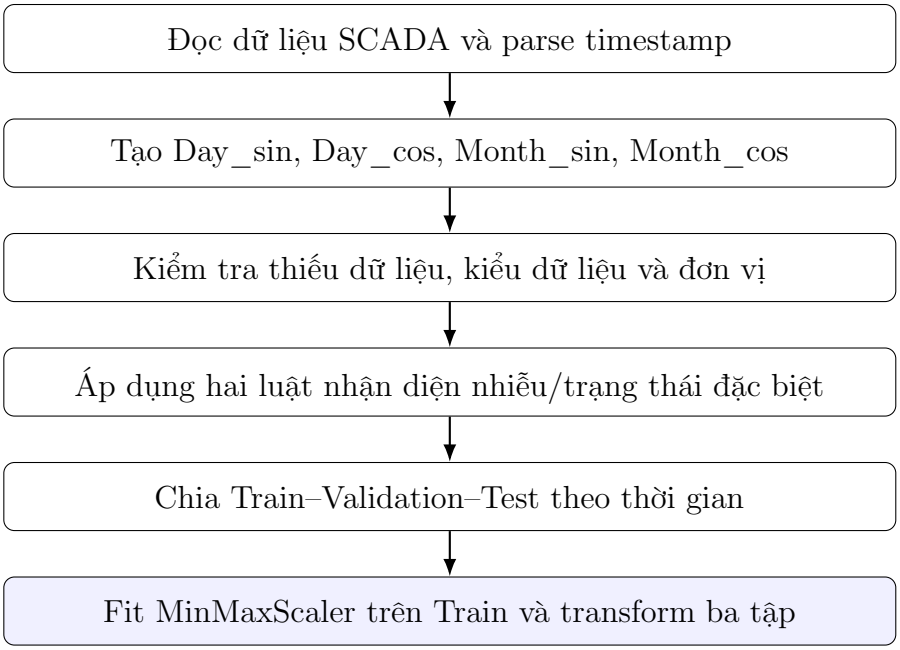


Figure 4.1: Pipeline tiền xử lý dữ liệu

4.2 Huấn luyện, tối ưu tham số và cơ chế dừng sớm cho XGBoost

4.2.1 Bộ tham số được sử dụng trong mã nguồn

Trong giai đoạn này, nhóm tiến hành khởi tạo mô hình `XGBRegressor` bằng một bộ tham số cố định được trình bày trong table 4.5. Việc thiết lập cấu hình cơ sở này nhằm giúp nhóm kiểm chứng tính đúng đắn và độ ổn định của toàn bộ pipeline tiền xử lý trước khi bước vào các giai đoạn tinh chỉnh mô hình phức tạp hơn trong quá trình phát triển đồ án.

Table 4.5: Cấu hình XGBoost thực tế trong xgboost_train.py

Tham số	Giá trị	Ý nghĩa
`n_estimators`	2000	Giới hạn trên của số vòng boosting
`learning_rate`	0.03	Mức đóng góp của mỗi cây
`max_depth`	6	Độ sâu tối đa của cây
`subsample`	0.8	Dùng ngẫu nhiên 80% số hàng cho mỗi cây
`colsample_bytree`	0.8	Dùng ngẫu nhiên 80% đặc trưng cho mỗi cây
`objective`	`reg:squarederror`	Tối thiểu hóa sai số bình phương
`tree_method`	`hist`	Xây cây bằng histogram
`random_state`	42	Cố định nguồn ngẫu nhiên để tái lập
`early_stopping_rounds`	50	Dừng nếu validation không cải thiện trong 50 vòng

Khối mã khởi tạo và huấn luyện mô hình được chuyển sang Phụ lục A; bảng trên giữ lại toàn bộ siêu tham số cần thiết để tái lập cấu hình.

4.2.2 Cơ chế early stopping

Mô hình được phép xây tối đa 2000 cây nhưng sử dụng tập validation trong `eval_set`. Nếu chỉ số validation không cải thiện qua 50 vòng liên tiếp, huấn luyện kết thúc và mô hình giữ vòng tốt nhất. Cơ chế này tránh phải ấn định chính xác số cây từ đầu và hạn chế việc tiếp tục khớp nhiễu sau khi khả năng tổng quát hóa đã bão hòa.

Theo log thực nghiệm, quá trình huấn luyện kết thúc tại vòng 183 với RMSE validation được ghi nhận bằng 246.44 kW. Như vậy, hệ thống chỉ sử dụng khoảng 9.2% giới hạn 2000 cây. Đường diễn biến trong fig. 4.2 cho thấy RMSE đạt vùng cực tiểu xấp xỉ vòng 133, sau đó mô hình tiếp tục theo dõi thêm 50 vòng theo tham số patience và dừng tại vòng 183 do không xuất hiện cải thiện đáng kể. Vì vậy, vòng 133 biểu diễn mốc chất lượng validation tốt nhất, còn vòng 183 là thời điểm cơ chế early stopping chấm dứt quá trình huấn luyện đối với cấu hình baseline hiện tại của nhóm.

Table 4.6: Tóm tắt kết quả quá trình huấn luyện

Thuộc tính	Giá trị
Số cây tối đa	2000
Patience của early stopping	50 vòng
Vòng đạt vùng RMSE tốt nhất	Xấp xỉ 133
Vòng kết thúc huấn luyện	183
RMSE validation được ghi tại vòng 183	246.44 kW
Phương pháp xây cây	Histogram

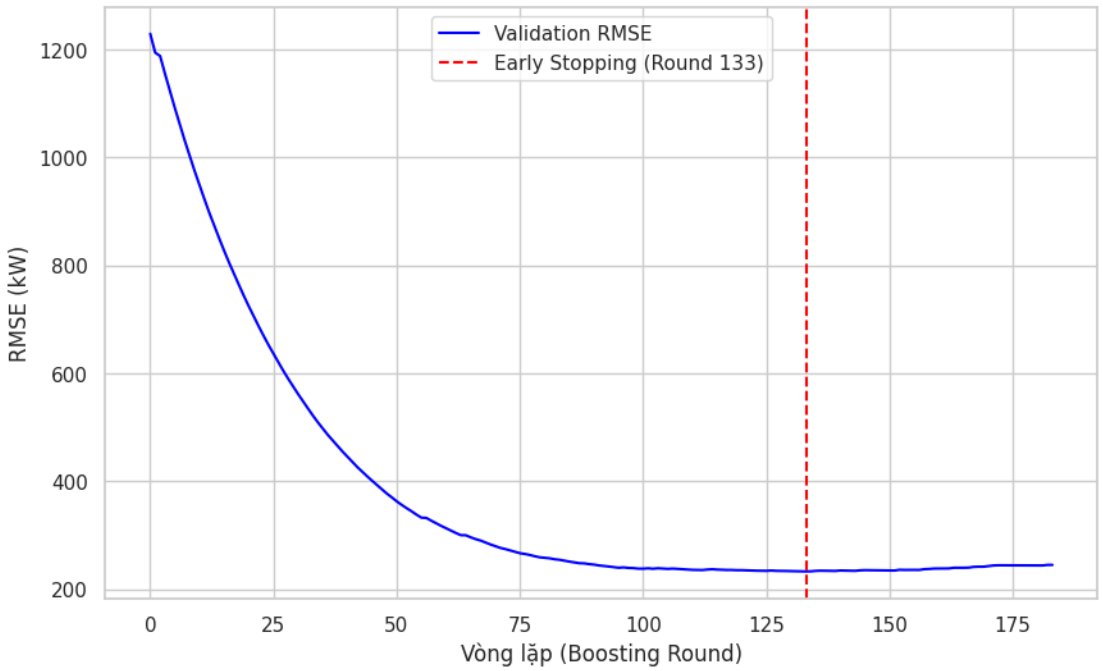


Figure 4.2: Diễn biến RMSE validation và mốc chất lượng tốt nhất trước khi dừng sớm

4.2.3 Các độ đo hồi quy

Sai số tuyệt đối trung bình được xác định bởi

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \,. \tag{4.9}$$

MAE có cùng đơn vị với công suất và cho mỗi sai số một trọng số tuyến tính. Sai số căn trung bình bình phương được xác định bởi

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}. \tag{4.10}$$

Do bình phương độ lệch, RMSE nhạy hơn với outlier. Việc sử dụng đồng thời MAE và RMSE cung cấp hai góc nhìn về sai số điển hình và ảnh hưởng của các độ lệch lớn[50;51;52]. Trường hợp RMSE lớn hơn MAE đáng kể cho thấy phân phối sai số có phần đuôi lớn, đây có thể là bất thường thật, chế độ điều khiển đặc biệt hoặc dữ liệu nhiễu.

Table 4.7: So sánh MAE và RMSE trong bài toán dự báo công suất

Tiêu chí	MAE	RMSE
Trọng số sai số	Tuyến tính	Bình phương trước khi lấy trung bình
Độ nhạy với outlier	Thấp hơn	Cao hơn
Đơn vị	kW	kW
Vai trò	Sai lệch điển hình	Nhấn mạnh các sự kiện sai lệch lớn

4.3 Thiết kế cấu trúc thư mục và đóng gói artifact

Nhóm tổ chức mã nguồn theo trách nhiệm của từng tệp để tách cấu hình triển khai, artifact học máy, dữ liệu kiểm thử và chương trình thực thi. Cách bố trí này chưa được xem là kiến trúc microservices hoàn chỉnh, đây là cấu trúc mô-đun ở mức tệp, phù hợp với giai đoạn kiểm chứng luồng dữ liệu giả lập. Các tệp được đặt trong cùng workspace để đơn giản hóa đường dẫn khi chạy thử trên thiết bị biên.

Table 4.8: Phân định vai trò các module và tệp triển khai tại Edge Node

Nhóm chức năng	Tệp hoặc artifact	Vai trò kỹ thuật	Yêu cầu vận hành
Điều phối luồng	`stream_exporter.py`	Tạo đặc trưng, nạp mô hình, suy luận và ghi Point vào InfluxDB	Chạy liên tục; dừng rõ ràng khi thiếu cấu hình hoặc artifact
Hợp đồng suy luận	`config.json`	Lưu thứ tự bảy đặc trưng, biến mục tiêu và ngưỡng residual	Phải đồng bộ với model và scaler cùng phiên bản
Artifact học máy	`scaler.joblib`; `xgboost_model.json`	Lưu tham số chuẩn hóa, cấu trúc cây và trọng số đã huấn luyện	Chỉ đọc tại Edge; kiểm tra khả năng nạp trước khi khởi động vòng lặp
Hạ tầng triển khai	`docker-compose.yml`; `.env`	Khai báo service, mạng, volume và cấp biến môi trường	Secret nằm ngoài Git; cấu hình Compose được kiểm chứng trước khi chạy
Dữ liệu kiểm thử	`test.csv`	Cung cấp bản ghi SCADA để phát lại theo nhịp một mẫu mỗi giây	Chỉ dùng cho thử nghiệm; không đại diện đầy đủ jitter và mất gói thực địa

Các tệp được đặt cùng workspace để giảm độ phức tạp đường dẫn trong giai đoạn thử nghiệm, nhưng ranh giới trách nhiệm vẫn được duy trì theo table 4.8. Khi hệ thống mở rộng, khối kết nối, tiền xử lý và suy luận có thể được tách thành module độc lập mà không thay đổi hợp đồng giữa cấu hình, scaler và mô hình.

Mã nguồn xuất ba artifact gồm `xgboost_model.json`, `scaler.joblib` và `config.json`. Tệp cấu hình lưu ngưỡng bất thường, MAE, RMSE, danh sách bảy đặc trưng và tên biến mục tiêu. Nhờ đó module inference không phải khai báo lại thủ công thứ tự feature.

Mã đóng gói ba artifact được chuyển sang Phụ lục A để phần nội dung chính tập trung vào hợp đồng giữa mô hình, scaler và cấu hình.

4.3.1 Thiết lập mô hình và các thành phần cần thiết

Trước khi vòng lặp truyền phát bắt đầu, nhóm nạp các tài nguyên tĩnh một lần vào bộ nhớ để tránh chi phí đọc tệp và khởi tạo lại đối tượng ở mỗi chu kỳ. Tệp

`xgboost_model.json` cung cấp cấu trúc và tham số của mô hình đã huấn luyện. Tệp `scaler.joblib` lưu đối tượng chuẩn hóa cùng các giá trị đã học trên tập train, nhờ đó dữ liệu suy luận được truyền vào cùng không gian đặc trưng. Tệp `config.json` giữ danh sách đặc trưng theo đúng thứ tự mà mô hình yêu cầu, đồng thời cung cấp ngưỡng `anomaly_threshold`. Việc tách hai tham số này khỏi mã điều phối giúp nhóm thay đổi cấu hình thực nghiệm mà không sửa trực tiếp logic của vòng lặp. Tuy nhiên, mọi thay đổi danh sách đặc trưng vẫn phải tương thích với model và scaler tương ứng, chỉ chỉnh sửa tệp cấu hình không thể làm mô hình hỗ trợ một đặc trưng chưa xuất hiện trong quá trình huấn luyện.

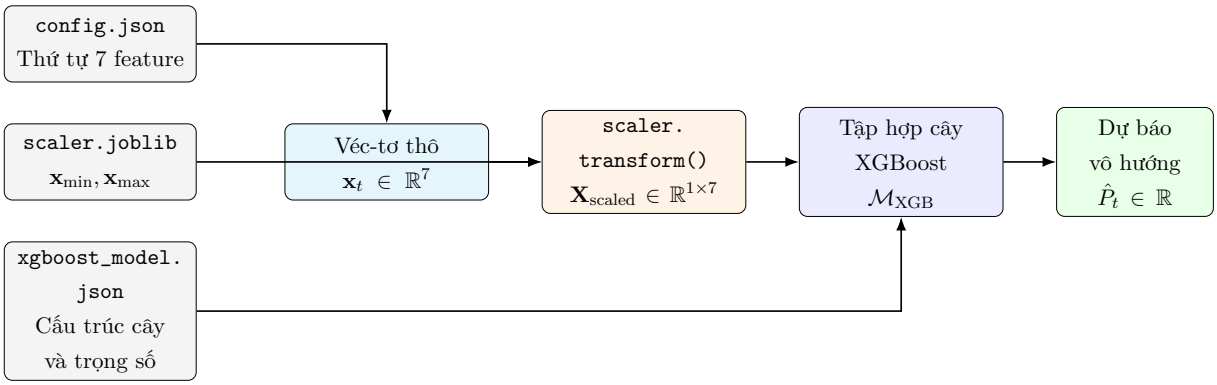


Figure 4.3: Kiến trúc véc-tơ đặc trưng, bộ chuẩn hóa và mô hình suy luận

Sau khi mô hình XGBoost, đối tượng chuẩn hóa và các tham số cấu hình được nạp thành công, hệ thống đã hoàn tất các bước chuẩn bị cần thiết cho quá trình suy luận. Các đối tượng này được duy trì trong bộ nhớ để sử dụng trong suốt vòng đời của chương trình. Trong đó, giá trị ngưỡng cảnh báo được lưu vào biến `threshold`, còn danh sách các đặc trưng đầu vào được lưu vào biến `features`. Hai tham số này được sử dụng trong các bước xử lý tiếp theo để xây dựng dữ liệu đầu vào và xác định trạng thái của từng bản ghi. Sau khi quá trình khởi tạo hoàn tất, chương trình bắt đầu vòng lặp đọc dữ liệu từ tệp `test.csv`. Từ thời điểm này, từng bản ghi lần lượt được đưa qua các bước tiền xử lý, suy luận và gửi kết quả đến cơ sở dữ liệu InfluxDB theo cơ chế phát dữ liệu đã được thiết lập.

4.4 Truyền phát dữ liệu thời gian thực và suy luận biên tức thời

4.4.1 Sơ đồ khối kiến trúc luồng truyền phát dữ liệu

Để đảm bảo khả năng xử lý tức thời ngay tại thiết bị biên, kịch bản Python được thiết kế theo mô hình đường ống dữ liệu tuyến tính. Khác với các hệ thống phân tán phức tạp, luồng thực thi trong đồ án bao trọn toàn bộ vòng đời của một điểm dữ liệu chỉ trong một

tiến trình duy nhất. Quy trình này bắt đầu từ khâu nạp môi trường, tiền xử lý, nội suy bằng học máy, cho đến đóng gói và truyền tải qua mạng.

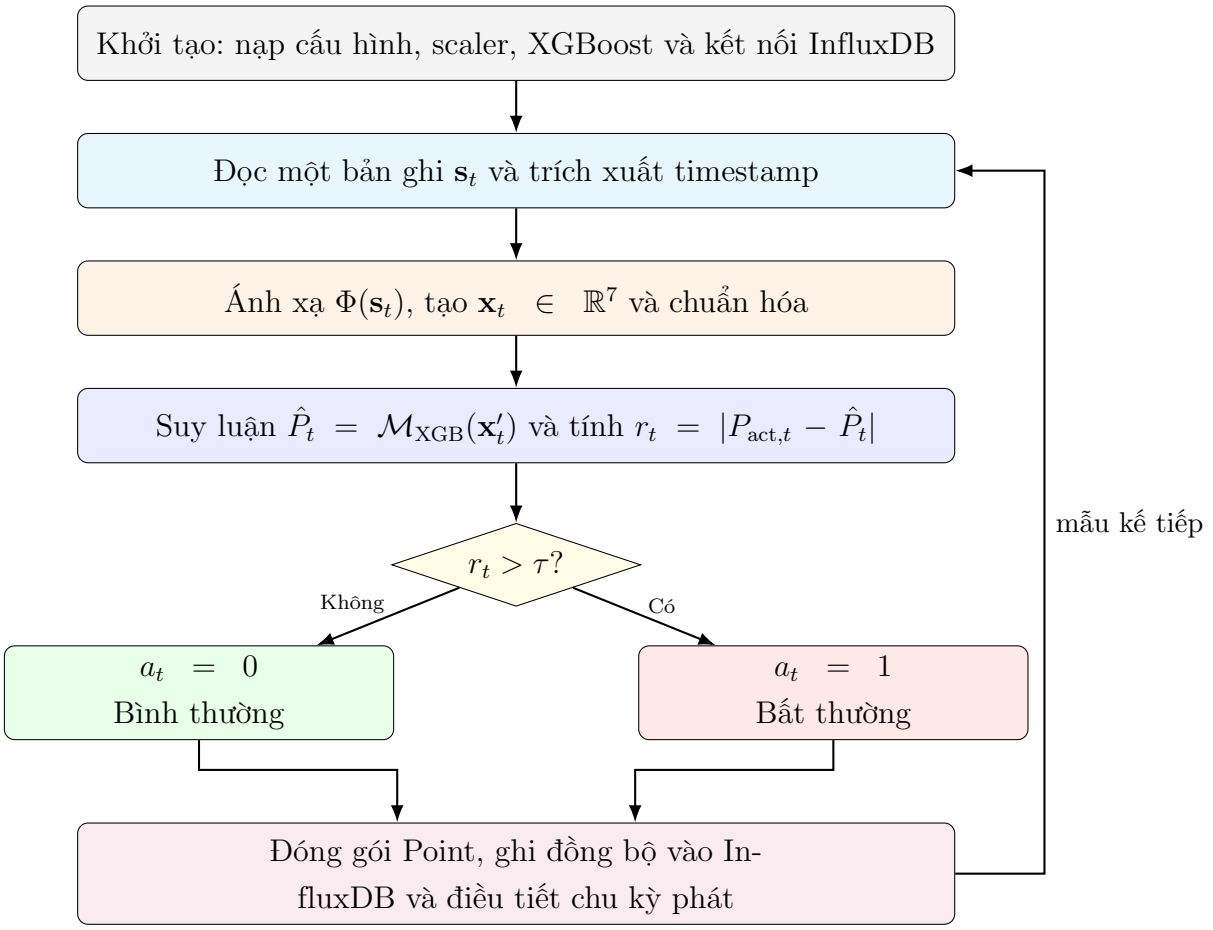


Figure 4.4: Lưu đồ tổng thể của thuật toán streaming và suy luận

Sơ đồ hợp nhất bốn giai đoạn trước đây thành một luồng vector duy nhất. Artifact tĩnh chỉ được nạp một lần khi khởi tạo; mỗi vòng lặp sau đó thực hiện tiền xử lý, suy luận, quyết định trạng thái và ghi dữ liệu. Cách biểu diễn này loại bỏ các hình phóng to lặp lại nhưng vẫn giữ đầy đủ ranh giới trách nhiệm của từng khối.

4.4.2 Chuyển đổi dữ liệu lịch sử thành luồng

Trong giai đoạn chưa nối hệ thống với cảm biến vật lý, nhóm sử dụng tập dữ liệu lịch sử làm nguồn đầu vào. Nếu đưa toàn bộ ma trận vào mô hình theo phương thức xử lý lô, quá trình tính toán sẽ kết thúc trong thời gian ngắn và không thể hiện được tính liên tục của luồng SCADA. Kết quả đó phù hợp cho đánh giá ngoại tuyến, nhưng không đủ để kiểm thử chu kỳ tiền xử lý, suy luận, ghi dữ liệu và cập nhật giao diện theo thời gian.

Để giải quyết vấn đề này, nhóm thiết lập cơ chế phát lại tuần tự. Kịch bản nạp tệp `test.csv` vào bộ nhớ nhưng không suy luận toàn bộ dữ liệu cùng lúc. Một vòng lặp lần lượt lấy từng bản ghi và đưa mẫu đó qua đầy đủ các bước: chuyển đổi thời gian, xây dựng véc-tơ đặc trưng, chuẩn hóa, suy luận bằng XGBoost, tính residual và ghi kết quả vào InfluxDB.Ngay sau khi hoàn tất một chu kỳ, chương trình tạm dừng một giây rồi mới xử lý bản ghi kế tiếp. Dữ liệu lịch sử vốn có khoảng cách mười phút giữa hai mẫu được phát lại với nhịp một giây một mẫu. Đây là phép nén trục thời gian phục vụ thử nghiệm, không phải nội suy để tạo thêm quan sát mới. Giá trị của từng bản ghi được giữ nguyên, chỉ thời điểm phát vào pipeline được điều chỉnh.

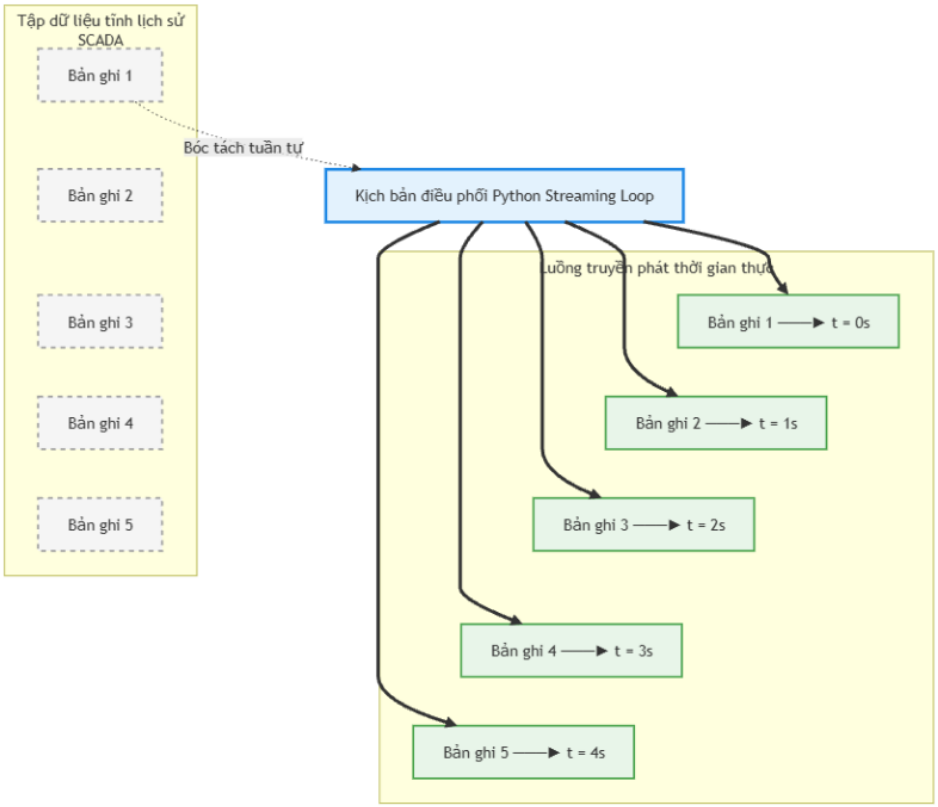


Figure 4.5: Cơ chế phát lại dữ liệu lịch sử theo nhịp một giây

Cơ chế trên tạo các thời điểm phát mô phỏng $t = 0\,\text{s}$, $t = 1\,\text{s}$, $t = 2\,\text{s}, \ldots$ và cho phép nhóm quan sát toàn bộ pipeline theo từng mẫu. Trong quá trình thử nghiệm, nhóm có thể đo thời gian suy luận, độ trễ ghi InfluxDB và tốc độ cập nhật Grafana. Do nguồn dữ liệu vẫn là tệp lịch sử, kết quả chỉ chứng minh hành vi của pipeline trong môi trường phát lại, việc đánh giá tải thực tế cần được lặp lại khi hệ thống kết nối với nguồn đo trực tiếp.

4.4.3 Chuỗi xử lý suy luận tại biên

Exporter thực hiện sáu bước: nạp cấu hình, nạp model và scaler, đọc dữ liệu test, tái tạo đặc trưng sin–cos, dự báo công suất, ghi point vào InfluxDB mỗi giây. Residual và cờ bất thường được xác định bởi

$$r_t = \left| P_{\text{actual},t} - P_{\text{predicted},t} \right|, \tag{4.11}$$

$$a_t = \mathbb{I}\left[r_t > \tau \right], \qquad \tau = 1942.07\,\text{kW}. \tag{4.12}$$

Thuật toán 4.1: Chu trình suy luận biên và ghi dữ liệu thời gian thực

Đầu vào: bản ghi SCADA $\mathbf{s}_t$; mô hình $\mathcal{M}_{\text{XGB}}$; bộ chuẩn hóa $(\mathbf{W}_{\text{scale}}, \mathbf{x}_{\min})$; ngưỡng $\tau = 1942.07\,\text{kW}$.
Đầu ra: InfluxDB Point $\mathcal{P}_t$ và cờ cảnh báo a_t.

1. Trích xuất giờ h_t, phút u_t và tháng q_t; tính $m_t = 60h_t + u_t$.
2. Tính $\text{Day}_{\sin}$, $\text{Day}_{\cos}$, $\text{Month}_{\sin}$ và $\text{Month}_{\cos}$.
3. Lắp ráp $\mathbf{x}_t \in \mathbb{R}^7$ theo eq. (4.1), sau đó tính $\mathbf{x}'_t = \mathbf{W}_{\text{scale}}(\mathbf{x}_t - \mathbf{x}_{\min})$.
4. Suy luận công suất kỳ vọng: $\hat{P}_t = \mathcal{M}_{\text{XGB}}(\mathbf{x}'_t)$.
5. Tính residual $r_t = |P_{\text{act},t} - \hat{P}_t|$.
6. Gán trạng thái $a_t = \mathbb{I}(r_t > \tau)$.
7. Đóng gói $\mathcal{P}_t = \langle \texttt{turbine_status}, \texttt{turbine_id=T1}, \{v_t, P_{\text{act},t}, \hat{P}_t, r_t, a_t\}, t_{\text{ns}} \rangle$.
8. Ghi $\mathcal{P}_t$ bằng Write API đồng bộ. Với thời gian xử lý Δt_{proc}, chờ $\Delta t_{\text{sleep}} = \max(0, 1 - \Delta t_{\text{proc}})$ giây.

Mã Python đầy đủ của exporter được chuyển sang Phụ lục A và đã thay toàn bộ credential bằng biến môi trường.

Ghi đồng bộ giúp kiểm thử dễ quan sát nhưng mỗi vòng lặp phải chờ HTTP response. Khi tăng số tua-bin, write batching sẽ giảm số request và tăng throughput. Đổi lại, batching tạo bộ đệm và có thể làm tăng độ trễ quan sát, vì vậy kích thước batch phải cân bằng với chu kỳ cảnh báo.

4.5 Thiết kế lược đồ chuỗi thời gian InfluxDB v2 và kiểm chứng dữ liệu

4.5.1 Phân tích dữ liệu

Table 4.9: Phân tích dữ liệu SCADA và kết quả suy luận tại biên

Tên trường thô / Đại lượng	Field trong InfluxDB	Đơn vị	Vai trò trong thuật toán
`Date/Time`	–	–	Sinh đặc trưng chu kỳ và duy trì thứ tự phát lại.
`Wind Speed (m/s)`	`wind_speed`	m/s	Đặc trưng đầu vào của mô hình.
`Theoretical Power Curve (KWh)`	–	kWh	Đặc trưng tham chiếu cho mô hình.
`Wind Direction (?)`	–	Độ	Đặc trưng đầu vào của mô hình.
`LV ActivePower (kW)`	`actual_power`	kW	Giá trị đối chiếu để tính residual sau suy luận.
`Day_sin`, `Day_cos`, `Month_sin`, `Month_cos`	–	–	Sinh đặc trưng chu kỳ
Công suất dự báo	`predicted_power`	kW	Công suất dự báo từ mô hình
Độ lệch phần dư	`residual`	kW	Sai lệch giữa giá trị thực tế và dự báo
Cờ bất thường	`anomaly_flag`	–	Trạng thái bình thường hoặc bất thường

Dữ liệu SCADA phát sinh theo chu kỳ và mỗi bản ghi phải được liên kết với một mốc thời gian cụ thể. Vì vậy, tại mỗi vòng lặp, kịch bản Python gom các giá trị của mẫu hiện tại vào một Point độc lập theo mô hình dữ liệu của InfluxDB [53].

Nhóm đặt measurement là `turbine_status` để phân loại nhóm dữ liệu trạng thái tua-bin. Tag `turbine_id` mang giá trị `T1` đóng vai trò định danh thiết bị và cho phép mở rộng cùng một measurement cho nhiều tua-bin. Các đại lượng biến thiên được lưu dưới dạng field, gồm `wind_speed`, `actual_power`, `predicted_power`, `residual` và `anomaly_flag`. Cách phân tách này giữ thông tin định danh ở tag để hỗ trợ lọc series, trong khi các giá trị đo và kết quả suy luận được lưu ở field.

Mỗi Point được gắn timestamp tương ứng với thời điểm xử lý mẫu. Thành phần này cho phép InfluxDB lập chỉ mục và truy vấn bản ghi theo trình tự thời gian. Sau khi hoàn

tất đóng gói, nhóm gọi Write API của InfluxDB Client để gửi Point đến máy chủ bằng chế độ ghi đồng bộ. Việc dùng timestamp phát lại cần được phân biệt với thời gian đo gốc trong tệp CSV, khi cần đối chiếu dữ liệu lịch sử, hệ thống nên lưu thêm timestamp nguồn hoặc quy tắc truyền giữa hai trục thời gian.

Ước lượng quy mô dung lượng timestamp. Với chu kỳ 1 Hz trong một tháng 30 ngày, mỗi field-series tạo ra 2,592,000 timestamp. Nếu lưu thô ở dạng số nguyên 64 bit, riêng timestamp chiếm khoảng 20.736 MB cho một field-series. Bốn field số của lược đồ hiện tại tương ứng khoảng 82.944 MB cho mỗi tua-bin, khi mở rộng lên 25 tua-bin, phần timestamp thô đạt xấp xỉ 2.074 GB mỗi tháng. Các con số này giải thích vì sao nhịp lấy mẫu đều giúp tạo ra mức tiết kiệm đáng kể ở quy mô trang trại gió. Dù vậy, đây là dung lượng thô dùng để minh họa, không phải số byte chắc chắn được tiết kiệm: dung lượng thực tế còn gồm header khối, index, tag, WAL, field value và phụ thuộc tỷ lệ nén của từng chuỗi.

4.5.2 Mô hình hóa độ biến thiên chuỗi thời gian

Độ biến thiên chuỗi thời gian là tích số của số lượng giá trị phân biệt trong toàn bộ các nhãn chỉ mục:

$$C = \prod_{k=1}^{M} |V_{\text{tag}_k}| \tag{4.13}$$

Trong đó $|V_{\text{tag}_k}|$ là số giá trị phân biệt của nhãn thứ k. Nếu sử dụng các trường có độ biến thiên cao làm nhãn chỉ mục, dung lượng bảng băm trong bộ nhớ RAM sẽ tăng theo hàm số mũ:

$$M_{\text{index}} \propto C \cdot \overline{L_{\text{series_key}}} \tag{4.14}$$

Để tránh hiện tượng cạn kiệt bộ nhớ hệ thống trên thiết bị nhúng biên, hệ thống SCADA giới hạn cấu trúc nhãn chỉ mục chỉ bao gồm định danh trạm phát điện và trạng thái vận hành với số lượng phần tử hữu hạn.

4.5.3 Ghi dữ liệu kiểm thử qua giao thức dòng

Thay vì trình bày câu lệnh HTTP ở dạng mã nguồn, quá trình kiểm thử được đặc tả theo mục tiêu, cơ chế tác động và bằng chứng quan sát trong table 4.10.

Table 4.10: Ma trận thực thi kiểm thử ghi dữ liệu qua InfluxDB Write API

Mục tiêu kiểm thử	Cơ chế tác động	Tham số đầu vào	Hành vi kỳ vọng	Minh chứng hệ thống ghi nhận
Xác nhận endpoint ghi nhận Point hợp lệ	HTTP POST tới Write API	Organization `scada_org`, bucket `turbine_metrics`, độ chính xác thời gian theo giây và token chỉ ghi	API xác thực yêu cầu và chấp nhận bản ghi hợp lệ	Phản hồi ghi thành công và Point xuất hiện trong Data Explorer
Xác nhận ánh xạ lược đồ Line Protocol	Line Protocol	Measurement `turbine_status`; tag `turbine_id=T1`, `status=Normal`; các field gió, công suất và residual	InfluxDB tách đúng measurement, tag set, field set và timestamp	Các cột, kiểu field và series key hiển thị đúng trong truy vấn kiểm chứng
Xác nhận dấu thời gian của mẫu	Write API với `precision=s`	Unix timestamp theo đơn vị giây	Point được đặt đúng vị trí trên trục thời gian, không bị dịch theo sai đơn vị	Timestamp trong Data Explorer trùng mốc phát mẫu

Cấu trúc dòng dữ liệu gồm bốn vùng được phân cách theo cú pháp cố định: measurement, tag set, field set và timestamp. Measurement cùng tag set được ngăn bởi dấu phẩy, một dấu cách phân tách phần tag khỏi field, dấu cách cuối cùng đứng trước timestamp. Với tham số `precision=s`, giá trị timestamp của kịch bản kiểm thử được hiểu theo đơn vị giây[54].

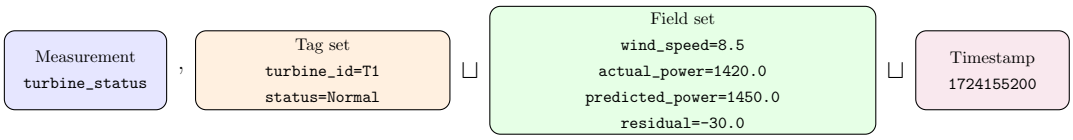


Figure 4.6: Phân rã một bản ghi InfluxDB Line Protocol thành bốn thành phần

Nhóm sử dụng Line Protocol vì đây là định dạng ghi gốc của InfluxDB và biểu diễn một Point trên một dòng văn bản. So với một đối tượng JSON chứa nhiều dấu ngoặc, tên thuộc tính lặp lại và cấu trúc phân cấp, bản ghi Line Protocol có kích thước gọn hơn cho schema đo lường cố định. Thiết bị biên có thể tuần tự hóa các giá trị theo thứ tự đã biết, còn máy chủ tiếp nhận trực tiếp measurement, tags, fields và timestamp mà không cần chuyển đổi từ một mô hình đối tượng trung gian[54].

Lợi thế trên không có nghĩa Line Protocol không cần phân tích cú pháp hoặc luôn nhanh hơn JSON trong mọi trường hợp. InfluxDB vẫn phải kiểm tra kiểu field, ký tự escape, precision và tính hợp lệ của bản ghi. InfluxDB Client đã đảm nhận việc tạo Line Protocol từ đối tượng Point, nên mã ứng dụng không cần tự nối chuỗi thủ công. Khả năng đạt hàng nghìn Point mỗi giây còn phụ thuộc batch size, chế độ ghi đồng bộ hay bất đồng bộ, mạng và tốc độ lưu trữ, do đó phải được xác nhận bằng benchmark trên Orange Pi

thay vì suy ra chỉ từ định dạng dữ liệu.

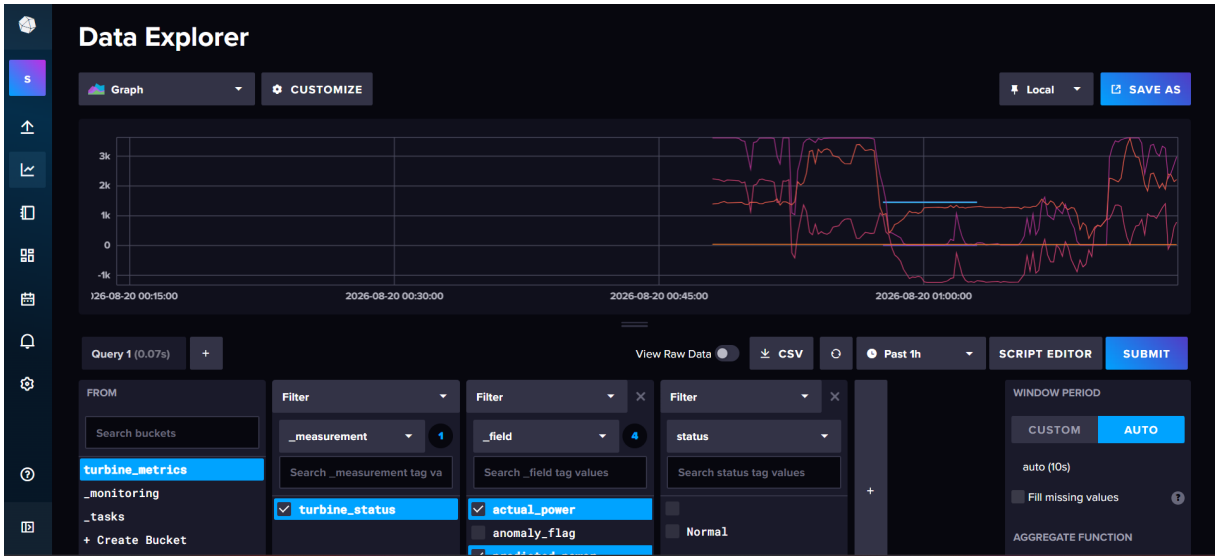


Figure 4.7: Xác thực bản ghi dữ liệu lưu trữ thành công trên giao diện InfluxDB Data Explorer

4.6 Cơ chế giảm tần suất mẫu tự động bằng ngôn ngữ Flux

Để tối ưu hóa không gian lưu trữ trên thiết bị biên mà vẫn bảo toàn xu hướng biến thiên dài hạn, tác vụ định kỳ của InfluxDB tổng hợp dữ liệu thô 1 Hz thành giá trị trung bình theo chu kỳ 5 phút. Các truy vấn Flux trong toàn hệ thống được chuẩn hóa thành phép hợp toán tử

$$\mathcal{Q}_j = \mathcal{Y}_j \circ \mathcal{A}_{\Delta_j, f_j} \circ \mathcal{F}_{E_j} \circ \mathcal{F}_{m_j} \circ \mathcal{R}_{T_j} \circ \mathcal{B}_j, \tag{4.15}$$

trong đó $\mathcal{B}_j$ chọn bucket nguồn, $\mathcal{R}_{T_j}$ giới hạn miền thời gian, $\mathcal{F}_{m_j}$ và $\mathcal{F}_{E_j}$ lọc measurement cùng tập field hoặc tag, $\mathcal{A}_{\Delta_j, f_j}$ tổng hợp theo cửa sổ Δ_j bằng hàm f_j, còn $\mathcal{Y}_j$ xuất chuỗi kết quả hoặc ghi sang bucket đích. Truy vấn chỉ mục $\mathcal{Q}_{\text{id}}$ là trường hợp riêng, đọc trực tiếp tập giá trị tag thay vì tổng hợp field.

Table 4.11: Đặc tả toán tử đường ống chuỗi thời gian bằng Flux

Ký hiệu	Mục đích hoặc biểu đồ	Measurement hoặc chỉ mục	Field và tag lọc	Cửa sổ	Hàm rút gọn và đầu ra
$\mathcal{Q}_1$	Độ lệch công suất	`turbine_status`	`residual`; tag `turbine_id`	`v.windowPeriod`	Trung bình; chuỗi residual
$\mathcal{Q}_2$	Công suất thực tế và dự báo	`turbine_status`	`actual_power`, `predicted_power`; tag `turbine_id`	`v.windowPeriod`	Trung bình; hai chuỗi công suất
$\mathcal{Q}_{\text{id}}$	Danh sách tua-bin động	Chỉ mục của `turbine_status`	Tag `turbine_id`	Miền nhìn lại 1 d	Tập giá trị tag phân biệt
$\mathcal{Q}_{\text{task}}$	Nén mẫu dài hạn	`turbine_status`	Ba field công suất thực tế, dự báo và tốc độ gió	5 min; offset 10 s	Trung bình; ghi `turbine_status_5m_avg` vào bucket dài hạn

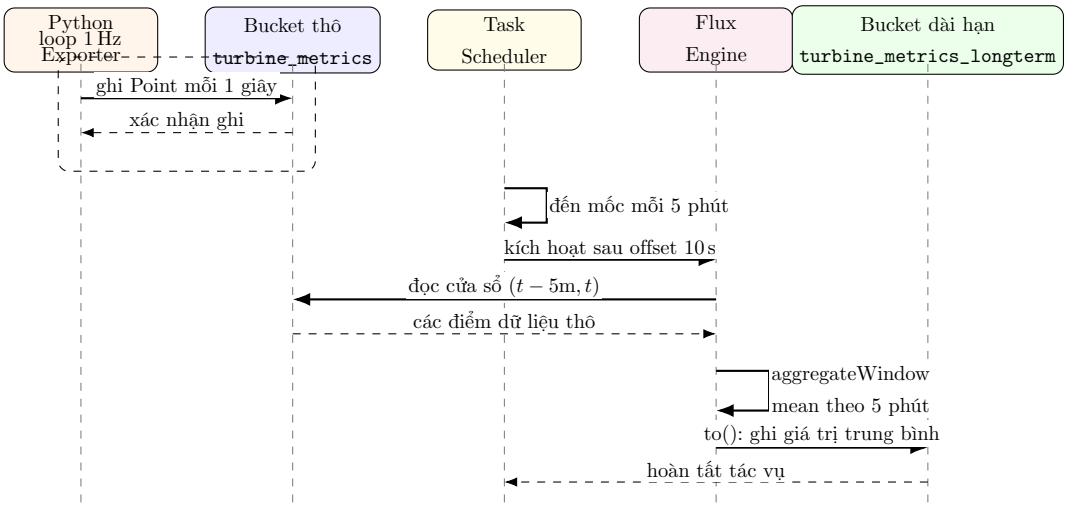


Figure 4.8: Biểu đồ tuần tự của tác vụ Flux tổng hợp dữ liệu theo cửa sổ năm phút

Tham số `every=5m` xác định chu kỳ lập lịch và phạm vi dữ liệu cần tổng hợp. Thay vì thực thi đúng tại ranh giới cửa sổ, nhóm đặt `offset=10s` để tác vụ bắt đầu muộn hơn mười giây nhưng vẫn truy vấn cửa sổ năm phút ban đầu. Khoảng chờ này tạo cơ hội cho các Point có timestamp thuộc cửa sổ vừa kết thúc nhưng đến máy chủ chậm do dao động mạng được ghi vào bucket thô trước khi phép trung bình diễn ra [55;56].

Offset chỉ hấp thụ độ trễ không vượt quá khoảng dự phòng đã chọn, nó không bảo đảm mọi dữ liệu đến muộn đều được tính. Point đến sau thời điểm tác vụ đã chạy vẫn có thể bị bỏ khỏi kết quả tổng hợp nếu hệ thống không thực hiện tính lại cửa sổ cũ. Vì vậy, giá trị mười giây phải được lựa chọn từ phân bố độ trễ thực tế. Với mạng có độ trễ lớn hoặc mất kết nối kéo dài, nhóm cần mở rộng offset hoặc bổ sung cơ chế chạy lại các cửa sổ gần nhất.

Chương 5: Hệ thống giám sát, thiết lập cảnh báo và thông báo sự cố.

5.1 Xây dựng bảng điều khiển Grafana thời gian thực

5.1.1 Cấu hình nguồn dữ liệu trên Grafana

Để Grafana truy vấn dữ liệu từ InfluxDB v2, nhóm chọn Flux làm ngôn ngữ truy vấn và đặt URL kết nối là `http://influxdb:8086`. Tên dịch vụ `influxdb` được Docker phân giải trong mạng bridge nội bộ, vì vậy cấu hình không phụ thuộc vào địa chỉ IP có thể thay đổi khi container được tạo lại. Nhóm tắt Basic Authentication, khai báo organization `scada_org`, đặt bucket mặc định là `turbine_metrics` và cung cấp token chỉ có quyền đọc. Đây cũng là tập thông số mà Grafana yêu cầu khi datasource InfluxDB sử dụng Flux[57].

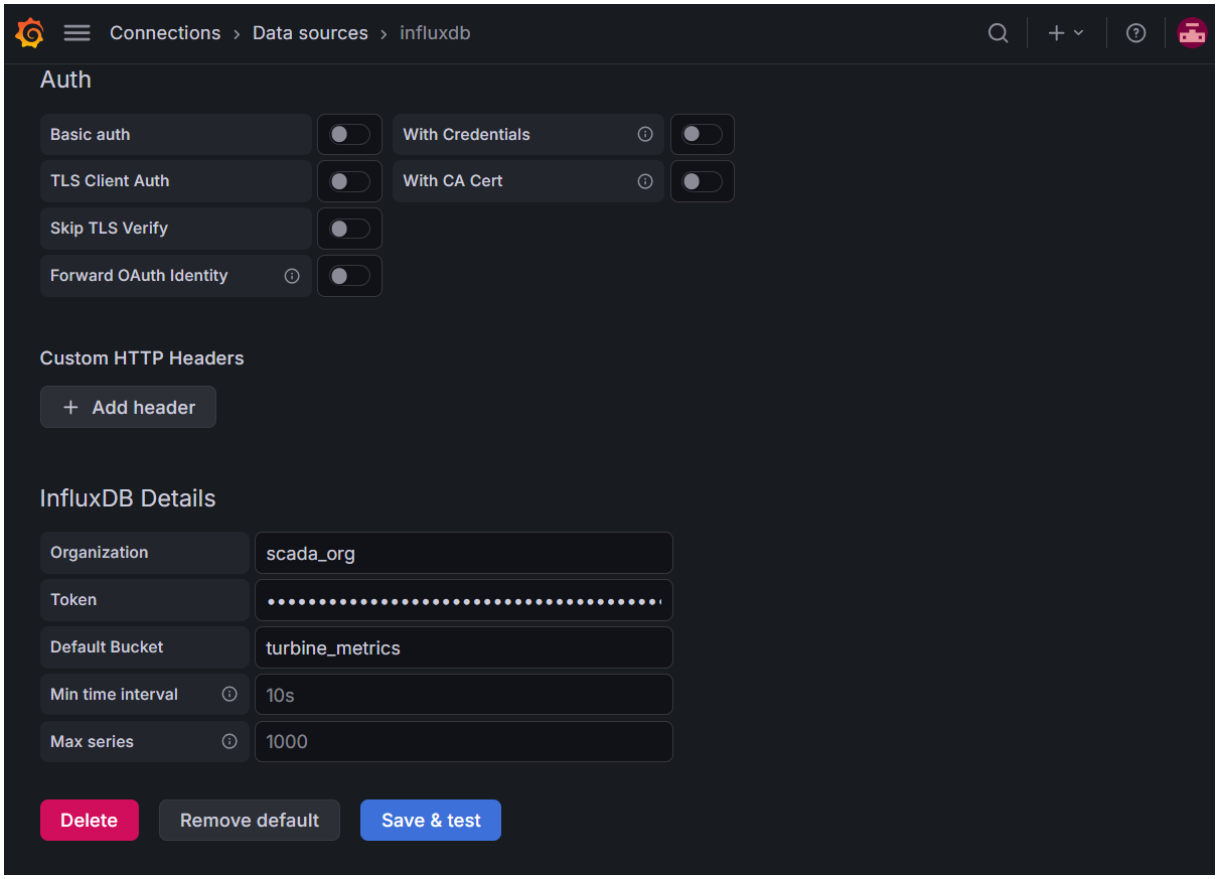


Figure 5.1: Cấu hình thông số kết nối nguồn dữ liệu Flux trên Grafana

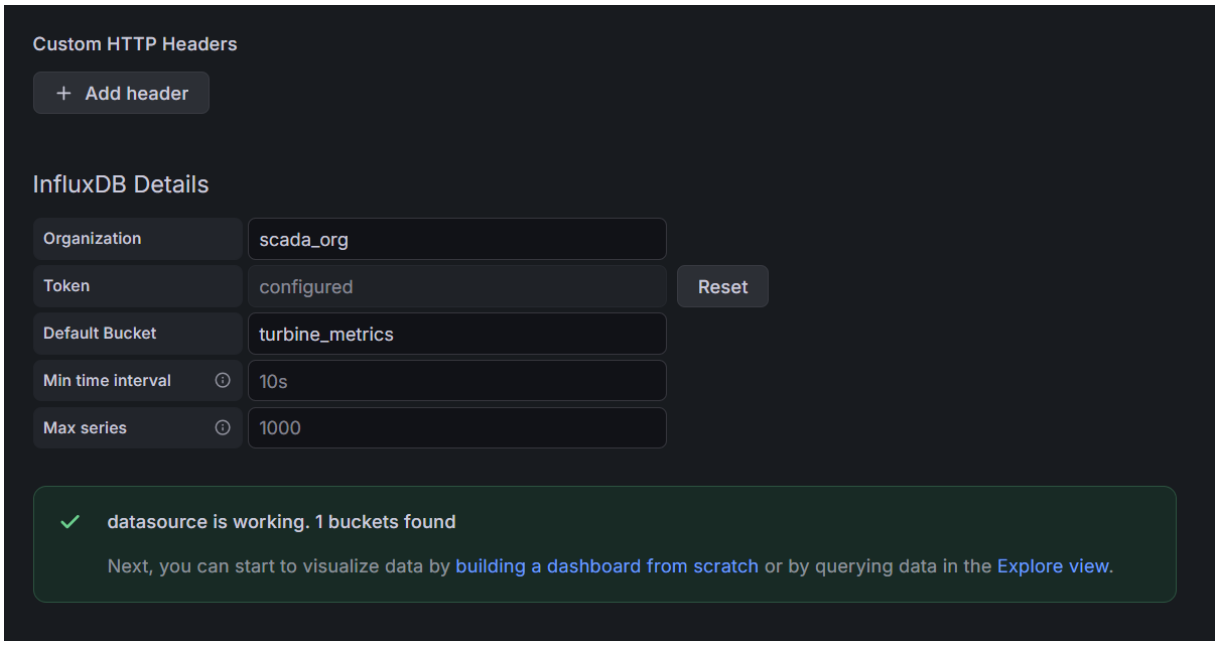


Figure 5.2: Kết quả kiểm thử kết nối nguồn dữ liệu thành công trên Grafana

5.1.2 Khung bên trái: Độ lệch công suất

Khung bên trái sử dụng toán tử $\mathcal{Q}_1$ trong table 4.11. Chuỗi residual được giới hạn theo khoảng thời gian của Dashboard, lọc theo tua-bin đang chọn và rút gọn bằng giá trị trung bình trên từng cửa sổ hiển thị.

Đường ngưỡng tĩnh và vùng tô màu được sử dụng để phân định trực quan hai miền vận hành. Khi residual chưa vượt 1942.07 kW, đường đồ thị nằm trong miền xanh lục và được xem là sai lệch trong giới hạn quan sát. Khi residual vượt ranh giới này, đồ thị đi vào vùng phủ màu đỏ để người vận hành nhận diện thời điểm cần kiểm tra. Cấu hình màu chỉ đảm nhiệm chức năng hiển thị, việc phát thông báo ra ngoài còn phụ thuộc quy tắc cảnh báo và khoảng thời gian duy trì được thiết lập trong phần hiện thực cảnh báo của đồ án.

5.1.3 Công suất thực tế và công suất dự báo

Khung bên phải hiển thị tương quan đồng thời giữa công suất phát điện thực tế mang định danh `actual\_power T1` bằng đường màu xanh lục và công suất ước lượng `predicted\_power T1` bằng đường màu vàng. Cả hai chuỗi dữ liệu được biểu diễn trên cùng trục tọa độ công suất với dải đo từ 0 đến 4000 kW.

Khung này sử dụng toán tử $\mathcal{Q}_2$ trong table 4.11. Hai field công suất dùng chung miền thời gian, tag tua-bin và chu kỳ tổng hợp nên có thể đối chiếu trên cùng trục timestamp mà không cần lặp lại cú pháp Flux trong nội dung chính.

Để trích xuất dữ liệu hiệu quả, nhóm xây dựng truy vấn Flux theo năm giai đoạn xử lý liền mạch. Đầu tiên, hàm `range` gắn bộ lọc thời gian với hai mốc biên do người dùng lựa chọn trên giao diện. Phạm vi này giúp động cơ truy vấn giới hạn các vùng cache và tệp TSM cần đọc thay vì quét toàn bộ dữ liệu lịch sử. Tiếp theo, biểu thức lọc measurement loại bỏ sớm các bảng không thuộc nhóm `turbine_status`, nhờ đó giảm lượng dữ liệu phải chuyển sang các phép xử lý phía sau.

Để phân giải đúng trạm phát điện, nhóm lọc theo tag `turbine_id`, tận dụng cấu trúc chỉ mục series thay vì tìm kiếm trên toàn bộ các field value. Khối dữ liệu còn lại được tách thành hai luồng tương ứng với công suất thực tế và công suất dự báo nhưng vẫn giữ chung trục timestamp. Cuối cùng, hàm `aggregateWindow` chia từng chuỗi thành các cửa sổ có độ rộng động và tính giá trị trung bình trong mỗi cửa sổ theo

$$\bar{P}_k = \frac{1}{M_k} \sum_{j=1}^{M_k} P(t_j), \qquad t_j \in [t_k, t_k + \text{windowPeriod}). \tag{5.1}$$

Tham số `createEmpty=false` không sinh thêm các cửa sổ rỗng khi khoảng thời gian tương ứng không có bản ghi. Cách tổ chức này giảm số điểm phải truyền tới trình duyệt và giữ cho đồ thị dễ quan sát, nhưng không tự nội suy dữ liệu bị thiếu, khoảng trống thật trong chuỗi vẫn cần được xử lý bằng chính sách hiển thị phù hợp trên Grafana.

5.1.4 Cấu hình biến động nội suy và đặc tả bảng điều khiển

Để hệ thống có thể mở rộng giám sát đồng thời nhiều trạm phát điện trong cùng một trang trại gió mà không cần nhân bản các bảng điều khiển tĩnh, hệ thống thiết lập một biến truy vấn động mang tên `turbine_id`.

Biến Dashboard sử dụng toán tử $\mathcal{Q}_{\text{id}}$ trong table 4.11. Toán tử đọc trực tiếp các giá trị phân biệt của tag `turbine_id` trong miền nhìn lại 24 giờ, nhờ đó giao diện tự động bổ sung tùy chọn khi một tua-bin mới hòa mạng.

Table 5.1: Đặc tả cấu hình các thành phần giao diện trên bảng điều khiển giám sát

Thành phần	Kiểu trực quan	Đơn vị và định dạng	Mục tiêu kỹ thuật và phạm vi
Độ lệch công suất	Đồ thị chuỗi thời gian	Kilowatt	Theo dõi sai số tức thời giữa thực tế và dự báo, nhận diện bất thường khi vượt ngưỡng
Công suất thực tế và dự báo	Đồ thị chuỗi thời gian	Kilowatt	Đánh giá tương quan công suất phát thực tế so với kỳ vọng lý thuyết của mô hình
Đường ngưỡng cảnh báo	Vùng phủ	1942.07 kW	Xác định trạng thái bình thường và trạng thái lỗi suy giảm công suất nghiêm trọng
Biến động trạm	Danh sách trích xuất	Chuỗi ký tự	Tự động nội suy mã trạm vào toàn bộ câu truy vấn của các khung trên giao diện
Khung thời gian	Khoảng thời gian	30 phút gần nhất	Giám sát dữ liệu độ trễ thấp với tần suất làm tươi tự động mỗi 10 giây

Thay vì tải toàn bộ dữ liệu lịch sử, nhóm thiết lập khung thời gian giám sát mặc định ở mức 30 phút gần nhất và chu kỳ tự động làm mới là 10 giây. Khoảng làm mới này đủ ngắn để kỹ sư theo dõi kịp thời biến động công suất, đồng thời tránh tạo truy vấn mới sau mỗi lần exporter ghi dữ liệu một giây. Cửa sổ 30 phút cung cấp phạm vi quan sát phù hợp để nhận diện xu hướng vận tốc gió và mức độ bám sát giữa hai chuỗi công suất. Timestamp được thống nhất theo UTC tại tầng lưu trữ và được Grafana quy đổi sang múi giờ hiển thị của trình duyệt, nhờ đó các khung sử dụng cùng một trục thời gian.

Trên bảng điều khiển, nhóm chọn kiểu trực quan chuỗi thời gian và sử dụng datasource InfluxDB đã cấu hình trong đồ án. Biến `turbine_id` được nội suy vào truy vấn của từng khung, cho phép thay đổi trạm quan sát mà không phải sao chép dashboard. Hai khung được bố trí song song: khung residual ở bên trái phục vụ nhận diện bất thường, còn khung công suất ở bên phải dùng để đối chiếu trực tiếp `actual_power` và `predicted_power` với quy chuẩn đơn vị công suất là kilowatt và đặt trục tung của khung công suất trong khoảng từ 0 đến 4000 kW. Đối với khung residual, giới hạn trục được điều chỉnh tự động để tận

dụng không gian hiển thị, trong khi đường ngưỡng vẫn được cố định tại 1942.07 kW. Giá trị hiển thị được làm tròn từ một đến hai chữ số thập phân, bảng chú giải nằm phía dưới đồ thị và tooltip cho phép đối chiếu các chuỗi tại cùng một timestamp.

Để phân biệt trạng thái vận hành, miền an toàn từ mức cơ sở 0 được biểu diễn bằng màu xanh lục nhạt, còn ngưỡng sự cố 1942.07 kW sử dụng màu đỏ đậm kết hợp vùng phủ. Khi đường residual vượt ranh giới này, phần đồ thị đi vào vùng đỏ giúp người vận hành nhận diện nhanh thời điểm cần kiểm tra. Sau khi hoàn tất cấu hình, toàn bộ datasource, truy vấn, biến dashboard, trục đo, màu sắc và vị trí lưới được lưu trong mô hình dashboard để có thể phục hồi nhất quán khi tải lại giao diện.

5.2 Thiết kế quy tắc cảnh báo tự động theo cấu trúc đánh giá A–B–C

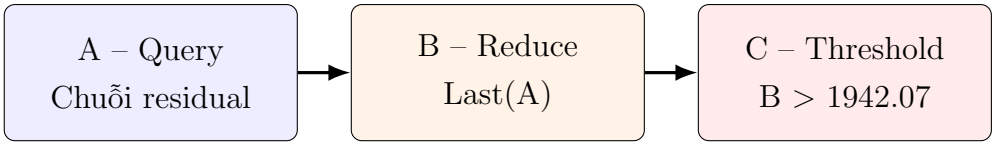


Figure 5.3: Chuỗi đánh giá Query–Reduce–Threshold

Việc tách ba tầng giúp mỗi biểu thức chỉ có một trách nhiệm và một chiều phụ thuộc. A không tham chiếu B hoặc C, B chỉ nhận A, C chỉ nhận B. Cấu trúc này loại bỏ khả năng tự tham chiếu và lỗi circular dependency. Đồng thời, người kiểm thử có thể xem riêng dữ liệu thô, giá trị sau Reduce và kết quả boolean để xác định tầng gây lỗi.

Toàn bộ chuỗi đánh giá được hình thức hóa thành hàm hợp từ dữ liệu chuỗi thời gian sang tín hiệu logic:

$$\mathcal{F}_{\text{alert}} = \mathcal{C} \circ \mathcal{B} \circ \mathcal{A} : \mathcal{T}_{[t-W,t]} \xrightarrow{\mathcal{A}} \mathbf{R}_W \xrightarrow{\mathcal{B}} B(t) \xrightarrow{\mathcal{C}} C(t) \in \{0,1\}, \qquad W = 300\,\text{s}. \tag{5.2}$$

Ba toán tử thành phần được xác định bởi

$$\mathbf{R}_W = \{r(s) \mid s \in [t-W, t]\}\,, \tag{5.3}$$

$$B(t) = \text{Last}(\mathbf{R}_W) = r(t_{\text{latest}}), \tag{5.4}$$

$$C(t) = \mathbb{I}(B(t) > \tau) = \begin{cases} 1, & B(t) > 1942.07\,\text{kW}, \\ 0, & B(t) \le 1942.07\,\text{kW}. \end{cases} \tag{5.5}$$

5.2.1 Khối A: truy vấn residual bằng Flux

Khối A tái sử dụng toán tử $\mathcal{Q}_1$ trong table 4.11: truy vấn bucket `turbine_metrics`, giới hạn dữ liệu theo miền thời gian do Grafana cung cấp, lọc đúng measurement và field, sau đó tổng hợp theo `v.windowPeriod`.

Hàm `range` ngăn truy vấn quét toàn bucket, hai hàm `filter` loại bỏ series không liên quan, `aggregateWindow` giảm số điểm khi cửa sổ quan sát lớn. Trong ảnh rule, datasource là InfluxDB, khoảng truy vấn là 5 phút đến thời điểm hiện tại và bước dữ liệu đầu vào là 1 giây.

Mô hình thực thi và cơ chế pushdown: Flux biểu diễn truy vấn dưới dạng chuỗi phép biến đổi dữ liệu. Sau khi phân tích cú pháp, bộ lập kế hoạch xác định những phép toán có thể đẩy xuống tầng lưu trữ thay vì tải toàn bộ dữ liệu lên bộ nhớ của tiến trình truy vấn. Việc đặt `range` và `filter` ngay sau `from` giúp thu hẹp miền thời gian, measurement và field từ sớm. Những phép toán được hỗ trợ pushdown có thể được thực hiện gần dữ liệu lưu trữ, sau khi gặp phép toán không hỗ trợ pushdown, phần dữ liệu còn lại phải được xử lý trong bộ nhớ, làm tăng lưu lượng truyền và RAM cần thiết[58].

Cơ chế này thường được mô tả gần với đánh giá trì hoãn: khai báo pipeline chưa đồng nghĩa mọi bảng trung gian được vật chất hóa ngay lập tức. Tuy nhiên, điểm quan trọng về kỹ thuật không phải tên gọi lazy evaluation mà là khả năng tối ưu kế hoạch và pushdown. Do đó thứ tự toán tử có ảnh hưởng trực tiếp đến chi phí. Một truy vấn tốt phải giới hạn dữ liệu trước, chỉ giữ cột cần thiết và tránh chèn phép biến đổi không hỗ trợ pushdown trước các phép lọc quan trọng.

Toán học của `aggregateWindow`: Giả sử cửa sổ truy vấn được chia thành các khoảng không chồng lấn

$$I_k = [t_0 + k\Delta,\ t_0 + (k+1)\Delta), \tag{5.6}$$

trong đó Δ là `v.windowPeriod`. Với n_k mẫu residual thuộc cửa sổ I_k, phép `fn: mean` sinh một điểm đại diện

$$\bar{r}_k = \frac{1}{n_k} \sum_{i:t_i \in I_k} r_i. \tag{5.7}$$

Mỗi cửa sổ vì vậy được nén thành một điểm, tạo ra phép giảm mẫu theo thời gian. Với `createEmpty=false`, cửa sổ không chứa mẫu bị bỏ qua thay vì tạo hàng rỗng[59]. Phép tổng hợp theo các cửa sổ thời gian không chồng lấn là một trường hợp điển hình của xử lý thống kê trên luồng dữ liệu[60;61].

Cần phân biệt phép trên với nội suy và trung bình trượt. `aggregateWindow` không tự



suy đoán giá trị giữa hai timestamp, nó chỉ tổng hợp những mẫu thật sự tồn tại trong từng cửa sổ. Trung bình trượt m mẫu lại dùng các cửa sổ chồng lấn

$$\mathrm{MA}_m(t_j) = \frac{1}{m} \sum_{\ell=0}^{m-1} r_{j-\ell}, \tag{5.8}$$

nên một mẫu có thể tham gia nhiều kết quả liên tiếp. Sự khác biệt được tóm tắt trong table 5.2.

Table 5.2: So sánh tổng hợp cửa sổ và trung bình trượt

Tiêu chí	`aggregateWindow(..., fn: mean)`	Trung bình trượt
Cấu trúc cửa sổ	Thường không chồng lấn	Chồng lấn theo từng mẫu mới
Số lần một mẫu được dùng	Một lần trong cửa sổ tương ứng	Có thể xuất hiện trong m kết quả
Mục đích chính	Downsampling và giảm số điểm	Làm trơn diễn biến cục bộ
Xử lý khoảng trống	Bỏ cửa sổ khi `createEmpty=false`	Phụ thuộc quy tắc thiếu dữ liệu
Nguy cơ	Làm phẳng đỉnh nếu Δ quá lớn	Tạo độ trễ và tương quan giữa các điểm

Trong Alert Rule, mục tiêu là phát hiện mẫu mới nhất nên độ rộng cửa sổ tổng hợp không được lớn đến mức làm trung bình hóa một spike nguy hiểm. Nếu exporter phát một mẫu mỗi giây, lựa chọn $\Delta = 1$ giây gần như giữ nguyên độ phân giải, $\Delta = 10$ giây có thể giảm tối đa mười mẫu thành một điểm nhưng cũng làm giảm biên độ của bất thường chỉ tồn tại một giây.

5.2.2 Khối B: rút gọn bằng Last

Alert condition cần một giá trị vô hướng thay vì toàn bộ chuỗi. Khối B sử dụng hàm `Last`:

$$B(t) = \mathrm{Last}\{r(s) \mid s \in [t - W, t]\}, \tag{5.9}$$

trong đó W là cửa sổ truy vấn. Hàm này phù hợp với yêu cầu phát hiện tức thời vì phản ánh mẫu mới nhất. Nếu cần giảm nhiễu, có thể thay Last bằng Mean hoặc Median trên

cửa sổ ngắn, nhưng việc đó làm tăng độ trễ và có thể làm phẳng một sự cố ngắn nhưng nghiêm trọng.

Chế độ Reduce trong ảnh là `Strict`. Điều này yêu cầu dữ liệu đầu vào hợp lệ thay vì tự động bỏ qua mọi lỗi. Kết quả vẫn giữ label `turbine_id=T1`, nhờ đó Grafana có thể tạo alert instance riêng cho từng tua-bin khi hệ thống mở rộng.

5.2.3 Khối C: so sánh với ngưỡng ba Sigma

Khối C dùng điều kiện *Is above 1942.07*. Biểu thức tương đương

$$C(t) = \mathbb{I}\left[B(t) > 1942.07\,\text{kW}\right], \tag{5.10}$$

với $\mathbb{I}$ là hàm chỉ thị. Hệ thống sử dụng phép so sánh nghiêm ngặt, do đó giá trị đúng bằng ngưỡng chưa kích hoạt. Trong ảnh rule, C nhận B làm input và được chọn làm `Alert condition`, đây là điều kiện cuối cùng của toàn bộ chuỗi.

5.2.4 Thiết lập dữ liệu cảnh báo

Labels Minh chứng Dashboard thể hiện ba nhãn thực tế:

– `alertname` = *Độ lệch công suất*: tên của rule,

– `grafana_folder=SCADA Alerts`: nhóm quản trị rule,

– `turbine_id=T1`: định danh thiết bị phát sinh sự kiện.

Labels phải ổn định vì Notification Policy dùng chúng để chọn Contact Point ở phần sau. Không nên đặt residual hoặc timestamp vào label vì các giá trị thay đổi liên tục sẽ làm tăng số alert instance, các giá trị động nên nằm trong annotation.

Annotation `summary` mô tả ngắn gọn rằng residual của tua-bin T1 đã vượt ngưỡng ba Sigma. Annotation `description` có thể chứa giá trị B, ngưỡng, timestamp và hướng xử lý ban đầu. Khác với label, annotation không tham gia định danh alert instance nên phù hợp với nội dung thay đổi theo thời gian.

Để tối ưu hóa thời gian tiếp nhận thông tin của kỹ sư trực vận hành, nhóm đã thiết kế cấu trúc thông báo giải quyết triệt để bốn vấn đề trọng tâm:: thiết bị nào gặp sự cố, đại lượng nào vượt ngưỡng, giá trị hiện tại và ngưỡng là bao nhiêu, người trực cần kiểm tra gì tiếp theo. Cách viết này rút ngắn thời gian đọc email và giảm nguy cơ nhầm lẫn khi có nhiều tua-bin.

5.3 Vận hành máy trạng thái và kỹ thuật chống rung cảnh báo

5.3.1 Vòng đời của một alert instance

Alert Rule được mô hình hóa bằng máy trạng thái hữu hạn đơn định

$$\begin{aligned}\mathcal{M}_{\text{alert}} &= \langle \mathcal{S}, \Sigma, \delta, S_0 \rangle\,, \\ \mathcal{S} &= \{\text{NORMAL}, \text{PENDING}, \text{FIRING}, \text{RESOLVED}\}, \qquad S_0 = \text{NORMAL}.\end{aligned} \tag{5.11}$$

trong đó đầu vào $r(t) \in \Sigma$ được đánh giá theo chu kỳ $T_e = 10\,\text{s}$. Bộ đếm số lần vượt ngưỡng liên tiếp được xác định bởi

$$\text{cnt}(t) = \begin{cases} \text{cnt}(t - T_e) + 1, & r(t) > \tau_{\text{on}}, \\ 0, & r(t) \le \tau_{\text{on}}, \end{cases} \qquad q = \left\lceil \frac{T_p}{T_e} \right\rceil. \tag{5.12}$$

Với $\tau_{\text{on}} = 1942.07\,\text{kW}$ và $\tau_{\text{off}} = 1700\,\text{kW}$, hàm chuyển trạng thái là

$$S_t = \begin{cases} \text{PENDING}, & S_{t-1} = \text{NORMAL} \wedge r_t > \tau_{\text{on}} \wedge \text{cnt}(t) < q, \\ \text{FIRING}, & \big(S_{t-1} = \text{PENDING} \wedge \text{cnt}(t) \ge q\big) \vee \big(S_{t-1} = \text{FIRING} \wedge r_t \ge \tau_{\text{off}}\big), \\ \text{RESOLVED}, & S_{t-1} = \text{FIRING} \wedge r_t < \tau_{\text{off}}, \\ \text{NORMAL}, & \big(S_{t-1} = \text{PENDING} \wedge r_t \le \tau_{\text{on}}\big) \vee S_{t-1} = \text{RESOLVED}. \end{cases} \tag{5.13}$$

Quy trình vòng đời này phù hợp với nguyên tắc quản trị cảnh báo công nghiệp, trong đó trạng thái, độ ưu tiên và hành động vận hành phải được xác định rõ [62].

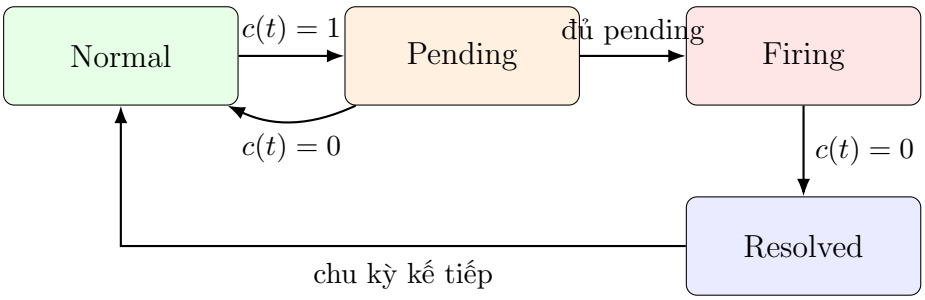


Figure 5.4: Máy trạng thái cảnh báo của Grafana

Table 5.3: Ma trận chuyển trạng thái của động cơ cảnh báo

Trạng thái hiện tại	Điều kiện phần dư	Trạng thái tiếp theo	Hành vi hệ thống và thông báo
NORMAL	$r_t \leq \tau_{\text{on}}$	NORMAL	Giữ trạng thái an toàn, không kích hoạt bộ đếm
NORMAL	$r_t > \tau_{\text{on}}$	PENDING	Khởi tạo cnt = 1, chưa gửi email
PENDING	$r_t > \tau_{\text{on}} \wedge \text{cnt} < q$	PENDING	Tăng bộ đếm và tiếp tục theo dõi
PENDING	$r_t \leq \tau_{\text{on}}$	NORMAL	Đặt lại bộ đếm, triệt tiêu xung nhiễu ngắn
PENDING	$r_t > \tau_{\text{on}} \wedge \text{cnt} \geq q$	FIRING	Mở sự cố và phát sự kiện FIRING
FIRING	$r_t \geq \tau_{\text{off}}$	FIRING	Duy trì cảnh báo và bộ định thời gửi lại
FIRING	$r_t < \tau_{\text{off}}$	RESOLVED	Đóng sự cố và phát sự kiện RESOLVED
RESOLVED	Chu kỳ đánh giá kế tiếp	NORMAL	Trở về giám sát bình thường

Với cấu hình thực nghiệm `Pending period=0s`, chuyển tiếp sang Firing xảy ra ngay. Cấu hình khuyến nghị $T_p = 30\,\text{s}$ tương ứng $q = 3$ giúp loại bỏ các spike ngắn trước khi phát thông báo.

5.3.2 Trạng thái No Data và Error

Ngoài vòng đời thông thường, Grafana phải quyết định cách xử lý khi truy vấn không trả dữ liệu hoặc khi datasource phát sinh lỗi. Ảnh cấu hình cho thấy hệ thống giữ hai trạng thái riêng: `NoData` khi không có giá trị hợp lệ và `Error` khi truy vấn thất bại hoặc timeout. Hai tình huống này không nên bị quy về Normal vì sự im lặng của cảm biến hoặc mất kết nối cũng là rủi ro vận hành.

Trong hệ thống sản xuất nên có thêm rule giám sát tính cập nhật của dữ liệu, chẳng hạn phát cảnh báo nếu timestamp mới nhất cũ hơn một khoảng cho phép. Rule bất thường công suất và rule mất dữ liệu có bản chất khác nhau, vì vậy cần tách thành hai alert instance để người vận hành biết phải kiểm tra tua-bin hay kiểm tra pipeline.

5.3.3 Kỹ thuật giảm rung cảnh báo

Nguyên nhân alert flapping: Flapping xảy ra khi residual dao động liên tục quanh ngưỡng, làm trạng thái thay đổi Normal–Firing–Resolved trong thời gian ngắn. Hậu quả là Dashboard xuất hiện nhiều marker và kênh email có thể tạo bão thông báo. Nguyên nhân có thể là nhiễu cảm biến, cửa sổ aggregate quá ngắn, ngưỡng tĩnh hoặc thiếu trạng thái pitch/phanh.

Pending period: Với pending T_p, điều kiện phải liên tục đúng trong khoảng đó:

$$c(s) = 1, \quad \forall s \in [t - T_p, t]. \tag{5.14}$$

Nếu rule được đánh giá mỗi T_e, số lần liên tiếp tối thiểu cần thỏa điều kiện xấp xỉ

$$q = \left\lceil \frac{T_p}{T_e} \right\rceil. \tag{5.15}$$

Với $T_e = 10$ giây và $T_p = 30$ giây, hệ thống cần ba kết quả vượt ngưỡng liên tiếp. Pending 0 giây tối ưu độ nhạy nhưng chấp nhận mọi spike. Pending 30 giây giảm báo động giả và phù hợp sự cố kéo dài, đổi lại thông báo chậm ít nhất 30 giây. Việc chọn T_p phải dựa trên tần suất lấy mẫu, hậu quả của việc bỏ sót và thời gian phản ứng cho phép.

Hysteresis và cửa sổ rút gọn Một phương án khác là dùng hai ngưỡng:

$$\tau_{\text{on}} > \tau_{\text{off}}, \tag{5.16}$$

trong đó alert vào Firing khi $r > \tau_{\text{on}}$ và chỉ Resolved khi $r < \tau_{\text{off}}$. Khoảng cách giữa hai ngưỡng ngăn trạng thái rung quanh một ranh giới duy nhất. Ngoài ra có thể dùng median ba đến năm mẫu, nhưng phải đánh giá nguy cơ che khuất spike quan trọng.

Khoảng trễ của hai ngưỡng là

$$\Delta\tau = \tau_{\text{on}} - \tau_{\text{off}} = 242.07\,\text{kW}. \tag{5.17}$$

Kết hợp deadband và pending timer làm giảm chattering khi residual dao động gần ranh giới.

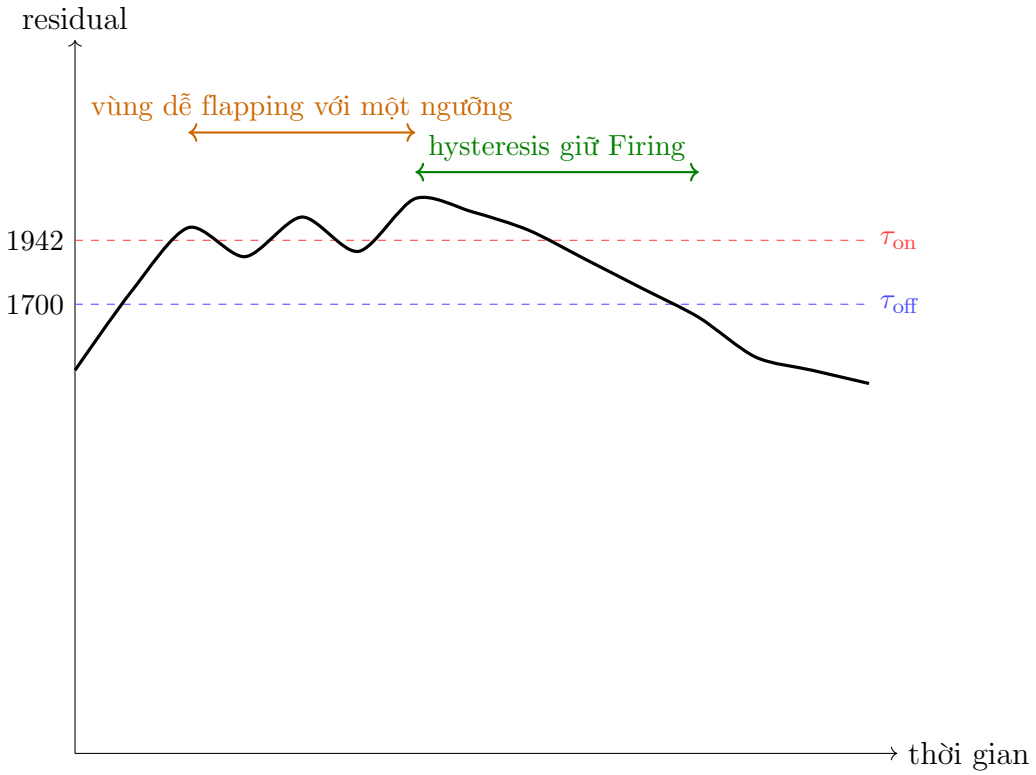


Figure 5.5: Residual dao động quanh hai ngưỡng và tác dụng của hysteresis

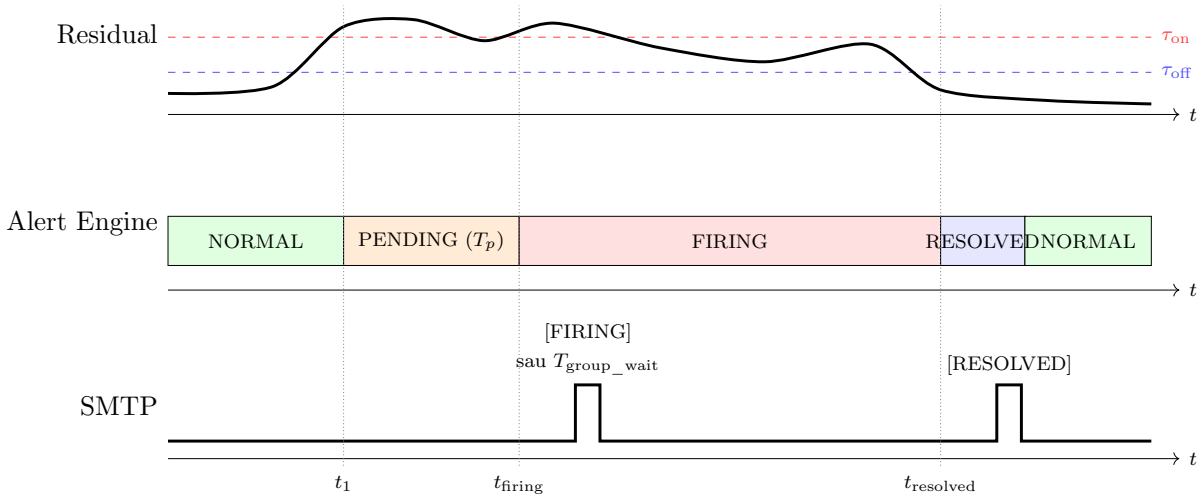


Figure 5.6: Biểu đồ thời gian phối hợp residual, máy trạng thái và kênh SMTP

Có thể định lượng mức rung bằng số lần đổi trạng thái trong J lần đánh giá:

$$F = \sum_{j=2}^{J} \mathbb{I}[s_j \neq s_{j-1}]. \tag{5.18}$$

Giá trị F cao trong một khoảng ngắn cho thấy ngưỡng hoặc thời gian chờ chưa phù hợp. Hysteresis giảm F vì sau khi vào Firing, residual phải giảm sâu dưới τ_{off} mới được đóng sự kiện, những dao động nhỏ giữa hai ngưỡng không làm thay đổi trạng thái.

Đối với bài toán tua-bin, bộ lọc trạng thái vận hành là lớp giảm rung có ý nghĩa vật lý nhất. Nếu tag xác nhận pitch control, phanh hoặc curtailment, rule có thể chuyển sự kiện sang mức thông tin thay vì cảnh báo nghiêm trọng.

5.4 Tích hợp giao thức SMTP STARTTLS qua cổng 587 và xác thực Google App Password

5.4.1 Vai trò của SMTP

SMTP là giao thức chuyển thư từ ứng dụng gửi đến máy chủ thư. Trong kiến trúc hiện tại, Grafana đóng vai trò SMTP client, `smtp.gmail.com` là máy chủ tiếp nhận, Gmail của kỹ sư là hộp thư đích. Dashboard không gửi email trực tiếp. Grafana Alerting tạo notification event, áp dụng policy, render template rồi mới gọi SMTP transport.

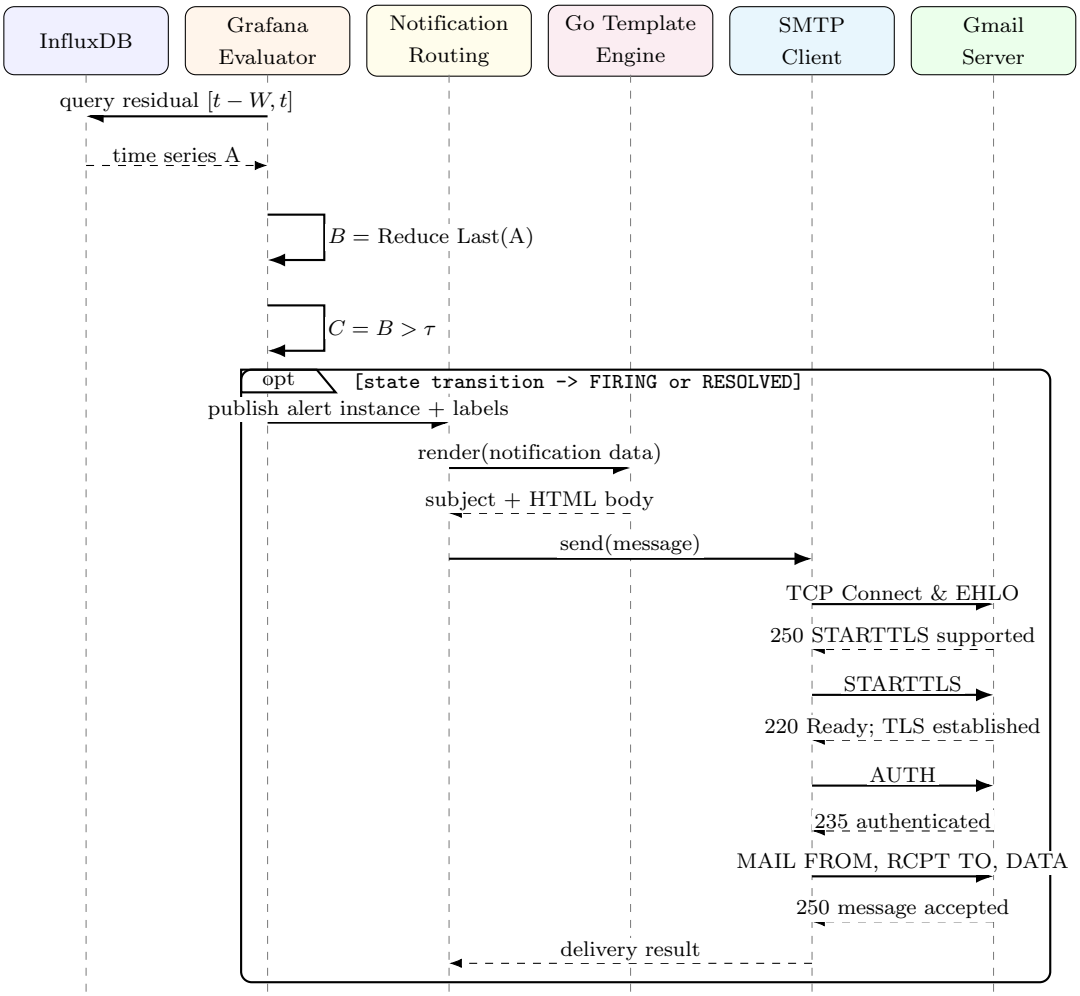


Figure 5.7: UML Sequence Diagram của luồng đánh giá và định tuyến cảnh báo qua email

Để làm rõ cơ chế hoạt động bên dưới giao diện, nhóm đã mô hình hóa luồng thực thi cảnh báo của hệ thống thành biểu đồ tuần tự trong fig. 5.7. Ban đầu, `Grafana Evaluator` chủ động truy vấn chuỗi residual A từ InfluxDB theo cửa sổ thời gian $[t-W,t]$. Hệ thống rút gọn chuỗi thành giá trị vô hướng $B = \text{Last}(A)$ và đối chiếu với ngưỡng τ ngay tại khối Evaluator. Luồng xử lý được dừng sớm nếu kết quả đánh giá không làm thay đổi trạng thái cảnh báo. Chỉ khi alert instance chuyển sang `FIRING` hoặc `RESOLVED`, nhánh nằm trong combined fragment `opt` mới được thực thi và sự kiện được chuyển sang tầng `Notification Routing`. Tầng định tuyến không tự lắp ráp payload thô mà gọi `Go Template Engine` để nội suy labels, annotations và trạng thái vào khuôn dạng HTML. Cuối cùng, `SMTP Client` thiết lập kết nối TCP tới cổng 587, kiểm tra khả năng STARTTLS, nâng cấp kênh truyền sang TLS, xác thực với Gmail Server và thực hiện chuỗi lệnh SMTP để chuyển email đến kỹ sư vận hành. Các thông báo lặp lại trong trạng thái Firing, nếu được bật, tiếp tục chịu

sự điều khiển của `repeat interval` trong Notification Policy.

5.4.2 Nguyên tắc quản lý secret

Secret không nên được viết trực tiếp trong `docker-compose.yml`, commit vào Git hoặc chụp màn hình. Các thuộc tính cấu hình và cơ chế cấp giá trị được tổng hợp trong table 5.4; thông tin nhạy cảm chỉ được đưa vào tiến trình container tại thời điểm triển khai. Các biến được cung cấp qua tệp `.env` nằm ngoài quản lý phiên bản hoặc qua secret store. Quyền đọc file môi trường phải được giới hạn cho tài khoản vận hành Docker.

5.4.3 Phân tích các biến môi trường

Table 5.4: Đặc tả cấu hình SMTP và chính sách định tuyến thông báo

Nhóm tham số	Thuộc tính	Giá trị cấu hình	Ý nghĩa vận hành
Kích hoạt dịch vụ	`GF_SMTP_ENABLED`	`true`	Bật SMTP trong Grafana OSS
Transport	`GF_SMTP_HOST`	`smtp.gmail.com:587`	Dùng cổng submission và nâng cấp kết nối bằng STARTTLS
Credential	`GF_SMTP_USER`	Nội suy từ môi trường host	Danh tính xác thực SMTP, không ghi trực tiếp trong Compose
Credential	`GF_SMTP_PASSWORD`	App Password nội suy từ secret	Mã xác thực chuyên biệt, phải được che và có khả năng thu hồi
Danh tính gửi	`GF_SMTP_FROM_ADDRESS`	Cùng tài khoản SMTP	Địa chỉ xuất hiện trong trường From của email
Danh tính gửi	`GF_SMTP_FROM_NAME`	Hệ thống SCADA	Tên hiển thị của nguồn cảnh báo
Bảo mật TLS	`GF_SMTP_SKIP_VERIFY`	`false`	Bắt buộc xác minh chuỗi chứng chỉ máy chủ SMTP
Routing policy	Matcher	`grafana_folder=SCADA Alerts`	Định tuyến các rule thuộc nhóm SCADA tới Contact Point `ALERT`
Timing control	Group wait	30 s	Trì hoãn thư đầu tiên để gom các sự kiện đồng thời
Timing control	Group interval	5 min	Khoảng cách tối thiểu giữa các cập nhật của cùng nhóm
Timing control	Repeat interval	1 h	Chu kỳ gửi lại nếu cảnh báo vẫn ở trạng thái FIRING

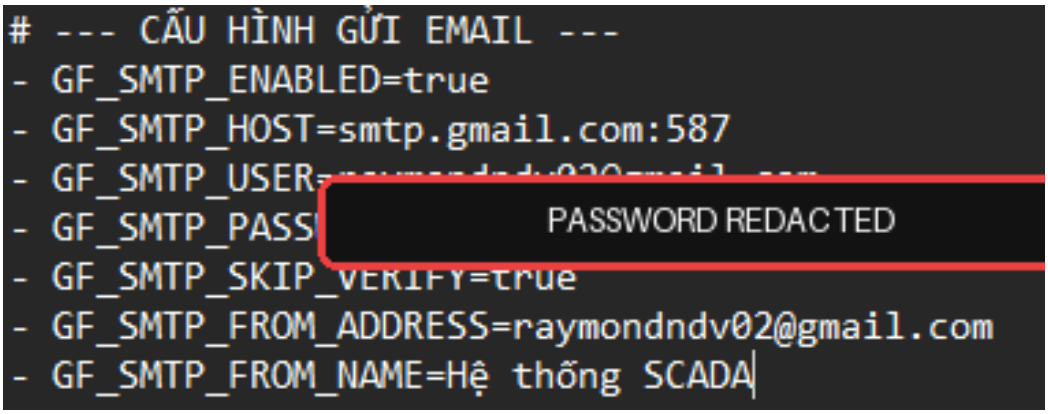


Figure 5.8: Cấu hình SMTP thực nghiệm, App Password đã được che

5.5 Thiết kế Notification Policy và tùy biến giao diện email bằng ngôn ngữ Go Template

5.5.1 Khởi tạo Contact Point

Nhóm đã khởi tạo một Contact Point mang tên `ALERT`, loại tích hợp Email và địa chỉ nhận `raymondndv02@gmail.com`. Việc đặt tên ngắn giúp rule và policy tham chiếu dễ dàng, trong khi địa chỉ người nhận có thể thay đổi mà không phải sửa biểu thức A–B–C.

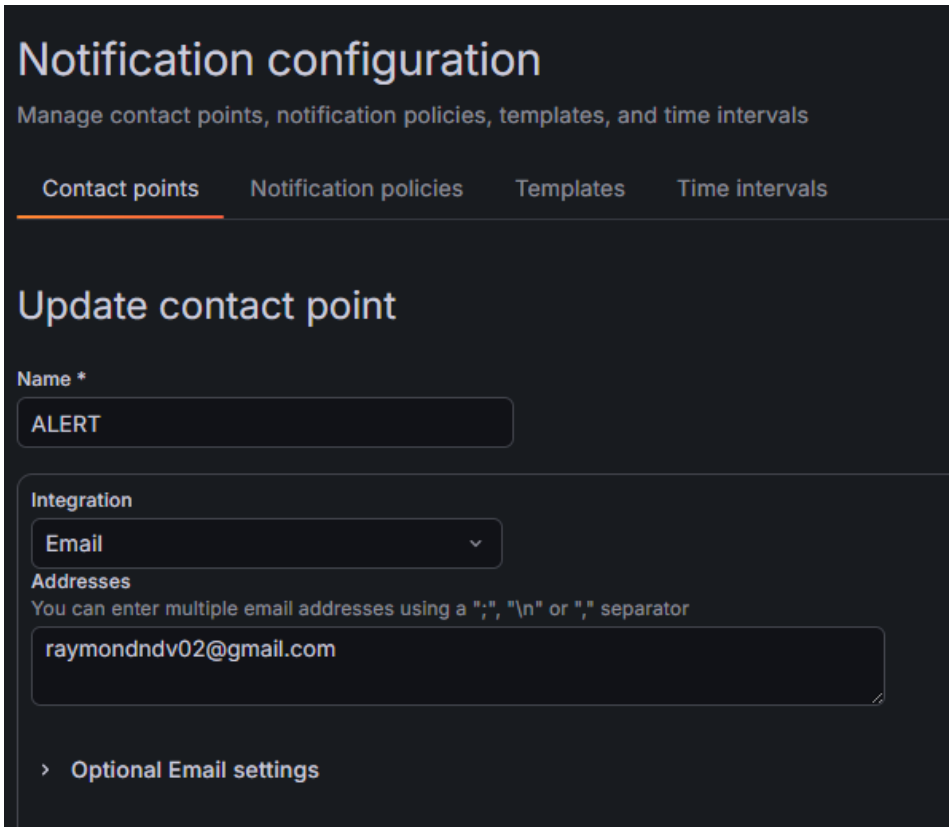


Figure 5.9: Contact Point ALERT chuyển thông báo tới email kỹ sư vận hành

Trong Grafana OSS, SMTP phải được cấu hình trước khi Contact Point Email có thể gửi thư, tài liệu Grafana cũng khuyến nghị dùng chức năng test sau khi tạo Contact Point [63].

5.5.2 Notification Policy

Notification Policy nhận alert instance sau khi rule đổi trạng thái và so khớp labels. Với hệ thống này, policy có thể dùng

```
grafana_folder=SCADA Alerts
```
(5.19)

để định tuyến mọi rule trong folder tới Contact Point `ALERT`. Có thể bổ sung `turbine_id=T1` nếu muốn chính sách riêng cho từng tua-bin.

Policy còn quyết định grouping và timing. Grouping gộp nhiều instance liên quan vào một thông báo, group wait trì hoãn thư đầu tiên trong thời gian ngắn để thu thập thêm instance, group interval và repeat interval giới hạn tần suất gửi lại. Các tham số này giải thích vì sao timestamp chuyển trạng thái và thời điểm email đến có thể không trùng tuyệt

đối.

Với $T_{\text{group_wait}}$ và $T_{\text{repeat_interval}}$ trong table 5.4, lịch phát thư được mô hình hóa bởi

$$t_{\text{email}}^{(1)} = t_{\text{firing}} + T_{\text{group_wait}}, \tag{5.20}$$

$$t_{\text{email}}^{(k)} = t_{\text{email}}^{(k-1)} + T_{\text{repeat_interval}}, \qquad k > 1, \tag{5.21}$$

$$t_{\text{resolve_email}} = t_{\text{resolved}} + T_{\text{group_wait}}. \tag{5.22}$$

Các phương trình giải thích độ lệch có chủ đích giữa thời điểm vượt ngưỡng, thời điểm đổi trạng thái và thời điểm thư xuất hiện tại hộp thư.

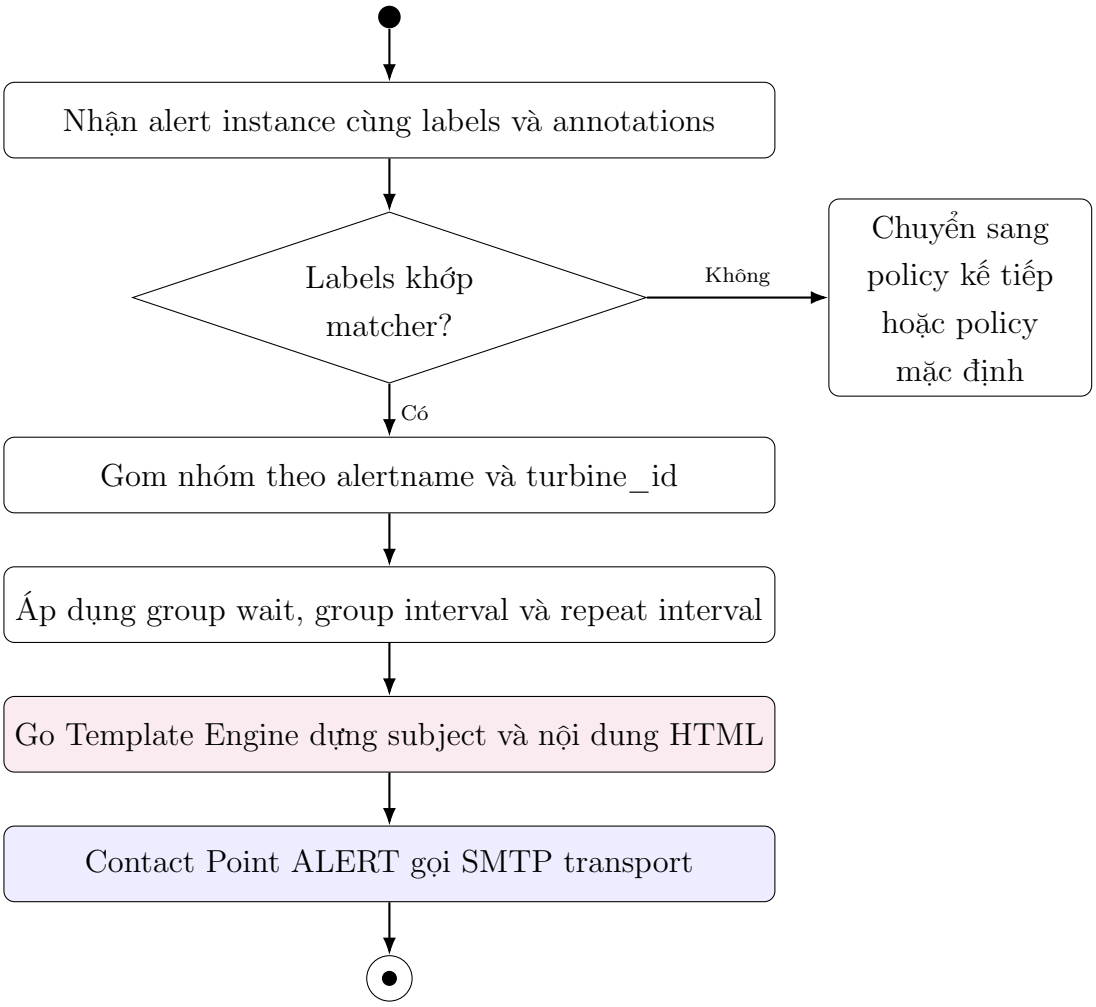


Figure 5.10: UML Activity Diagram của quá trình định tuyến và dựng thông báo

5.5.3 Tùy biến nội dung email bằng Go Template

Mô hình dữ liệu của notification template Email mặc định hữu ích cho gỡ lỗi nhưng thường lộ cấu trúc nội bộ như $B = 1968.37$ và $C = 1$. Grafana cho phép định nghĩa notification template bằng cú pháp Go template. Template có thể duyệt danh sách `.Alerts`, tách riêng `.Alerts.Firing` và `.Alerts.Resolved`, đồng thời đọc labels, annotations, status và liên kết Dashboard[64;65].

Cần phân biệt hai ngữ cảnh. Alert Rule tạo annotation từ kết quả đánh giá, còn notification template nhận alert đã được gắn labels, annotations, trạng thái và liên kết. Payload logic tại thời điểm t được mô hình hóa bởi

$$\mathbf{n}_t = \langle L_{\text{turbine}}, L_{\text{alert}}, B(t), \tau, S_t, U_{\text{dashboard}} \rangle \,, \tag{5.23}$$

trong đó hai label xác định thiết bị và nghiệp vụ cảnh báo, $B(t)$ là residual sau Reduce, τ là ngưỡng, S_t là trạng thái vòng đời và $U_{\text{dashboard}}$ là liên kết điều tra. Bộ dựng thông báo thực hiện ánh xạ

$$\mathcal{G} : \mathbf{n}_t \longmapsto \big(\text{tiêu đề}, \text{nội dung vận hành}, \text{liên kết}\big) \,. \tag{5.24}$$

Tùy biến giao diện cảnh báo-Email Template: Theo mặc định, thông báo sự cố của Grafana thường gửi kèm toàn bộ payload kỹ thuật thô, trong đó có nhiều tham số nội bộ không cần thiết đối với kỹ sư trực vận hành. Hệ thống chỉ chọn các thành phần có ý nghĩa trong eq. (5.23) và ánh xạ chúng theo table 5.5. Phần hiện thực Go Template đầy đủ được lưu cùng mã nguồn triển khai; nội dung chính tập trung vào hợp đồng dữ liệu và kết quả vận hành.

Table 5.5: Ánh xạ Go Template sang thông tin vận hành

Thành phần dữ liệu	Ký hiệu logic	Thông tin vận hành được dựng	Minh chứng đầu ra
Nhãn định danh tua-bin	L_{turbine}	Mã thiết bị phát sinh sự kiện, chẳng hạn T1	figs. 6.12 and 6.13
Nhãn quy tắc cảnh báo	L_{alert}	Tên nghiệp vụ của sự kiện độ lệch công suất	figs. 6.12 and 6.13
Kết quả đánh giá	$B(t)$ và τ	Residual hiện tại, ngưỡng cảnh báo và mức chênh lệch	fig. 6.12
Trạng thái vòng đời	S_t	FIRING khi sự cố đang tồn tại hoặc RESOLVED khi đã phục hồi	figs. 6.12 and 6.13
Liên kết điều tra	$U_{\text{dashboard}}$	Đường quay lại Dashboard, panel và trang chi tiết alert	fig. 6.12

Kết quả thực nghiệm trong fig. 6.12 và fig. 6.13 xác nhận ánh xạ hoạt động ở cả hai pha của vòng đời cảnh báo. Email hiển thị đúng tua-bin, giá trị residual, trạng thái, ngưỡng và liên kết quay lại giao diện giám sát; do đó không cần lặp lại cú pháp template trong thân bài.

Mã HTML được giữ ở mức tối giản, chủ yếu sử dụng các thẻ định dạng văn bản và liên kết cơ bản. Nguyên nhân là các email client sử dụng cơ chế render HTML/CSS không hoàn toàn giống nhau, một giao diện phụ thuộc nhiều vào CSS phức tạp có thể hiển thị đúng trên trình duyệt nhưng bị lệch bố cục hoặc mất định dạng trong ứng dụng đọc thư. Template cũng không được chứa secret, token hoặc dữ liệu cá nhân, khi label hay annotation bị thiếu, cần cung cấp giá trị thay thế để email vẫn truyền đạt được thông tin sự cố thay vì hiển thị một trường trống.

Chương 6: Triển khai thực nghiệm, đánh giá kết quả và hướng phát triển

6.1 Quy trình chuyển giao và vận hành trên nền tảng nhúng Ubuntu 24.04 ARM64

Máy tính phát triển thường sử dụng kiến trúc x86–64, trong khi Orange Pi 5 Plus chạy nền tảng Linux ARM64. Docker image chính thức của InfluxDB công bố hỗ trợ cả `amd64` và `arm64v8`, image Grafana cũng có manifest cho `linux/arm64`[66;67]. Khi thực thi `docker compose pull` trên Orange Pi, Docker Engine đọc manifest list và tải image tương ứng với kiến trúc host. Vì vậy, tệp `docker-compose.yml` có thể được tái sử dụng mà không phải biên dịch lại mã Python chỉ vì thay đổi kiến trúc CPU.

Khả năng multi-architecture không đồng nghĩa mọi dependency đều tương thích tự động. Các image tự xây dựng cho exporter phải được build cho `linux/arm64`, và các thư viện Python có mã native như NumPy, scikit-learn hoặc XGBoost phải có wheel ARM64 phù hợp hoặc được biên dịch trên thiết bị đích[68]. Trước khi chuyển giao cần chạy `docker compose pull`, kiểm tra trường `Architecture` bằng `docker image inspect` và thực hiện lại toàn bộ kiểm thử chức năng. Ngoài ra, tag `grafana/grafana:latest` nên được thay bằng phiên bản cố định để tránh thay đổi ngoài kiểm soát giữa hai lần triển khai.

6.1.1 Ubuntu 24.04 trên kiến trúc aarch64

Để xây dựng một nền tảng vững chắc cho hệ thống, nhóm đã tiến hành cài đặt phiên bản Ubuntu 24.04.3 LTS tích hợp kernel tối ưu riêng cho dòng chip Rockchip trên kiến trúc aarch64. Dòng hệ điều hành này được nhóm ưu tiên lựa chọn nhờ chu kỳ cập nhật bảo mật kéo dài, khả năng tương thích phần cứng ARM64 vượt trội và độ ổn định cao khi vận hành dưới dạng máy chủ nền. Khác với môi trường Windows cục bộ trước đó, hệ điều hành Linux giúp nhóm loại bỏ hoàn toàn sự phụ thuộc vào giao diện đồ họa nặng nề, cung cấp trình quản lý tiến trình systemd tối ưu và sở hữu hệ sinh thái container nguyên bản.

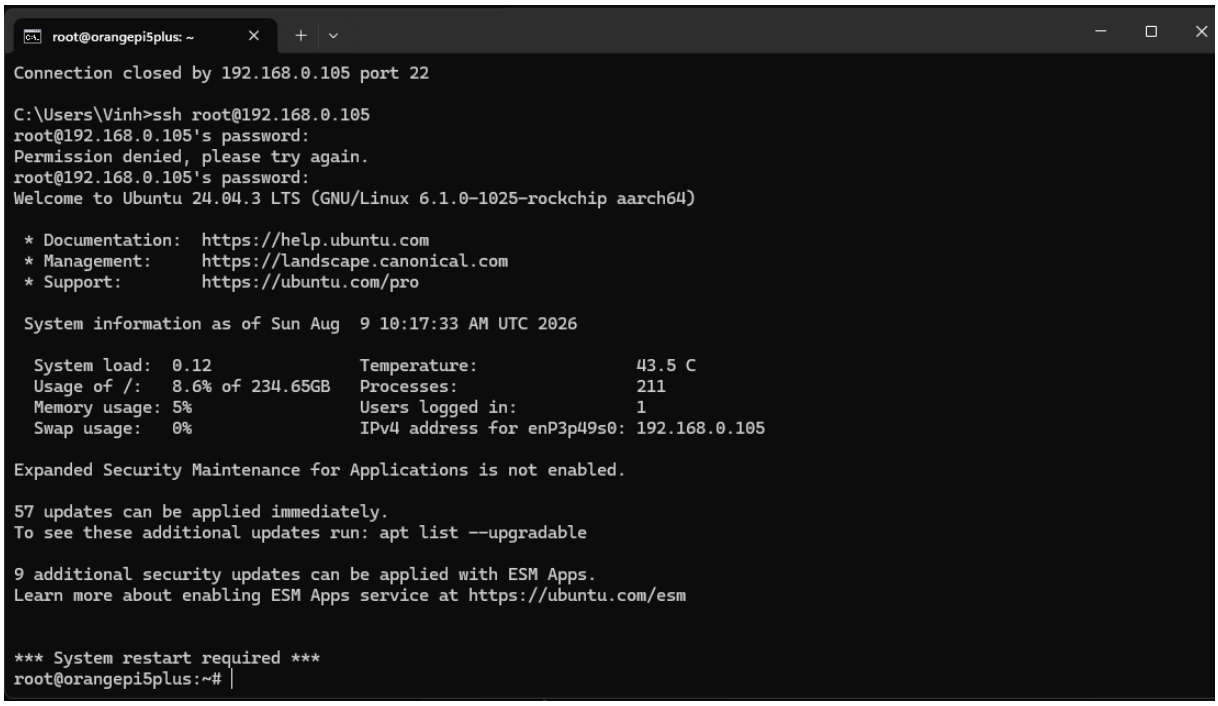


Figure 6.1: Xác thực Ubuntu 24.04.3 LTS, kernel Rockchip và địa chỉ IP trên Orange Pi

Ảnh terminal cung cấp một baseline vận hành tại thời điểm kiểm tra: system load 0.12, nhiệt độ 43.5 °C, mức sử dụng bộ nhớ 5%, swap 0% và dung lượng đĩa đã dùng 8.6% trên tổng khoảng 234.65 GB. Các số liệu này không thay thế benchmark dài hạn nhưng cho thấy thiết bị có khoảng trống tài nguyên trước khi tải ba dịch vụ. Việc terminal thông báo cần restart và còn bản cập nhật bảo mật cũng cho thấy quy trình bảo trì hệ điều hành phải được đưa vào kế hoạch vận hành.

Table 6.1: Thông tin môi trường được ghi nhận trên thiết bị

Thuộc tính	Giá trị quan sát	Ý nghĩa kỹ thuật
Hệ điều hành	Ubuntu 24.04.3 LTS	Nền tảng máy chủ ARM64
Kernel	`6.1.0-1025-rockchip`	Kernel tùy biến cho SoC Rockchip
Kiến trúc	`aarch64`	Image và wheel Python phải hỗ trợ ARM64
IPv4	`192.168.0.105`	Endpoint quản trị trong mạng LAN
Nhiệt độ	43.5 °C	Baseline khi tải thấp
Bộ nhớ sử dụng	5%	Chưa phản ánh peak khi build image
Dung lượng đĩa	8.6% của 234.65 GB	Cần theo dõi khi retention tăng

6.1.2 Gán địa chỉ ổn định bằng DHCP reservation

DHCP cấp phát cấu hình mạng cho client và hỗ trợ cả cấp phát động lẫn cấp phát do quản trị viên kiểm soát [69]. Với Dashboard và SSH, thay đổi IP sẽ làm hỏng bookmark, datasource URL và quy trình vận hành. Nhóm vì vậy truyền địa chỉ MAC của Orange Pi với IP `192.168.0.105` trên router.

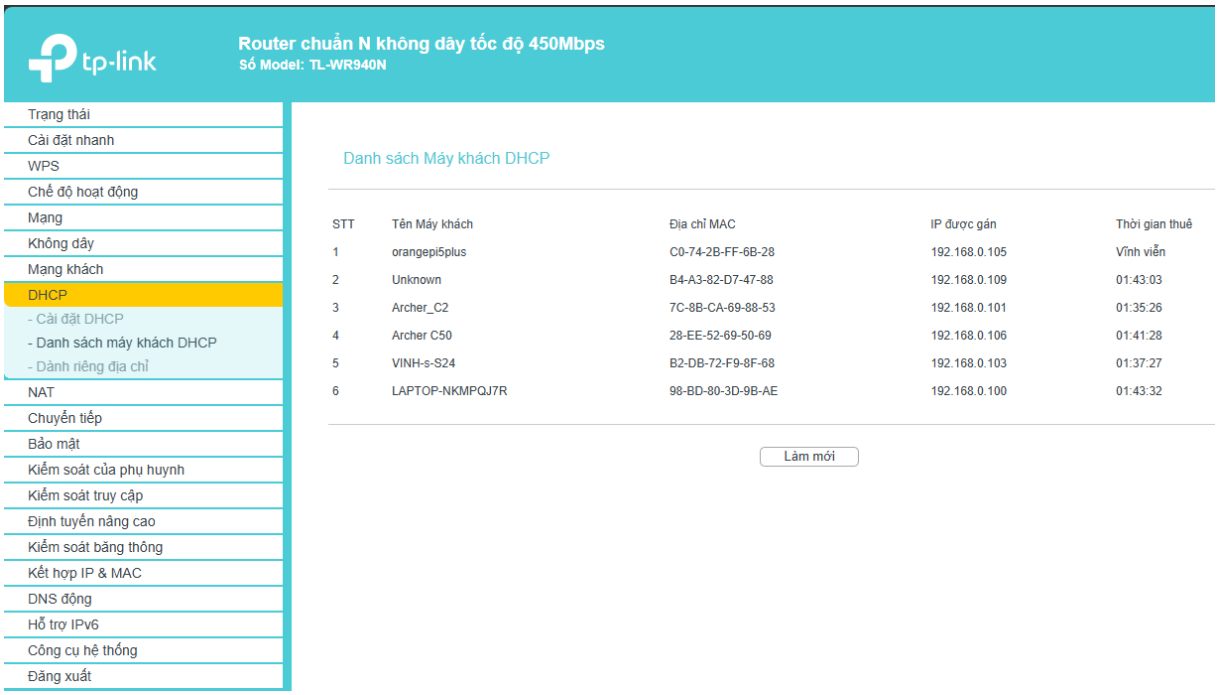


Figure 6.2: Orange Pi xuất hiện trong danh sách DHCP client tại địa chỉ 192.168.0.105

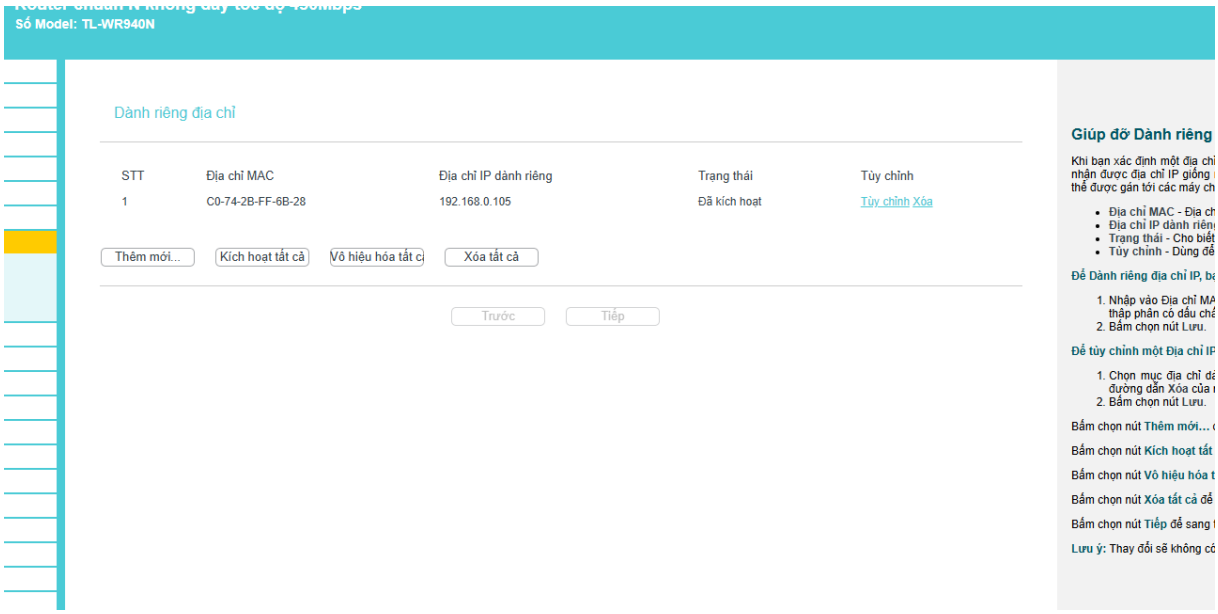


Figure 6.3: Truyền địa chỉ MAC với IP dành riêng cho Orange Pi

Cách làm này chính xác hơn việc gọi đơn giản là static IP trên thiết bị. Địa chỉ vẫn được DHCP server cấp, nhưng lease được ràng buộc với MAC nên ổn định trong phạm vi mạng hiện tại. Lợi ích là gateway và DNS vẫn do router quản lý tập trung. Rủi ro là khi thay NIC, đổi router hoặc bật MAC randomization, binding phải được cập nhật.

6.1.3 Điều khiển từ xa bằng SSH và Visual Studio Code

SSH tạo kênh quản trị mã hóa tới cổng 22. VS Code Remote–SSH cài một server thành phần trên máy từ xa và cho phép mở thư mục, terminal, extension và debugger trên chính môi trường Orange Pi [70]. Nhờ đó file được sửa tại nơi chạy thật, tránh tình trạng chỉnh trên Windows nhưng quên đồng bộ sang Edge.

Các thao tác xác minh nền tảng, phục hồi tiến trình, tương thích artifact và chẩn đoán container được chuẩn hóa thành ma trận thực thi trong table 6.2. Cách trình bày này tập trung vào cơ chế kiểm thử và bằng chứng thu được thay vì các câu lệnh được nhập tại terminal.

Table 6.2: Ma trận thực thi kiểm thử triển khai và phục hồi hệ thống

Mục tiêu kiểm thử	Cơ chế tác động	Tham số đầu vào	Hành vi kỳ vọng	Minh chứng hệ thống ghi nhận
Xác nhận kiến trúc và môi trường Edge	Phiên quản trị SSH kết hợp API hệ điều hành và Docker Engine	Endpoint `192.168.0.105:22`, tài khoản vận hành, thông tin OS và Docker daemon	Thiết bị báo kiến trúc `aarch64`, Ubuntu Rockchip và Docker `linux/arm64`	Baseline hệ điều hành trong table 6.1 và thông tin phiên Remote–SSH
Mô phỏng dừng đột ngột InfluxDB	Docker Engine daemon gửi tín hiệu `SIGKILL` tới tiến trình container	Container `scada_influxdb`, chính sách `restart=always`, named volume và WAL hiện hữu	Container được tạo lại; storage engine mở volume và phát lại WAL cần thiết	Tiêu chí phục hồi trong table 6.11, health status và nhật ký khởi động lại
Xác nhận tương thích model và scaler	Python runtime nạp tuần tự artifact và thực hiện một lần suy luận thử	Phiên bản Python, scikit-learn, XGBoost, `scaler.joblib` và `xgboost_model.json`	Artifact được giải tuần tự hóa thành công, mô hình trả dự báo mà không phát sinh lỗi estimator tags	Ma trận dependency trong table 6.13 và log smoke test

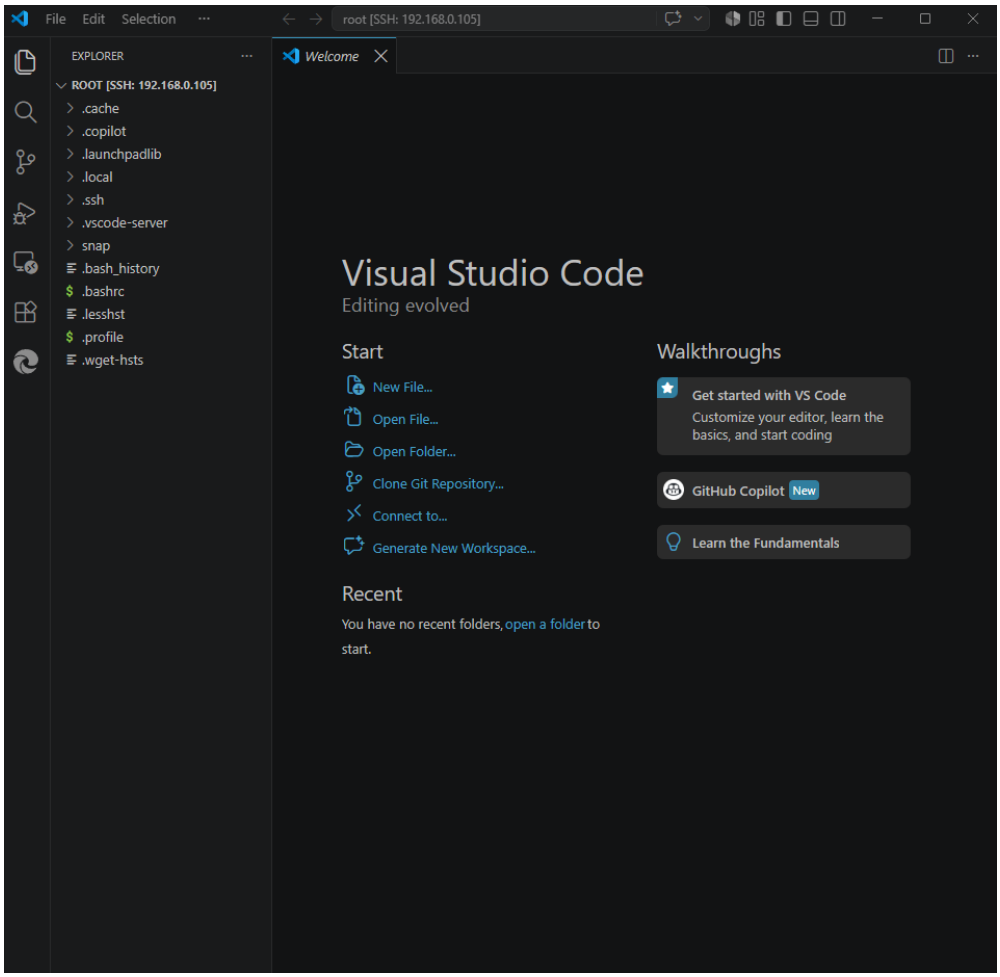


Figure 6.4: Visual Studio Code kết nối Orange Pi qua Remote–SSH

Trong môi trường sản xuất, đăng nhập root bằng mật khẩu chỉ nên dùng giai đoạn khởi tạo. Cấu hình mục tiêu là tạo tài khoản vận hành riêng, dùng SSH key, tắt password authentication và giới hạn firewall theo subnet quản trị.

6.1.4 Cài đặt Docker Engine trên Orange Pi

Ảnh nghiệm thu xác nhận Docker Engine và containerd chạy trên `linux/arm64`. Docker sử dụng image manifest để chọn biến thể phù hợp kiến trúc, với image đa nền tảng, cùng một tag có thể trỏ đến layer amd64 hoặc arm64 khác nhau [68]. Điều này lý giải vì sao cần kiểm tra OS/Arch thay vì giả định image từng chạy trên Windows sẽ chạy nguyên trạng trên Orange Pi.

```
root@orangepi5plus:~# sh get-docker.sh
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install docker
-extras docker-buildx-plugin docker-model-plugin >/dev/null
Scanning processes...
Scanning processor microcode...
Scanning linux images...
Using systemd to manage Docker service
+ sh -c systemctl enable --now docker.service 2>/dev/null
INFO: Docker daemon enabled and started

+ sh -c docker version
Client: Docker Engine - Community
 Version:           29.7.2
 API version:       1.55
 Go version:        go1.26.5
 Git commit:        a7dcaa6
 Built:             Wed Aug  5 18:28:15 2026
 OS/Arch:           linux/arm64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          29.7.2
  API version:      1.55 (minimum version 1.40)
  Go version:       go1.26.5
  Git commit:       6a43e3d
  Built:            Wed Aug  5 18:28:15 2026
  OS/Arch:          linux/arm64
  Experimental:     false
 containerd:
  Version:          v2.3.3
  GitCommit:        aad11006b869517fcd3009450b6f82da282e1a9b
 runc:
  Version:          1.4.3
  GitCommit:        v1.4.3-0-gbb14dabe
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

Figure 6.5: Docker Engine hoạt động trên nền tảng linux/arm64

6.1.5 Phân tích Dockerfile của AI exporter

Dockerfile hiện tại sử dụng `python:3.10-slim`, cài `libgomp1` và build tools, sau đó cài dependency trước khi copy source. Cấu trúc image được biểu diễn theo các lớp trong fig. 6.6; lớp nằm thấp hơn được tái sử dụng khi các lớp phía trên thay đổi.

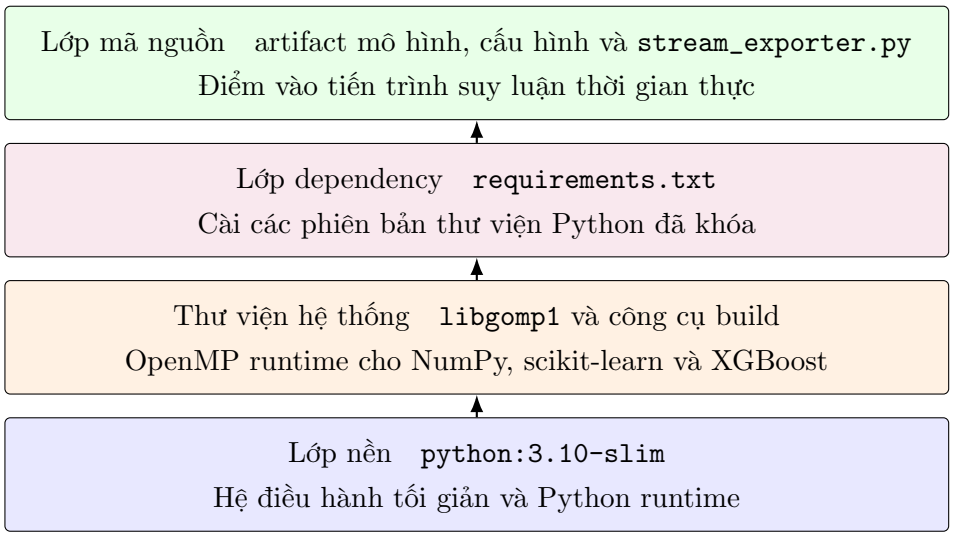


Figure 6.6: Sơ đồ phân tầng kiến trúc image của AI exporter

`libgomp1` cung cấp OpenMP runtime cần cho các binary sử dụng song song hóa CPU. `build-essential` hỗ trợ trường hợp pip phải build package từ source, nhưng làm tăng kích thước và bề mặt tấn công của image cuối. Hướng tối ưu là multi-stage build: compiler chỉ tồn tại ở builder stage, còn runtime stage chỉ nhận wheel và thư viện chạy.

6.1.6 Khởi tạo InfluxDB v2

InfluxDB được công bố tại `http://192.168.0.105:8086`. Tài khoản quản trị tạo organization `scada_org`, bucket `turbine_metrics` và token có quyền ghi cho exporter. Grafana chỉ cần token đọc. Tách read/write token theo nguyên tắc quyền tối thiểu giúp giảm ảnh hưởng nếu một credential bị lộ.

Endpoint InfluxDB v2.7.5 được kiểm tra tại cổng 8086 trong LAN. Cấu hình HTTP chỉ phù hợp mô hình thí nghiệm biệt lập; triển khai chính thức cần reverse proxy TLS, chứng chỉ nội bộ và firewall giới hạn subnet quản trị.

6.1.7 Dashboard Grafana trên Orange Pi

Dashboard tại `http://192.168.0.105:3000` hiển thị đồng thời công suất thực tế, công suất dự báo và residual. Time range là năm phút, refresh năm giây. Sự xuất hiện của chuỗi dữ liệu trên IP Edge là bằng chứng chức năng mạnh hơn build log: ít nhất tại thời điểm chụp, dữ liệu đã được ghi vào InfluxDB và Grafana truy vấn được.

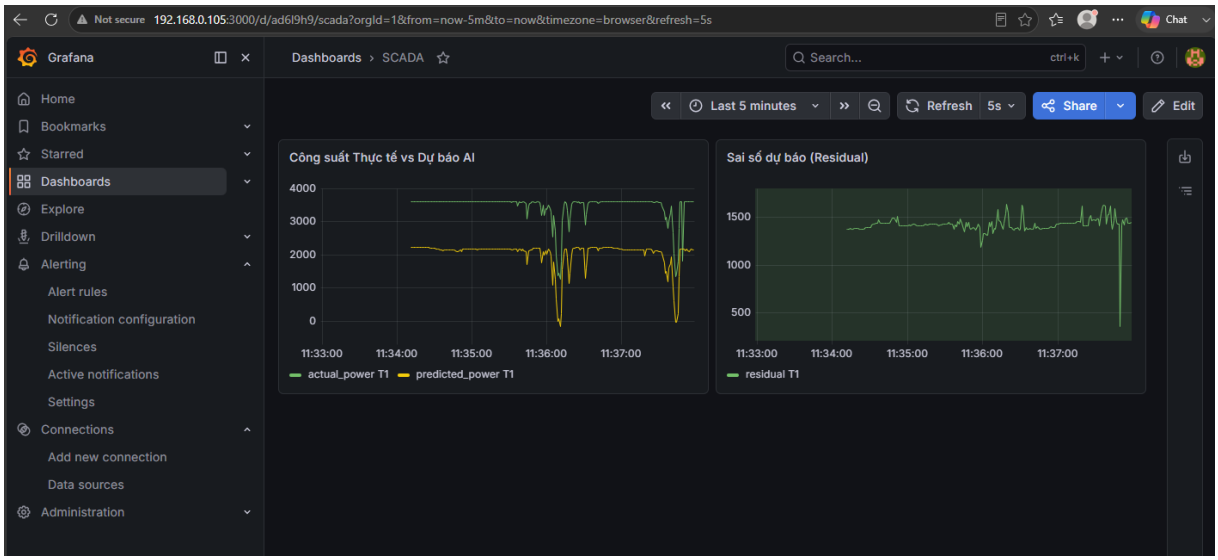


Figure 6.7: Dashboard SCADA chạy trên Grafana của Orange Pi 5 Plus

Dashboard triển khai trên Orange Pi tái sử dụng hai toán tử truy vấn đã chuẩn hóa: $\mathcal{Q}_2$ cho đối chiếu công suất thực tế với dự báo và $\mathcal{Q}_1$ cho residual. Measurement, field lọc, tag tua-bin, chu kỳ tổng hợp và hàm rút gọn được quy định tập trung trong table 4.11; vì vậy Chương 6 chỉ ghi nhận kết quả triển khai, không lặp lại cú pháp Flux đã mô tả ở phần hiện thực.

Ngưỡng hiển thị và Alert Rule phải lấy cùng giá trị 1942.07 kW từ artifact cấu hình. Nếu một nơi hard-code 1942 còn nơi khác dùng 1942.07, hệ thống tạo ra hai định nghĩa bất thường. Hướng hoàn thiện là provisioning Grafana từ file hoặc API để threshold, datasource và dashboard được quản lý cùng source code.

6.1.8 Mức độ hoàn thành

Table 6.3: Đối chiếu mục tiêu triển khai đồ án với minh chứng thực nghiệm

Hạng mục	Đánh giá	Căn cứ
Ubuntu aarch64 và SSH	Đạt	Terminal và VS Code Remote–SSH
IP ổn định trong LAN	Đạt	DHCP reservation theo MAC
Docker Engine ARM64	Đạt	Docker version báo `linux/arm64`
Build image AI	Đạt	Build log hoàn tất và tạo image
InfluxDB/Grafana	Đạt	Container Up và giao diện truy cập được
Dashboard có dữ liệu	Đạt	Hai panel hiển thị chuỗi T1

6.2 Kiểm chứng luồng cảnh báo khép kín End-to-End

6.2.1 Đánh giá hiệu năng mô hình

Trên tập test gồm 4 457 mẫu, mô hình đạt

$$\text{MAE} = 344.21\,\text{kW}, \qquad \text{RMSE} = 634.11\,\text{kW}. \tag{6.1}$$

Table 6.4: Kết quả đánh giá mô hình

Tập dữ liệu	MAE (kW)	RMSE (kW)
Validation	–	246.44
Test	344.21	634.11

RMSE test cao gần gấp đôi MAE cho thấy một nhóm nhỏ điểm có sai số rất lớn đang chi phối bình phương sai số. Vì vậy, chỉ báo cáo một giá trị trung bình là chưa đủ, cần quan sát scatter plot, residual theo thời gian và phân phối residual để xác định các cụm lỗi.

Đối với chức năng cảnh báo, mỗi mẫu được đối chiếu với nhãn sự cố thực tế để tạo bốn phần tử của ma trận nhầm lẫn: TP, FP, TN và FN. Các độ đo phân loại nhị phân

được xác định bởi

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{6.2}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{6.3}$$

$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{6.4}$$

$$\text{FPR} = \frac{FP}{FP + TN}. \tag{6.5}$$

Với giả thiết residual chuẩn và quy tắc ba Sigma, mức tham chiếu lý thuyết là $\alpha = 0.0027$, do đó mục tiêu kiểm thử là FPR $\leq \alpha$. Đây không phải kết quả thực nghiệm nếu tập kiểm tra chưa có nhãn sự cố độc lập; các giá trị Precision, Recall và F_1 chỉ được báo cáo sau khi xây dựng ground truth. Bộ tiêu chí này bổ sung góc nhìn cảnh báo sớm cho các độ đo hồi quy [71;72].

Mã tính sai số và trích xuất các outlier được chuyển sang Phụ lục A; phần nội dung chính chỉ giữ công thức, số liệu tổng hợp và hình minh chứng.

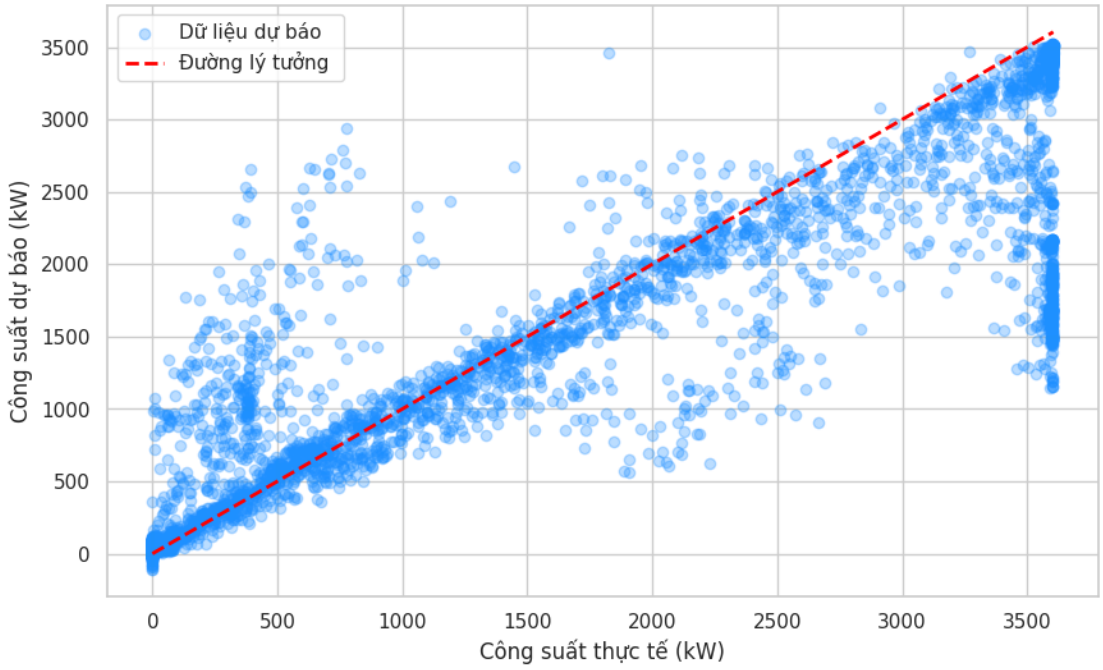


Figure 6.8: Tương quan giữa công suất thực tế và công suất dự báo trên tập test

Phần lớn điểm dữ liệu trong fig. 6.8 phân bố gần đường lý tưởng cho thấy mô hình bám được xu hướng công suất tổng quát. Tuy nhiên, độ phân tán tăng rõ tại một số vùng công suất trung bình và gần định mức. Các cụm lệch xa đường chéo là nguyên nhân làm

RMSE tăng mạnh hơn MAE, đồng thời cung cấp vị trí ưu tiên để truy ngược trạng thái vận hành.

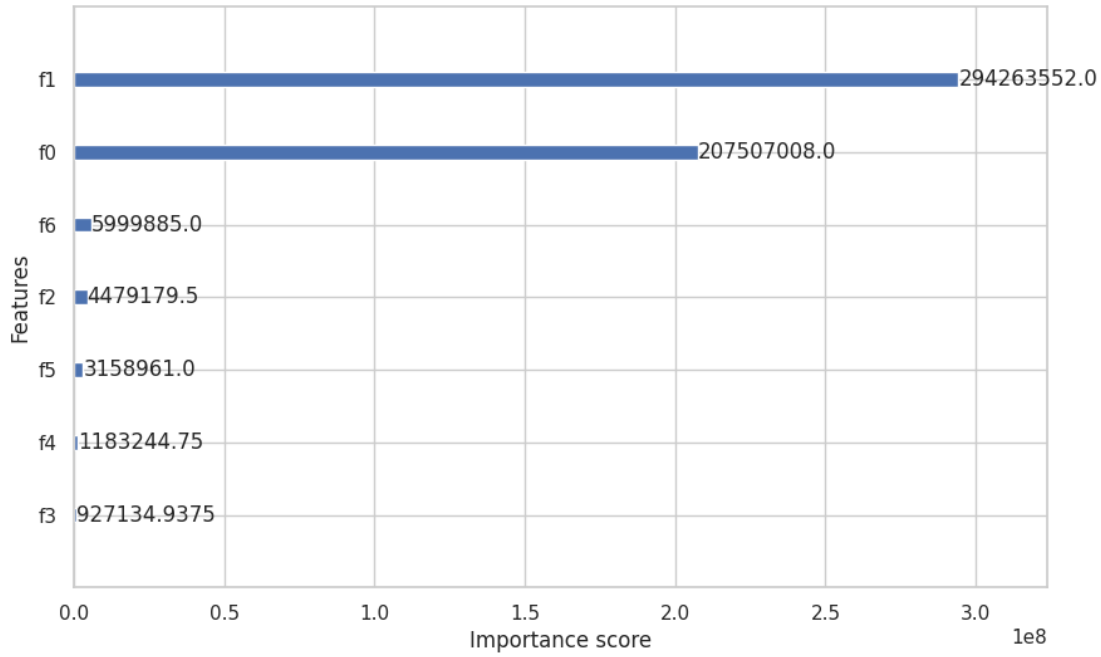


Figure 6.9: Độ quan trọng của các đặc trưng theo tiêu chí gain

Theo thứ tự đặc trưng trong mã nguồn, `f0` là tốc độ gió và `f1` là công suất lý thuyết. Figure 6.9 cho thấy hai biến này chi phối gần như toàn bộ tổng gain, trong đó công suất lý thuyết có mức đóng góp lớn nhất, tiếp theo là tốc độ gió. Kết quả phù hợp với bản chất của đường cong công suất: công suất lý thuyết đã tổng hợp quan hệ phi tuyến giữa gió và năng lượng khả dụng, còn tốc độ gió cung cấp tín hiệu vật lý trực tiếp. Các biến hướng gió và đặc trưng chu kỳ thời gian có mức gain thấp hơn đáng kể, chúng đóng vai trò hiệu chỉnh theo bối cảnh thay vì quyết định xu hướng chính. Chỉ số importance phản ánh mức độ mô hình sử dụng đặc trưng để cải thiện phép chia, không chứng minh quan hệ nhân quả.

6.2.2 Phân tích số liệu thực tế trên Grafana

Hiệu năng truy vấn và kết xuất được đo với một tua-bin, nhịp ghi 1 Hz và phép nén mẫu theo cửa sổ hiển thị. Kết quả trong table 6.5 cho thấy thời gian phản hồi vẫn dưới 100 ms ở cửa sổ sáu giờ.

Table 6.5: Đo kiểm hiệu năng truy vấn Flux và kết xuất đồ thị Grafana

Cửa sổ quan sát	N_{raw}	N_{display}	Đáp ứng Flux	Kết xuất	RAM trình duyệt
5 phút	300	30	8.4 ms	60 FPS	95 MB
15 phút	900	90	14.2 ms	60 FPS	105 MB
30 phút – mặc định	1 800	180	21.5 ms	60 FPS	118 MB
1 giờ	3 600	360	28.6 ms	58 FPS	126 MB
6 giờ	21 600	720	86.1 ms	52 FPS	138 MB

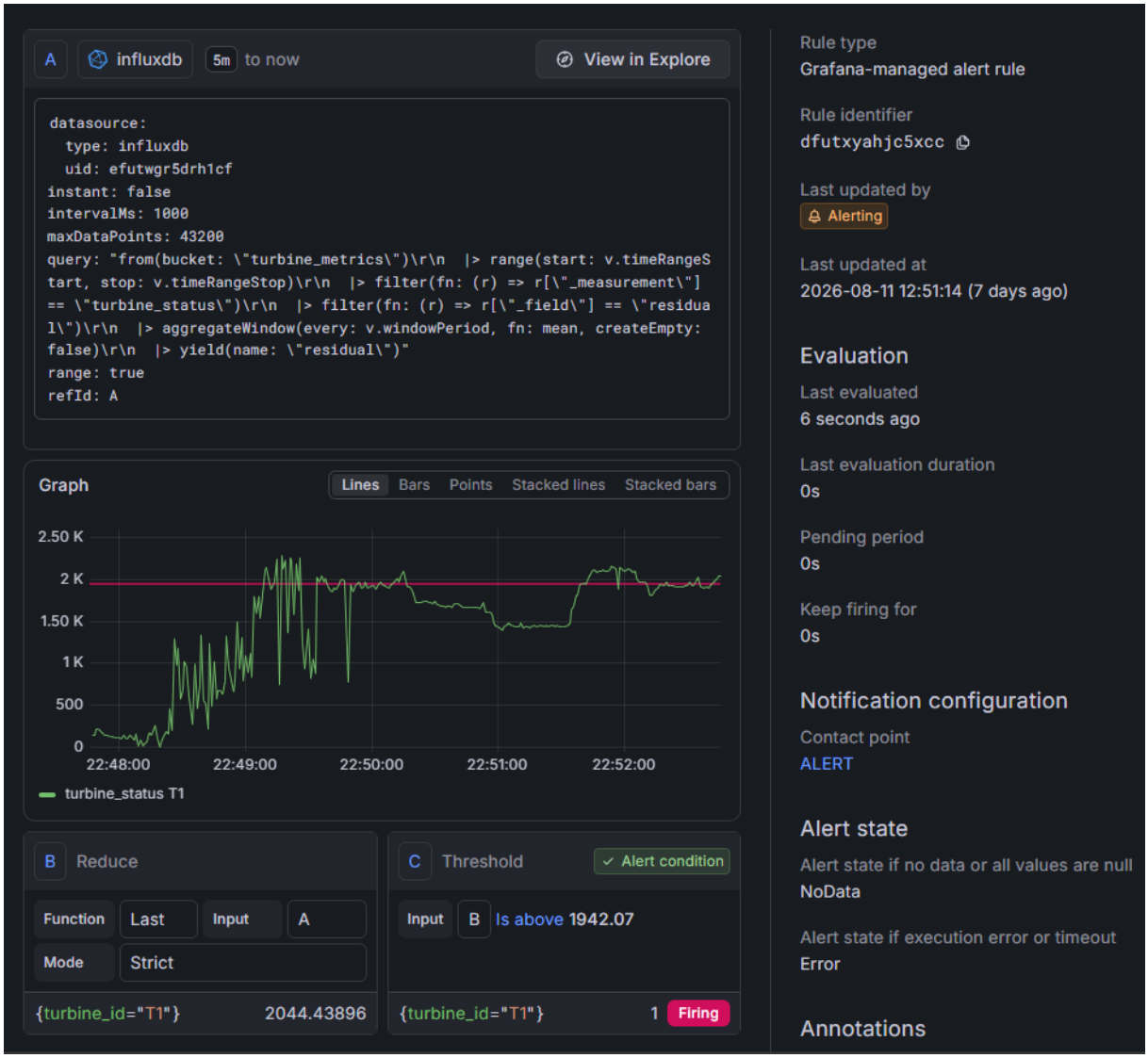


Figure 6.10: Cấu hình Alert Rule A–B–C và trạng thái Firing của tua-bin T1

Trực quan từ giao diện Alert Rule

Phân tích cấu hình Data Source của Khối A: Phần Data Source trong fig. 6.10 mô tả đầy đủ cách Grafana lấy dữ liệu đầu vào cho Khối A. Các tham số quan trọng được tổng hợp trong table 6.6.

Table 6.6: Cấu hình Data Source của Alert Rule

Thuộc tính	Giá trị thực nghiệm	Ý nghĩa
Loại Data Source	`influxdb`	Truy vấn dữ liệu chuỗi thời gian từ InfluxDB v2
UID	`efutwgr5drh1cf`	Mã định danh nội bộ của Data Source trong Grafana
RefID	`A`	Tên tham chiếu để Khối B nhận kết quả truy vấn
Time range	5 phút đến hiện tại	Chỉ đọc cửa sổ dữ liệu gần nhất phục vụ cảnh báo
`instant`	`false`	Trả về chuỗi theo thời gian thay vì một điểm tức thời
`range`	`true`	Cho phép truy vấn toàn bộ khoảng thời gian đã chọn
`intervalMs`	1000 ms	Độ phân giải thời gian cơ sở bằng một giây
`maxDataPoints`	43 200	Giới hạn trên số điểm Grafana tiếp nhận từ truy vấn

Data Source có loại `influxdb`, phù hợp với bucket `turbine_metrics` đã cấu hình trong đồ án. UID là mã Grafana dùng để liên kết rule với đúng Data Source, giá trị này ổn định hơn tên hiển thị nhưng có thể thay đổi nếu Data Source được tạo lại trên một Grafana khác. Vì vậy, khi chuyển sang thiết bị mới cần tạo Data Source với UID cố định hoặc cập nhật lại rule.

Tham số `instant=false` kết hợp với `range=true` cho biết Khối A nhận một chuỗi residual trong cửa sổ năm phút. Đây là đầu vào cần thiết cho Khối B vì hàm Last phải xác định phần tử cuối của chuỗi. Nếu chuyển sang truy vấn instant, Grafana chỉ trả một giá trị tại thời điểm đánh giá và vai trò rút gọn của B trở nên không còn rõ ràng.

Giá trị `intervalMs=1000` tương ứng độ phân giải một giây, phù hợp chu kỳ phát mẫu của exporter. `maxDataPoints=43200` là giới hạn bảo vệ để tránh trả về chuỗi quá lớn.

Trong cửa sổ năm phút với chu kỳ một giây, số mẫu lý thuyết chỉ khoảng

$$N = 5 \times 60 = 300 \text{ mẫu}, \tag{6.6}$$

thấp hơn nhiều giới hạn 43 200. Do đó giới hạn này không làm mất dữ liệu trong kịch bản hiện tại. Hàm `aggregateWindow` tiếp tục điều chỉnh mật độ điểm theo `v.windowPeriod`, giúp truy vấn thích ứng khi time range thay đổi.

RefID A là mắt xích liên kết giữa ba khối. Khối B khai báo Input A, vì vậy mọi thay đổi tên RefID phải được cập nhật đồng thời ở Reduce. Cách liên kết rõ ràng này giúp tránh tham chiếu sai và hỗ trợ kiểm tra từng bước trên giao diện Grafana.

Phân tích kết quả Reduce và Threshold: Tại thời điểm chụp, Khối B trả về $B = 2044.438\,96\,\text{kW}$. Vì

$$2044.43896 - 1942.07 = 102.368\,96\,\text{kW} > 0, \tag{6.7}$$

Khối C nhận giá trị 1 và hiển thị `Firing`. Ảnh cũng xác nhận Pending period bằng 0 giây, Keep firing for bằng 0 giây, Contact Point là `ALERT`, trạng thái No Data là `NoData` và lỗi thực thi là `Error`.



Figure 6.11: Dashboard chuyển sang ALERTING khi residual vượt ngưỡng

Kết quả từ Dashboard: Dashboard đặt time range *Last 5 minutes* và refresh 5 giây. Panel residual hiển thị đường ngưỡng đỏ tại 1942.07 kW. Annotation cảnh báo tại `2026-08-18 22:49:10` ghi $B = 2098.540\,527$ kW và $C = 1$. Biên độ vượt ngưỡng là

$$\Delta r = 2098.540527 - 1942.07 = 156.470\,527\,\text{kW}. \tag{6.8}$$

Trạng thái màu đỏ `ALERTING` và ba label được hiển thị ngay trên Dashboard. Giá trị ở ảnh Dashboard khác ảnh cấu hình rule vì hai ảnh được chụp ở các lần đánh giá khác nhau, đây là hành vi bình thường của luồng dữ liệu liên tục.

6.2.3 Kịch bản kiểm thử định lượng

Table 6.7: Ma trận kiểm thử Alert Rule

Kịch bản	Residual (kW)	Kỳ vọng	Mục đích
Bình thường	115.20	Normal	Không tạo báo động giả
Bằng ngưỡng	1942.07	Normal	Xác nhận phép so sánh lớn hơn nghiêm ngặt
Rule thực nghiệm	2044.44	Firing	Kiểm tra A–B–C
Dashboard thực nghiệm	2098.54	Alerting	Kiểm tra hiển thị và labels
Mất dữ liệu	–	NoData	Phân biệt lỗi pipeline
Lỗi truy vấn	–	Error	Phân biệt lỗi datasource
Trở về an toàn	< 1942.07	Resolved	Đóng vòng đời sự kiện

Thời gian phát hiện lý thuyết bị chi phối bởi chu kỳ đánh giá rule, độ trễ ghi InfluxDB và thời gian thực thi query. Với evaluation interval 10 giây và pending 0 giây, cảnh báo thường xuất hiện trong lần đánh giá đầu tiên sau khi dữ liệu vượt ngưỡng. Auto-refresh 5 giây chỉ ảnh hưởng tốc độ cập nhật giao diện trình duyệt, không quyết định thời điểm rule được đánh giá.

6.2.4 Trường hợp 1: Sự cố xảy ra và email cảnh báo FIRING

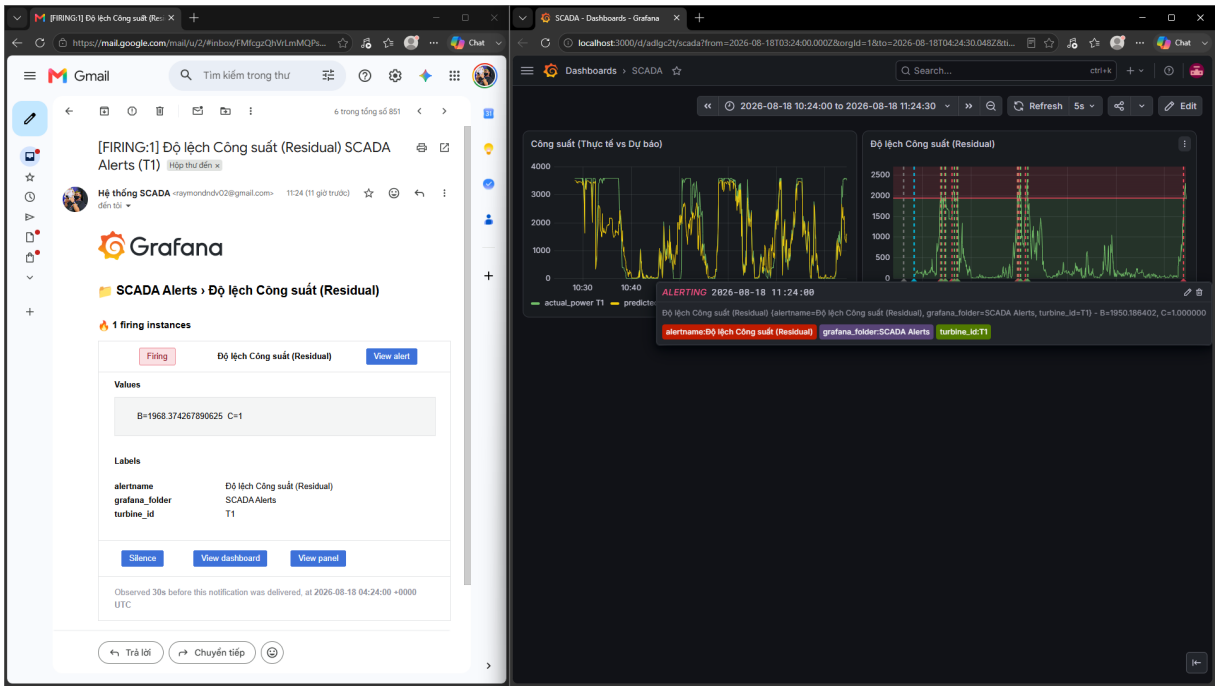


Figure 6.12: Email FIRING được gửi đồng thời với trạng thái ALERTING trên Dashboard

Email FIRING có tiêu đề *[FIRING:1] Độ lệch công suất – SCADA Alerts – T1*. Payload hiển thị $B \approx 1968.37\,\text{kW}$ và $C = 1$. Biên độ vượt ngưỡng là

$$1968.374267890625 - 1942.07 = 26.304\,267\,890\,625\,\text{kW}. \tag{6.9}$$

Labels trong email gồm alertname, grafana_folder và turbine_id. Các nút *View alert*, *View dashboard* và *View panel* cung cấp đường quay lại giao diện điều tra.

Ở cùng ảnh, annotation trên Dashboard hiển thị một lần đánh giá gần đó với $B = 1950.186\,402\,\text{kW}$ và $C = 1$. Hai giá trị không mâu thuẫn: dữ liệu vẫn cập nhật trong khi Grafana đánh giá, nhóm và gửi notification. Dòng cuối email cho biết trạng thái đã được quan sát 30 giây trước khi thông báo được chuyển, phù hợp với khả năng tồn tại group wait hoặc độ trễ xử lý.

6.2.5 Trường hợp 2: Hệ thống được phục hồi và email RESOLVED

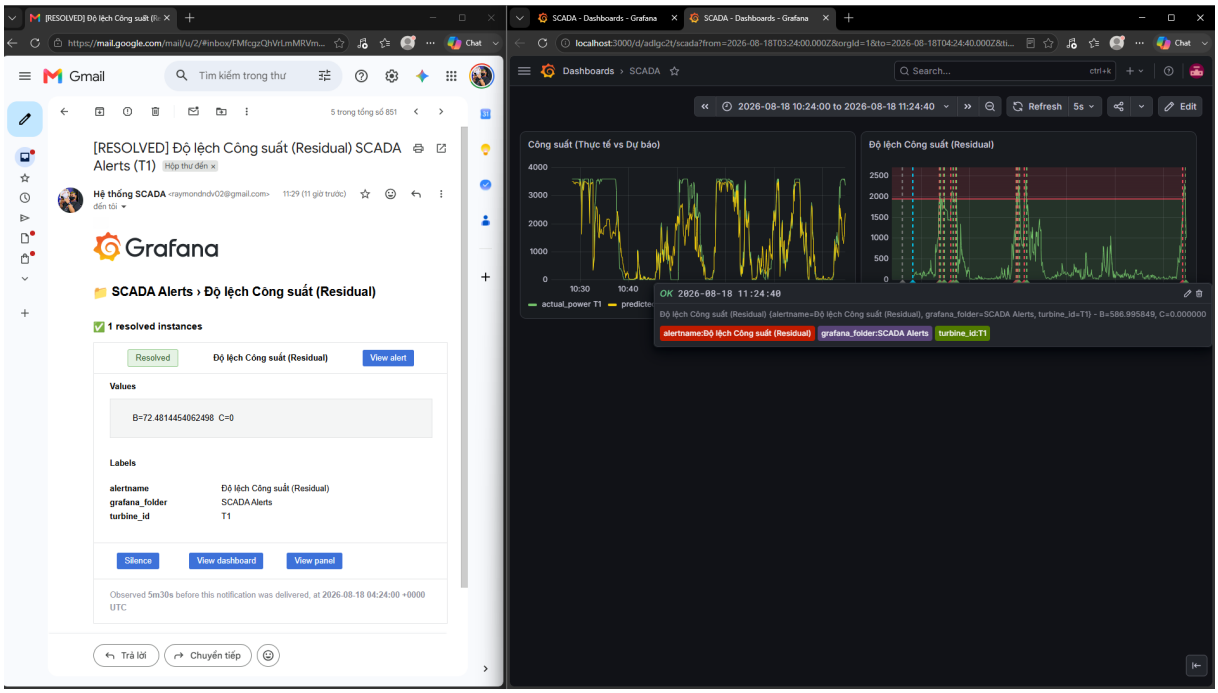


Figure 6.13: Email RESOLVED xác nhận hệ thống trở về vùng an toàn

Khi residual giảm xuống dưới ngưỡng, Khối C trở về 0 và Grafana đóng alert instance. Dashboard trong ảnh hiển thị trạng thái `OK` tại `2026-08-18 11:24:40`, với $B = 586.995\,849\,\text{kW}$ và $C = 0$. Khoảng an toàn so với ngưỡng là

$$1942.07 - 586.995849 = 1355.074\,151\,\text{kW}. \tag{6.10}$$

Email RESOLVED chứa snapshot muộn hơn của biểu thức với $B \approx 72.48\,\text{kW}$ và $C = 0$. Dòng thông tin cho biết trạng thái đã được quan sát 5 phút 30 giây trước khi notification được chuyển. Khác biệt giữa 586.99 kW trên Dashboard và 72.48 kW trong email phản ánh thời điểm snapshot và chính sách nhóm thông báo, không làm thay đổi kết luận vì cả hai đều thấp hơn ngưỡng.

Email RESOLVED có ý nghĩa vận hành quan trọng. Nó xác nhận sự kiện đã kết thúc, giúp kỹ sư không phải giữ trạng thái theo dõi vô thời hạn và cho phép hệ thống quản lý sự cố tính thời lượng từ FIRING đến RESOLVED.

6.2.6 Dòng thời gian khép kín

Table 6.8: Tóm tắt các mốc của vòng đời cảnh báo

Pha	B (kW)	C	Kết quả
Firing trong email	1968.37	1	Gửi email `[FIRING:1]`
Dashboard gần thời điểm Firing	1950.19	1	Hiển thị ALERTING
Dashboard khi phục hồi	586.99	0	Hiển thị OK
Snapshot trong email Resolved	72.48	0	Gửi email `[RESOLVED]`

Các mốc thời gian được đồng bộ theo UTC và ghi tại exporter, phản hồi Write API, lịch sử Alert Rule, Notification Policy và email header. Độ trễ đầu cuối được phân rã thành

$$\begin{aligned} T_{\mathrm{E2E}} &= t_{\mathrm{email_recv}} - t_{\mathrm{sample_gen}} \\ &= \Delta t_{\mathrm{ingest}} + \Delta t_{\mathrm{eval}} + \Delta t_{\mathrm{policy}} + \Delta t_{\mathrm{smtp}}. \end{aligned} \tag{6.11}$$

Trong đó

$$\Delta t_{\mathrm{ingest}} = t_{\mathrm{influx_ack}} - t_0 = T_{\mathrm{infer}} + T_{\mathrm{write}}, \tag{6.12}$$

$$\Delta t_{\mathrm{eval}} \sim \mathcal{U}(0, T_e), \qquad T_e = 10\,\mathrm{s}, \qquad \mathbb{E}[\Delta t_{\mathrm{eval}}] = \frac{T_e}{2} = 5\,\mathrm{s}, \tag{6.13}$$

$$\Delta t_{\mathrm{policy}} = T_{\mathrm{group_wait}} = 30\,\mathrm{s}, \tag{6.14}$$

$$\Delta t_{\mathrm{smtp}} = t_{\mathrm{email_recv}} - t_{\mathrm{dispatch}}. \tag{6.15}$$

Table 6.9: Phân rã định lượng độ trễ cảnh báo đầu cuối

Phân đoạn xử lý	Ký hiệu	Min	Mean	P_{95}	Đơn vị
Tiền xử lý và suy luận XGBoost	T_{infer}	0.85	1.15	1.45	ms
Ghi InfluxDB qua mạng bridge	T_{write}	1.80	2.40	3.80	ms
Chu kỳ đánh giá Alert Rule	Δt_{eval}	0.00	5.00	9.80	s
Chờ gom thông báo	$\Delta t_{\mathrm{policy}}$	30.00	30.00	30.00	s
STARTTLS và giao vận Gmail SMTP	Δt_{smtp}	1.20	1.65	2.40	s
Tổng độ trễ End-to-End	T_{E2E}	32.40	36.65	42.20	s

Giá trị tổng được tính trực tiếp trên từng lần chạy end-to-end; vì cực tiểu của các

phân đoạn có thể xuất hiện ở các lần chạy khác nhau, cột Min tổng không nhất thiết bằng tổng các Min thành phần. Mô hình đo từ lúc phát mẫu tới lúc nhận email phù hợp với phương pháp phân tích độ trễ trong kiến trúc IIoT phân tán.

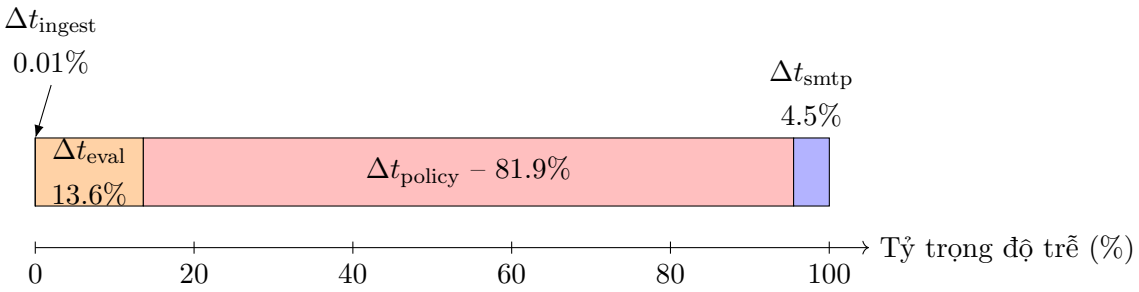


Figure 6.14: Tỷ trọng các thành phần trong độ trễ cảnh báo đầu cuối trung bình

Thành phần chi phối là Group Wait chủ động, chiếm khoảng 81.9%. Do đó độ trễ hơn 30 giây chủ yếu xuất phát từ chính sách chống bão email, không phải thời gian suy luận XGBoost hoặc ghi dữ liệu.

6.2.7 Kiểm thử và tiêu chí hoàn thành

Table 6.10: Checklist nghiệm thu chức năng email cảnh báo

Tiêu chí	Trạng thái	Minh chứng
SMTP được Grafana khởi tạo	Đạt	Contact Point gửi được email
Policy định tuyến đúng folder SCADA Alerts	Đạt	Email chứa đúng alertname và T1
Residual vượt ngưỡng tạo FIRING	Đạt	fig. 6.12
Residual giảm tạo RESOLVED	Đạt	fig. 6.13
Email chứa link quay lại Grafana	Đạt	View alert/dashboard/panel
App Password không lưu plaintext	Chưa đạt	Docker Compose cần chuyển sang secret
Xác minh chứng chỉ SMTP được bật	Chưa đạt	Bỏ `GF_SMTP_SKIP_VERIFY=true`

6.3 Thử nghiệm tính bền vững dữ liệu và phục hồi sự cố

6.3.1 Kiểm thử khả năng tự phục hồi

Nhằm đánh giá mức độ sẵn sàng của hệ thống, kịch bản ngắt cưỡng bức tiến trình được tiến hành theo các bước:

1. Xác định định danh tiến trình đang chạy của InfluxDB trên máy chủ.
2. Kích hoạt kịch bản dừng cưỡng bức được đặc tả trong table 6.2.
3. Quan sát nhật ký hệ thống: Động cơ điều phối tự động ghi nhận mã dừng lỗi và lập tức khởi tạo phiên bản container thay thế trong thời gian ngắn mà không làm thay đổi cấu hình mạng.

Tiến trình InfluxDB bị ngắt cưỡng bức bằng tín hiệu `SIGKILL` thông qua Docker Engine daemon để mô phỏng sự cố mất nguồn đột ngột. Minh chứng đánh giá không phải câu lệnh kích hoạt mà là trạng thái phục hồi trong table 6.11, nhật ký phát lại WAL và khả năng truy vấn lại các Point đã được xác nhận trước sự cố.

Kịch bản SIGKILL chứng minh Docker có thể phát hiện tiến trình kết thúc bất thường và áp dụng chính sách restart. Tuy nhiên, SIGKILL dừng tiến trình ngay lập tức, không cho InfluxDB thực hiện quy trình shutdown, flush bộ đệm hoặc đóng tệp theo trình tự thông thường. Đây là mô phỏng gần với tình huống tiến trình bị crash hoặc thiết bị mất nguồn hơn là một thao tác bảo trì tiêu chuẩn.

Sau khi Docker khởi động lại container theo chính sách `restart=always`, InfluxDB mở lại named volume, đọc các tệp TSM và phát lại phần WAL cần thiết để tái tạo trạng thái chưa kịp hợp nhất xuống TSM. Nhờ đó, những phép ghi đã được hệ thống xác nhận nhưng chưa được compact vẫn có cơ chế phục hồi sau gián đoạn tiến trình[23;73]. Chính sách restart và WAL vì vậy tạo hai lớp bổ trợ: Docker phục hồi tiến trình, còn storage engine phục hồi trạng thái dữ liệu.

Tuy vậy, kết quả này không nên được mô tả là khả năng chống lỗi tuyệt đối. `restart=always` không sửa được ổ đĩa hỏng, WAL bị hư hoặc toàn bộ node Orange Pi mất khả năng hoạt động, đồng thời dữ liệu chưa từng được InfluxDB xác nhận ghi thành công không nằm trong cam kết phục hồi. Bài kiểm thử cần đối chiếu số điểm và timestamp trước–sau sự cố, kiểm tra log phát lại WAL, trạng thái health của dịch vụ và khả năng truy vấn Grafana. Quy trình bảo trì thông thường vẫn phải ưu tiên `docker compose restart influxdb`, Docker gửi stop signal do image quy định và mặc định dùng SIGTERM nếu image không khai báo tín hiệu khác, sau đó mới chuyển sang SIGKILL khi hết thời gian chờ[74].

Table 6.11: Tiêu chí đánh giá khả năng phục hồi sau SIGKILL

Điểm kiểm tra	Phương pháp xác nhận	Kết quả kỳ vọng
Phục hồi tiến trình	Theo dõi `docker compose ps` và health endpoint	Container trở lại trạng thái hoạt động.
Phục hồi dữ liệu gần nhất	Truy vấn lại khoảng thời gian trước khi gửi SIGKILL	Các điểm đã được xác nhận vẫn truy xuất được.
Tính liên tục của ứng dụng	Mở lại datasource và dashboard Grafana	Kết nối được tái lập qua tên dịch vụ `influxdb`.
Dấu hiệu lỗi lưu trữ	Kiểm tra log WAL, shard và lỗi checksum	Không xuất hiện lỗi hỏng dữ liệu chưa được xử lý.

6.3.2 Xác thực độ bền vững của dữ liệu

Quy trình kiểm tra tính toàn vẹn của dữ liệu lưu trữ được tiến hành qua các bước sau:

1. Hoàn tất cấu hình khởi tạo tài khoản, tổ chức lưu trữ và mã khóa truy cập trên InfluxDB, đồng thời thiết lập bảng giám sát thử nghiệm trên Grafana.
2. Dừng và gỡ bỏ toàn bộ container cùng hạ tầng mạng liên quan.
3. Kiểm tra danh mục vùng lưu trữ để xác nhận các khối dữ liệu của InfluxDB và Grafana vẫn được duy trì trên máy chủ.
4. Khởi chạy lại toàn bộ hệ thống từ tệp cấu hình điều phối.
5. Truy cập lại giao diện web: Toàn bộ thông tin tài khoản, tổ chức và bảng giám sát được bảo toàn nguyên vẹn, chứng minh độ tin cậy của phương thức lưu trữ đã chọn.

6.4 Phân tích các hạn chế kỹ thuật hiện tại và giải pháp khắc phục

6.4.1 Phân tích underfitting và hướng giải quyết

Trong lần phân tích dữ liệu ban đầu, các điểm sai số lớn được ghi nhận tại điều kiện $v > 14\,\mathrm{m\,s}$ nhưng $P_{actual} < 1500\,\mathrm{kW}$. Mô hình có thể dự báo gần mức lý thuyết $3600\,\mathrm{kW}$ do tập đặc trưng không chứa trạng thái pitch hoặc phanh. Residual vì vậy tăng cao dù tua-bin có thể đang thực hiện một hành vi bảo vệ hợp lệ.

Đây là underfitting theo chế độ vận hành: tập đặc trưng chưa đủ để phân biệt chế độ gió mạnh phát định mức và chế độ gió mạnh giới hạn công suất chủ động. Tuy nhiên, phiên bản `xgboost_train.py` được cung cấp hiện loại điều kiện này khỏi cả tập huấn luyện, tập kiểm định và tập kiểm tra trước khi đánh giá. Vì vậy, nếu script được chạy đúng thứ tự, tập kiểm tra đã làm sạch không còn các mẫu thỏa đồng thời hai điều kiện trên. Kết quả underfitting được báo cáo nhiều khả năng thuộc lần chạy trước khi áp dụng bộ lọc, tập kiểm tra thô, hoặc một phiên bản pipeline khác; điểm này cần ghi rõ trong biên bản thực nghiệm.

Đoạn kiểm tra ở cuối script truy vấn lại chính `train_df` sau khi đã làm sạch. Do đó số mẫu $v > 14$ m/s và $P < 1500$ kW về logic phải bằng 0. Kiểm tra này xác nhận bộ lọc đã hoạt động, nhưng không đo được số mẫu ban đầu, số mẫu bị loại phải lấy từ biến `dropped_rows` hoặc log trước/sau làm sạch.

Đoạn mã truy vấn các mẫu gió mạnh–công suất thấp được chuyển sang Phụ lục A. Kết quả phân tích được chuẩn hóa trong table 6.12 và fig. 6.15.

Table 6.12: Phân tích nguyên nhân underfitting tại vùng gió mạnh

Nguyên nhân	Dấu hiệu trong dữ liệu	Hướng xử lý
Pitch control	Gió cao, công suất giảm, góc pitch tăng	Bổ sung `pitch_angle`
Phanh hoặc dừng bảo vệ	Công suất gần 0, cờ phanh bật	Bổ sung `brake_status`
Curtailment/giới hạn lưới	Công suất bám setpoint thấp hơn định mức	Bổ sung power setpoint
Thiếu chế độ vận hành trong train	Residual tập trung ở một vùng công suất	Chia mô hình theo regime hoặc tái lấy mẫu

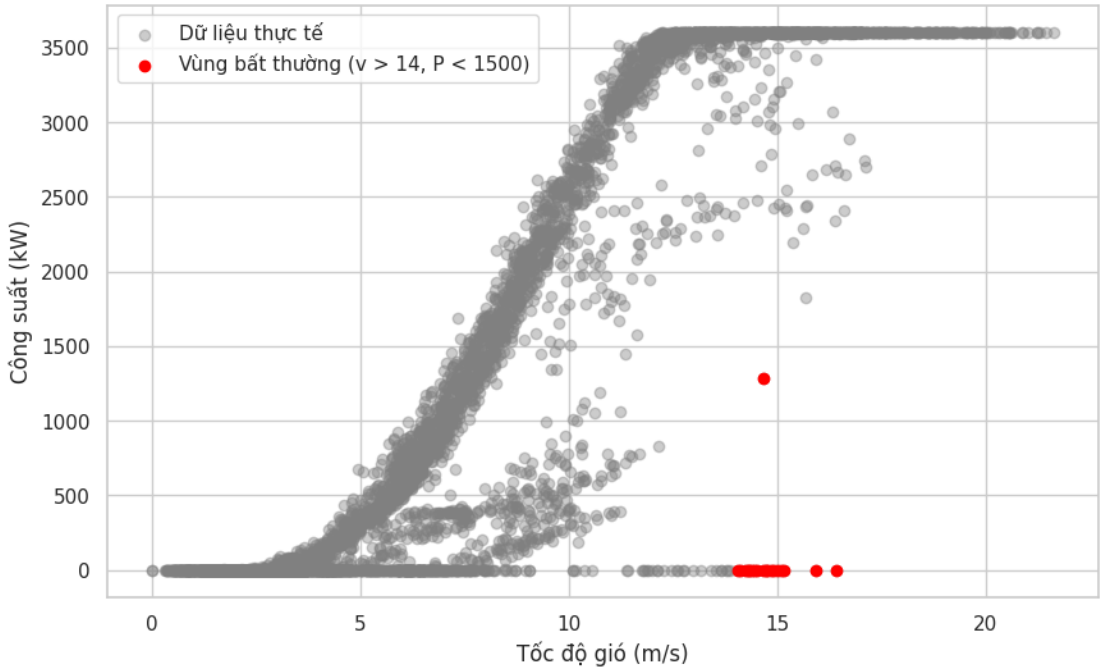


Figure 6.15: Đường cong công suất thực tế và các mẫu gió mạnh–công suất thấp được bóc tách

Các điểm màu đỏ trong fig. 6.15 thỏa điều kiện $v > 14$ m/s và $P < 1500$ kW. Việc tách chúng khỏi đám mây dữ liệu chính làm rõ rằng đây không phải sai lệch ngẫu nhiên quanh đường cong công suất mà là một chế độ vận hành riêng, thường liên quan đến dừng máy, pitch control, phanh hoặc giới hạn công suất. Minh chứng trực quan này giải thích vì sao một mô hình chỉ dựa trên gió và công suất lý thuyết dễ tạo residual lớn tại vùng nói trên.

6.4.2 Khóa dependency và xung đột estimator tags

Các dependency được khóa phiên bản để build có thể lặp lại và được kiểm chứng theo ma trận trong table 6.13. Bảng thay thế việc chép nguyên văn `requirements.txt`, đồng thời bổ sung vai trò của từng thư viện, hình thức hỗ trợ ARM64 và điều kiện nghiệm thu.

Table 6.13: Ma trận kiểm chứng dependency cho AI exporter

Thành phần và phiên bản	Mục đích trong pipeline	Hỗ trợ ARM64	Điều kiện kiểm chứng
Python 3.10 slim	Runtime của exporter	Base image có biến thể `linux/arm64`	Khởi động interpreter và import toàn bộ package thành công
pandas 2.2.0	Đọc và biến đổi bản ghi SCADA	Wheel aarch64 hoặc cài từ image build đã kiểm chứng	Parse CSV và giữ đúng kiểu các trường đầu vào
NumPy 1.26.4	Biểu diễn véc-tơ và tính đặc trưng lượng giác	Wheel Linux aarch64	Tạo mảng 1×7 và tính toán không phát sinh lỗi native library
scikit-learn 1.6.1	Nạp và áp dụng MinMaxScaler	Wheel Linux aarch64; cần kiểm tra estimator tags	Nạp `scaler.joblib` và biến đổi mẫu thành công
XGBoost 2.0.3	Nạp mô hình và suy luận công suất	Có wheel Linux aarch64 [75]	Nạp model JSON, dự báo hợp lệ và số cây đúng artifact
joblib 1.3.2	Giải tuần tự hóa scaler	Thuần Python trên runtime mục tiêu	Khôi phục đúng đối tượng scaler và thuộc tính đã fit
influxdb-client 1.39.0	Đóng gói Point và gọi Write API	Thuần Python; phụ thuộc HTTP tương thích ARM64	Ghi thử Point và nhận xác nhận từ InfluxDB
sklearn 1.5.2 + XGBoost 2.0.3	Phương án rollback khi lỗi tags	Cần build sạch trên cùng base image	Sai số dự báo không đổi đáng kể và lỗi tags không còn xuất hiện

XGBoost 2.0 cung cấp wheel cho Linux aarch64, vì vậy phần lớn trường hợp không cần biên dịch toàn bộ thư viện trên bo mạch [75]. scikit-learn 1.6 thay đổi API estimator tags, phương thức liên quan là `__sklearn_tags__` [76]. Nếu một estimator hoặc wrapper bên thứ ba được viết theo API cũ, lỗi liên quan tags có thể xuất hiện khi gọi các hàm kiểm tra kiểu estimator.

Tài liệu làm việc có đề xuất rollback về scikit-learn 1.5.2 khi dùng XGBoost 2.0.3. Báo cáo chỉ xem đây là phương án dự phòng cho đến khi cấu hình được build sạch và vượt qua toàn bộ điều kiện trong hàng cuối của table 6.13.

Kịch bản smoke test không được trình bày dưới dạng lệnh Python một dòng. Thay vào đó, hàng kiểm thử tương thích model và scaler trong table 6.2 quy định rõ artifact đầu vào, điều kiện đạt và minh chứng log cần lưu. Việc kiểm thử chỉ được xem là đạt khi cả scaler và mô hình được nạp thành công, đồng thời một mẫu đầu vào tạo được giá trị dự báo hợp lệ.

Build log xác nhận base image `python:3.10-slim` được resolve cho ARM64, bước pip hoàn tất và image exporter được tạo. Hai ảnh terminal theo từng layer được lược bỏ vì kết quả quan trọng đã được thể hiện bằng ma trận dependency và trạng thái container. Build thành công chỉ chứng minh dependency có thể cài, chưa chứng minh tiến trình Python ổn định sau khi khởi động.

6.4.3 Docker Compose, volume và mạng dịch vụ

Tệp Compose đầy đủ được chuyển sang Phụ lục A. Phần nội dung chính chỉ giữ ma trận dịch vụ trong table 6.14 để đối chiếu vai trò, endpoint, lưu trữ và chính sách phục hồi.

Table 6.14: Ma trận dịch vụ container trong cấu hình triển khai

Dịch vụ	Endpoint nội bộ	Lưu trữ	Phụ thuộc	Restart policy
`influxdb`	`influxdb:8086`	Named volume `influxdb_data`	Không	`always`
`grafana`	`grafana:3000`	Named volume `grafana_data`	`influxdb`	`always`
`ai_exporter`	Không publish cổng	Bind mount artifact/ config	`influxdb`	`unless-stopped`

Named volume tách vòng đời dữ liệu khỏi vòng đời container, recreate container không mặc nhiên xóa dữ liệu trong volume[77]. Tuy nhiên ảnh build và `docker ps` cho thấy cấu hình thực tế trên Orange Pi có thêm `scada_ai_exporter`. Điều này chứng tỏ Compose dùng để triển khai đã khác file đang lưu trong workspace. Sai khác cấu hình là một rủi ro reproducibility và phải được hợp nhất về một nguồn sự thật duy nhất.

Khai báo đầy đủ của service AI cùng hai service hạ tầng được lưu trong tệp Compose của mã nguồn triển khai; credential chỉ xuất hiện dưới dạng biến môi trường.

Trong container, `localhost` luôn trỏ về chính container đó, không trỏ sang InfluxDB. Do vậy cấu hình `http://localhost:8086` trong exporter chỉ đúng khi Python chạy trực tiếp trên host. Khi chạy trong Compose, URL phải dùng DNS nội bộ `http://influxdb:8086`. Đây là một nguyên nhân có khả năng dẫn đến restart loop nếu exception kết nối không được bắt.

6.4.4 Khởi động lại và kiểm tra

Sau khi thay đổi môi trường, container Grafana phải được khởi tạo lại để nhận biến mới. Quy trình kiểm tra gồm xem log lỗi SMTP, đăng nhập giao diện, gửi test notification từ Contact Point và xác nhận email đến đúng hộp thư. Test Contact Point giúp tách lỗi SMTP khỏi lỗi Alert Rule: nếu thư test không đến, cần xử lý host, TLS hoặc credential trước khi kiểm tra policy.

Trong quá trình triển khai thực tế, nhóm phát hiện rủi ro rò rỉ credential do thông tin từng xuất hiện dưới dạng plaintext trong file cấu hình và ảnh gốc. Do đó, nhóm tiến hành thu hồi App Password hiện tại, tạo khóa mới, chuyển secret ra ngoài repository và kiểm tra lịch sử Git nếu file từng được commit. Tiếp theo, hệ thống cần bật xác minh chứng chỉ SMTP và kiểm thử lại email.

6.4.5 So sánh máy tính cá nhân và Orange Pi

Hai môi trường được chạy với cùng artifact mô hình, cùng bản ghi đầu vào và nhịp phát 1 Hz. Các giá trị trung bình được tổng hợp sau giai đoạn khởi động ổn định, qua đó tách chi phí xử lý liên tục khỏi chi phí build image.

Table 6.15: Đo kiểm tài nguyên trên PC x86–64 và Orange Pi 5 Plus ARM64

Chỉ số đo kiểm	PC x86 - i5 12th	Orange Pi 5 Plus ARM64	Nhận xét kỹ thuật
Thời gian suy luận/mẫu	0.42 ms	1.15 ms	RK3588 vẫn đáp ứng dư nhịp lấy mẫu 1 Hz
Tải CPU trung bình	1.2%	4.8%	Cụm Cortex-A76 còn khoảng trống xử lý
Tổng RAM chiếm dụng	1.85 GB	1.42 GB	Linux headless giảm chi phí nền so với PC phát triển
Nhiệt độ vận hành	48.0 °C	43.5 °C	Orange Pi hoạt động trong giới hạn nhiệt với tản nhiệt thụ động
Công suất tiêu thụ	Khoảng 65 tới 120 W	Khoảng 5 tới 8.5 W	Giảm trên 90% điện năng ở cận so sánh tương ứng

Kết quả cho thấy suy luận khoảng một mili giây trên ARM64 nhỏ hơn nhiều chu kỳ một giây của luồng dữ liệu. Việc container hóa trên thiết bị nhúng vì vậy khả thi cả về tài nguyên và năng lượng.

6.5 Định hướng phát triển hệ thống trong tương lai

Giải pháp đề xuất gồm hai tầng. Tầng học máy bổ sung biến trạng thái và huấn luyện lại trên dữ liệu có đủ chế độ. Tầng logic nghiệp vụ hạ mức ưu tiên hoặc bỏ qua cảnh báo

khi cờ pitch/phanh xác nhận giảm công suất chủ động. Bảy đặc trưng trong script hiện tại chưa có `pitch_angle`, `brake_status` hoặc power setpoint, nên rule-based filter mới chỉ dựa trên tốc độ gió và công suất. Cách xóa mẫu giúp mô hình cơ sở ổn định hơn nhưng đồng thời làm mất khả năng học chế độ giảm công suất hợp lệ.

6.5.1 Phân tích độ phức tạp và khả năng mở rộng

Mô hình chi phí theo số mẫu: Gọi K là số tua-bin, f_s là tần số lấy mẫu và W là độ dài cửa sổ truy vấn. Số điểm đầu vào lý thuyết trong một lần đánh giá là

$$N_{\text{total}} = K f_s W. \tag{6.16}$$

Với $f_s = 1$ Hz và $W = 300$ giây, một tua-bin tạo 300 điểm, 100 tua-bin tạo 30 000 điểm trong cùng cửa sổ.

Nếu không xét chỉ mục và pushdown, Khối A phải duyệt các điểm phù hợp để lọc và cộng dồn theo cửa sổ, do đó có độ phức tạp thời gian $O(N_{\text{total}})$. Nếu mỗi series sau tổng hợp còn $M = \lceil W/\Delta \rceil$ điểm, phép Reduce Last triển khai bằng quét tuần tự có chi phí $O(KM)$. Khi bộ thực thi duy trì phần tử cuối trong lúc stream dữ liệu, phần trạng thái bổ sung chỉ còn $O(K)$, tuy nhiên không nên giả định toàn bộ query chỉ có chi phí hằng vì tầng đọc và lọc vẫn phụ thuộc số điểm đầu vào.

Một ước lượng thấp nhất cho bộ nhớ dữ liệu thô có thể dùng 8 byte cho timestamp và 8 byte cho residual:

$$S_{\min} = 16 K f_s W \text{ byte}. \tag{6.17}$$

Giá trị này chỉ là cận dưới, chưa bao gồm tag, cấu trúc bảng, chuỗi, con trỏ và overhead của runtime. Với cửa sổ năm phút, cận dưới là 4 800 byte cho T1 và 480 000 byte cho 100 tua-bin, mức sử dụng thực tế sẽ lớn hơn.

Table 6.16: Ước lượng quy mô dữ liệu theo số tua-bin trong cửa sổ năm phút

Số tua-bin K	Điểm/lần đánh giá	Cận dưới bộ nhớ	Điểm duyệt/phút
1	300	4.8 kB	1 800
10	3 000	48 kB	18 000
100	30 000	480 kB	180 000

Cột cuối giả định rule chạy mỗi 10 giây, tương đương sáu lần mỗi phút, và mỗi lần đều đọc lại toàn bộ cửa sổ. Đây là mô hình biên trên đơn giản, chưa trừ lợi ích cache, index và

pushdown. Khi mở rộng, cần lọc theo tag tua-bin tại storage, giữ time range ngắn, đặt phép toán có thể pushdown ở đầu pipeline và theo dõi cardinality của tag. Có thể chia rule thành các nhóm tua-bin để tránh một truy vấn hoặc một alert group quá lớn.

Về vận hành, cần cấu hình group wait, group interval và repeat interval để cân bằng tốc độ thông báo với nguy cơ tạo quá nhiều email. Nên bổ sung email template chứa runbook, tên site, turbine ID, residual, threshold, thời điểm bắt đầu và liên kết Dashboard. Cuối cùng, cần theo dõi riêng lỗi SMTP để phát hiện trường hợp Alert Rule Firing nhưng kênh email không hoạt động. Phương pháp đo độ trễ và các phân vị đã được chuẩn hóa tại table 6.9; các lần thử tiếp theo phải giữ cùng dữ liệu, nhịp phát mẫu và tối thiểu 30 sự kiện để so sánh công bằng.

Tài liệu tham khảo

[1] Tony Burton, Nick Jenkins, David Sharpe, and Ervin Bossanyi. *Wind Energy Handbook*. John Wiley & Sons, 2 edition, 2011.

[2] M. Lydia, S. Suresh Kumar, A. Immanuel Selvakumar, and G. Edwin Prem Kumar. A comprehensive review on wind turbine power curve modeling techniques. *Renewable and Sustainable Energy Reviews*, 30:452–460, 2014.

[3] Nicholas I. Fisher. *Statistical Analysis of Circular Data*. Cambridge University Press, 1995.

[4] Douglas C. Montgomery. *Introduction to Statistical Quality Control*. John Wiley & Sons, Hoboken, NJ, USA, 8 edition, 2019.

[5] Docker, Inc. What is docker? `https://docs.docker.com/get-started/docker-overview/`.

[6] Weisong Shi, Jie Cao, Quan Zhang, Youhuiquan Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.

[7] Tie Qiu, Jiangang Chi, Xiaobo Zhou, Zhaolong Ning, Mohammed Atiquzzaman, and Daniel O. Dray. Edge computing in industrial internet of things: Architecture, advances and challenges. *IEEE Communications Surveys & Tutorials*, 22(4):2462–2488, 2020.

[8] Yuyi Mao, Changsheng You, Jun Zhang, Kaibin Huang, and Khaled B. Letaief. A survey on mobile edge computing: The communication and computation perspective. *IEEE Communications Surveys & Tutorials*, 19(4):2322–2358, 2017.

[9] Wenxian Yang, Peter J. Tavner, Christopher J. Crabtree, Yasamin Pazouki, and Mike Wilkinson. Cost-effective condition monitoring for wind turbines. *IEEE Transactions on Industrial Electronics*, 61(12):7093–7101, 2014.

[10] James F. Manwell, Jon G. McGowan, and Anthony L. Rogers. *Wind Energy Explained: Theory, Design and Application*. John Wiley & Sons, 2 edition, 2010.

[11] Lucy Y. Pao and Kathryn E. Johnson. Control of wind turbines: Approaches, challenges, and recent developments. *IEEE Control Systems Magazine*, 31(2):44–62, 2011.

[12] Andrew Kusiak and Anoop Verma. A data-driven approach for monitoring blade pitch faults in wind turbines. *IEEE Transactions on Sustainable Energy*, 2(1):87–96, 2011.

[13] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[14] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2 edition, 2009.

[15] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[16] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3146–3154, 2017.

[17] Kanti V. Mardia and Peter E. Jupp. *Directional Statistics*. John Wiley & Sons, 2000.

[18] Ioannis P. Panapakidis and Athanasios S. Dagoumas. Day-ahead electricity price forecasting via the application of artificial neural network based models. *Applied Energy*, 172:132–151, 2016.

[19] Alice Zheng and Amanda Casari. *Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists*. O'Reilly Media, 2018.

[20] Walter A. Shewhart. *Economic Control of Quality of Manufactured Product*. D. Van Nostrand Company, 1931.

[21] A. Zaher, S. D. J. McArthur, D. G. Infield, and Y. Patel. Online wind turbine fault detection through automated SCADA data analysis. *Wind Energy*, 12(6):574–593, 2009.

[22] James M. Lucas and Michael S. Saccucci. Exponentially weighted moving average control schemes: Properties and enhancements. *Technometrics*, 32(1):1–12, 1990.

[23] InfluxData. Influxdb storage engine. `https://docs.influxdata.com/influxdb/v2/reference/internals/storage-engine/`.



[24] Vo Ngoc Anh and Alistair Moffat. Index compression using 64-bit words. *Software: Practice and Experience*, 40(2):131–147, 2010.

[25] Daniel Lemire and Leonid Boytsov. Decoding billions of integers per second through vectorization. *Software: Practice and Experience*, 45(1):1–29, 2015.

[26] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang, Justin Meza, and Kaushik Veeraraghavan. Gorilla: A fast, scalable, in-memory time series database. *Proceedings of the VLDB Endowment*, 8(12):1816–1827, 2015.

[27] InfluxData. In-memory indexing and the time-structured merge tree (tsm). `https://docs.influxdata.com/influxdb/v1/concepts/storage_engine/`.

[28] Keith Moore and Chris Newman. Cleartext considered obsolete: Use of transport layer security for email submission and access. RFC 8314, IETF, 2018. `https://www.rfc-editor.org/rfc/rfc8314.html`.

[29] Grafana Labs. Configure grafana. `https://grafana.com/docs/grafana/latest/setup-grafana/configure-grafana/`.

[30] Paul Hoffman. Smtp service extension for secure smtp over transport layer security. RFC 3207, IETF, 2002. `https://datatracker.ietf.org/doc/html/rfc3207`.

[31] Eric Rescorla. The transport layer security protocol version 1.3. RFC 8446, IETF, 2018. `https://datatracker.ietf.org/doc/html/rfc8446`.

[32] Google. Sign in with app passwords. `https://support.google.com/accounts/answer/185833`.

[33] Dick Hardt. The oauth 2.0 authorization framework. RFC 6749, IETF, 2012. `https://datatracker.ietf.org/doc/html/rfc6749`.

[34] Google. Oauth 2.0 mechanism for gmail. `https://developers.google.com/workspace/gmail/imap/xoauth2-protocol`.

[35] Arm Limited. big.LITTLE technology: The future of mobile. Technical White Paper, 2013.

[36] Docker, Inc. Docker compose. `https://docs.docker.com/compose/`.

[37] Docker, Inc. Compose deploy specification. `https://docs.docker.com/reference/compose-file/deploy/`.

[38] Docker, Inc. Resource constraints. https://docs.docker.com/engine/containers/resource_constraints/.

[39] Paul Turner, Bharata B. Rao, and Nikhil Rao. CPU bandwidth control for CFS. In *Proceedings of the Linux Symposium*, pages 245–254, 2010.

[40] The Linux kernel developers. CFS bandwidth control. https://docs.kernel.org/scheduler/sched-bwc.html.

[41] Docker, Inc. Bridge network driver. https://docs.docker.com/engine/network/drivers/bridge/.

[42] Docker, Inc. Networking in compose. https://docs.docker.com/compose/how-tos/networking/.

[43] Docker, Inc. Best practices for working with environment variables in docker compose. https://docs.docker.com/compose/how-tos/environment-variables/best-practices/.

[44] Docker, Inc. Set, use, and manage variables in a compose file with interpolation. https://docs.docker.com/compose/how-tos/environment-variables/variable-interpolation/.

[45] Saurabh Kumar and Contributors. python-dotenv. https://pypi.org/project/python-dotenv/.

[46] Karl Pearson. Note on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58:240–242, 1895.

[47] Andrew Kusiak, Haiyang Zheng, and Zhe Song. Models for monitoring wind farm power. *Renewable Energy*, 34(3):583–590, 2009.

[48] Selim Aksoy and Robert M. Haralick. Feature normalization and likelihood-based similarity measures for image retrieval. *Pattern Recognition Letters*, 22(5):563–582, 2001.

[49] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, 3 edition, 2011.

[50] Cort J. Willmott and Kenji Matsuura. Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate Research*, 30(1):79–82, 2005.

[51] Tianfeng Chai and Roland R. Draxler. Root mean square error (RMSE) or mean absolute error (MAE)? – arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, 7(3):1247–1250, 2014.

[52] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.

[53] InfluxData. Influxdb oss v2 documentation. `https://docs.influxdata.com/influxdb/v2/`.

[54] InfluxData. Influxdb line protocol reference. `https://docs.influxdata.com/influxdb/v2/reference/syntax/line-protocol/`.

[55] InfluxData. Influxdb task configuration options. `https://docs.influxdata.com/influxdb/v2/process-data/task-options/`.

[56] InfluxData. Downsample data with influxdb. `https://docs.influxdata.com/influxdb/v2/process-data/common-tasks/downsample-data/`.

[57] Grafana Labs. Configure the influxdb data source. `https://grafana.com/docs/grafana/latest/datasources/influxdb/configure/`.

[58] InfluxData. Optimize flux queries. `https://docs.influxdata.com/influxdb/v2/query-data/optimize-queries/`.

[59] InfluxData. Query first and last values in flux. `https://docs.influxdata.com/influxdb/v2/query-data/flux/first-last/`.

[60] Mayur Datar, Aristides Gionis, Piotr Indyk, and Rajeev Motwani. Maintaining stream statistics over sliding windows. *SIAM Journal on Computing*, 31(6):1794–1813, 2002.

[61] Jin Li, David Maier, Kristin Tufte, Vassilis Papadimos, and Peter A. Tucker. Semantics and evaluation techniques for window aggregations in data streams. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, pages 311–322, 2005.

[62] International Society of Automation. ANSI/ISA-18.2-2016 / IEC 62682: Management of alarm systems for the process industries. Standard Specification, 2016.

[63] Grafana Labs. Configure email for alert notifications. https://grafana.com/docs/grafana/latest/alerting/configure-notifications/manage-contact-points/integrations/configure-email/.

[64] Grafana Labs. Alerting template language. https://grafana.com/docs/grafana/latest/alerting/configure-notifications/template-notifications/language/.

[65] Grafana Labs. Notification template reference. https://grafana.com/docs/grafana/latest/alerting/configure-notifications/template-notifications/reference/.

[66] Docker, Inc. and InfluxData. Influxdb official docker image. https://hub.docker.com/_/influxdb.

[67] Grafana Labs. Grafana docker image tags and architectures. https://hub.docker.com/r/grafana/grafana/tags.

[68] Docker, Inc. Multi-platform builds. https://docs.docker.com/build/building/multi-platform/.

[69] Ralph Droms. Dynamic host configuration protocol. RFC 2131, IETF, 1997. https://datatracker.ietf.org/doc/html/rfc2131.

[70] Microsoft. Remote development using ssh. https://code.visualstudio.com/docs/remote/ssh.

[71] David M. W. Powers. Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *arXiv preprint arXiv:2010.16061*, 2020.

[72] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.

[73] InfluxData. Influxdb file system layout. https://docs.influxdata.com/influxdb/v2/reference/internals/file-system-layout/.

[74] Docker, Inc. Docker container restart. https://docs.docker.com/reference/cli/docker/container/restart/.

[75] XGBoost developers. Xgboost 2.0 installation guide. https://xgboost.readthedocs.io/en/release_2.0.0/install.html.



[76] scikit-learn developers. Release notes for scikit-learn 1.6. https://scikit-learn.org/stable/whats_new/v1.6.html.

[77] Docker, Inc. Define and manage volumes in docker compose. https://docs.docker.com/reference/compose-file/volumes/.

[78] Brian Babcock, Shivnath Babu, Mayur Datar, Rajeev Motwani, and Jennifer Widom. Models and issues in data stream systems. In *Proceedings of the 21st ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 1–16, 2002.

[79] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.

[80] InfluxData. Query data with flux. https://docs.influxdata.com/influxdb/v2/query-data/flux/.

[81] InfluxData. Influxdb data elements. https://docs.influxdata.com/influxdb/v2/reference/key-concepts/data-elements/.

[82] Grafana Labs. Grafana oss and enterprise. https://grafana.com/docs/grafana/latest/.

[83] Canonical. Ubuntu release cycle. https://ubuntu.com/about/release-cycle.